

Cuckoo Hashing with Bounded Displacement for Exact-Match Flow Tables

Soumyajit Kolay

School of Computer Engineering

KIIT Deemed University

Bhubaneswar, India

2650026@kiit.ac.in

Abstract—Exact-match dictionaries are fundamental components of high-throughput packet-processing systems, where each arriving packet may require a membership test, state lookup, or flow-table update under a tightly constrained service budget. Cuckoo hashing is attractive in this setting because membership queries inspect only a small, fixed number of candidate locations, while the complexity associated with collision resolution is moved primarily into insertion. This separation is beneficial when reads dominate writes, but it creates an important systems problem: a collision chain can become substantially more expensive than a normal lookup, and a failed placement can require rebuilding or otherwise reorganizing a large portion of the table.

This manuscript studies a bounded-displacement variant of d -ary cuckoo hashing in which the displacement process is explicitly capped by a constant parameter, denoted by `MaxKicks`, and an auxiliary stash is used as an overflow mechanism for entries that are not placed within the prescribed displacement budget. The central design objective is not to claim that bounded displacement eliminates insertion failures, but to make the amount of work performed by the primary displacement path independent of the table size while preserving a bounded candidate set for lookup. The supplied C++ benchmark harness implements classical two-way cuckoo hashing, linear probing, chained hashing, and the proposed bounded-displacement structure, and it also contains a sensitivity experiment over several displacement bounds. No numerical performance results are asserted in this revised manuscript unless they are supported by an explicitly recorded benchmark execution.

The present version therefore separates established theoretical properties, implementation-defined guarantees, and empirical observations. In particular, it does not identify deterministic constant-time successful insertion as an established consequence of a bounded kick limit, and it does not describe stash recovery as a global hash-function rehash. The remainder of the manuscript develops the model, implementation, analysis, and experimental protocol needed to determine the actual trade-off between displacement work, stash occupancy, insertion failure, and lookup cost.

Index Terms—Exact-Match Flow Tables, Cuckoo Hashing, Bounded Displacement, d -ary Hashing, Stash, Deterministic Latency, Network Data Plane, Hash Table Performance

I. INTRODUCTION

A. Exact-Match Dictionaries as Time-Constrained Data Structures

A dynamic exact-match dictionary maps a key to associated state and supports, at minimum, membership queries and updates. In a networking system, the key may encode a packet-flow identifier, while the associated state may represent

forwarding information, connection state, policy metadata, counters, expiration information, or an index into a larger state store. The computational role of the dictionary is consequently simple to state but difficult to implement at scale: given a key x , the system must determine whether x is currently resident and, when it is, retrieve its associated state without introducing an execution-time tail that is inconsistent with the surrounding packet-processing pipeline.

The distinction between average performance and bounded operation cost is particularly important in network processing. A data structure can exhibit excellent mean throughput while occasionally executing a much longer sequence of dependent memory operations. Such an outlier may be irrelevant for an offline batch workload yet important for a packet-processing pipeline in which service time directly affects queue occupancy, batching behavior, memory-level parallelism, or deadline compliance. Accordingly, exact-match table design is not exclusively a matter of minimizing expected instruction count. It is also a problem of controlling the distribution of memory references and update-path work under the load and key-distribution regimes that the deployment is expected to encounter.

This observation motivates a careful distinction between three quantities that are frequently conflated. First, *lookup complexity* describes the number of candidate locations examined by a query after the table has been constructed. Second, *insertion complexity* describes the amount of work needed to place a new key while preserving the placement invariant of all previously stored keys. Third, *recovery complexity* describes what the implementation does after the ordinary insertion path cannot complete within its prescribed budget. A data structure may guarantee a small number of lookup probes while having a comparatively expensive update path. Cuckoo hashing is a canonical example of this separation [1].

For a networking-oriented data structure, the third quantity is not an implementation detail that can simply be omitted from the analysis. If recovery invokes a table-wide rebuild, then a rare insertion can touch an amount of memory proportional to the table population. If recovery is incremental, the corresponding cost can instead be bounded by the size of a small auxiliary structure, but only if the recovery procedure itself is correctly specified and its success conditions are understood. The present work is concerned with this bound-

ary between bounded normal insertion work and exceptional recovery work.

B. The Temporal Interpretation of Line Rate

Networking terminology often uses the phrase *line rate* to denote the sustained packet throughput supported by a link or pipeline. The phrase should not be interpreted as a universal bound on application-level execution time, because the available computation budget depends on packet size, link framing, batching, pipeline depth, implementation technology, memory hierarchy, and other factors. For example, a 100 Gb/s Ethernet interface does not imply that an arbitrary software routine has exactly one 6.72 ns computational budget per packet. A 64-byte minimum Ethernet frame occupies 512 transmission bits, corresponding to 5.12 ns of serialization at 100 Gb/s; additional Ethernet preamble and inter-frame gap time contribute to the wire-time interval between successive frames. The appropriate computational budget therefore depends on which layer of the processing pipeline is being modeled.

This distinction is important because unsupported numerical timing statements can make an otherwise sound algorithmic argument appear more precise than it is. In a journal submission, the appropriate practice is to state the abstract systems requirement first and then quantify it only after the complete physical and software assumptions have been specified. The present manuscript consequently uses *line-rate constraint* as a systems requirement rather than assigning a universal nanosecond bound to every implementation.

The same principle applies to data-structure complexity. An $O(1)$ operation in the RAM model is not synonymous with a fixed nanosecond latency on an out-of-order processor, nor does a fixed bound on the number of logical probes automatically imply a fixed bound on wall-clock time. Logical probe count is nevertheless useful because it provides a machine-independent abstraction from which implementation-specific effects such as cache residency, memory-bank conflicts, SIMD layout, prefetching, and instruction-level parallelism can subsequently be studied.

C. Why Cuckoo Hashing Remains Relevant

Cuckoo hashing was introduced by Pagh and Rodler as an open-addressed dictionary in which a key is assigned a small number of candidate locations and is stored in one of them [1]. In the standard two-choice formulation, two tables are maintained and a successful lookup examines at most one location in each table. This property is qualitatively different from linear probing, where the number of cells inspected by a lookup can grow with local clustering. The basic cuckoo construction therefore moves much of the collision-resolution burden from queries to updates.

That trade-off is attractive whenever query volume is much larger than update volume. Network state tables commonly exhibit exactly this asymmetry: a flow may be queried once per packet while the corresponding entry is installed, refreshed, or deleted far less frequently. A data structure that performs

a small fixed number of candidate checks for every query can therefore be worthwhile even when insertion is more complex than a conventional open-addressed table.

The attraction should not, however, be summarized as the claim that cuckoo hashing simply has constant-time operations in all circumstances. The classical worst-case constant-time statement concerns lookup and, in the usual formulation, deletion. The insertion process can involve repeated eviction of resident keys. The original analysis uses a bounded insertion search and resorts to rebuilding when the placement attempt does not terminate within the allowed number of iterations [1]. Hence, the central algorithmic question is not whether cuckoo hashing has bounded lookup; that property is established. The relevant systems question is how update-path work is controlled when the table is heavily occupied or when the random placement graph contains structures that are unfavorable to the chosen insertion heuristic.

D. Collision Resolution in the Comparator Structures

Two conventional collision-resolution mechanisms provide useful baselines for understanding why the displacement problem is interesting. In separate chaining, keys hashing to the same bucket are stored together, commonly in dynamically allocated nodes or contiguous container storage. The logical lookup work is proportional to the length of the corresponding bucket representation. Under a uniform hashing model with moderate load, this can be acceptable. However, the access pattern may involve pointer chasing or repeated dependent memory accesses, and the cost of a negative lookup is directly related to the amount of data that must be inspected before the absence of a key can be established.

Linear probing instead keeps the table in a flat array and scans successive positions after a collision. Its primary advantage is spatial locality: a search proceeds through contiguous slots, making the structure amenable to cache-based implementations. Its primary disadvantage is clustering. Once a contiguous run of occupied positions forms, insertions and unsuccessful searches can traverse the resulting cluster. The expected behavior is therefore strongly dependent on the load factor and the assumptions on the key-to-slot mapping [2]. Near capacity, the tail of the probe-length distribution is the relevant concern rather than the average alone.

These baselines also illustrate a methodological point. A fair comparison must define the operation being measured before interpreting a timing number. A lookup-probe count in chained hashing is a count of entries examined in a bucket; a probe in linear probing is a slot examined along a contiguous sequence; and a cuckoo lookup examines a prescribed set of candidate locations. These are not identical units of work, and they should not be reported as though they represent the same hardware cost. For this reason, the revised benchmark distinguishes algorithm-specific logical measurements from wall-clock measurements and avoids claiming a cross-structure timing ratio unless the measurement procedure supports it.

E. Classical Cuckoo Insertion and Its Recovery Path

The standard two-choice cuckoo insertion process is conceptually simple. If one candidate location is occupied, the incoming key may replace the resident key, causing the displaced key to continue to its alternate location. The process can repeat for a sequence of resident keys. When an empty location is encountered, the current insertion terminates. If the sequence becomes cyclic or exceeds the permitted search budget, the current hash functions may be discarded and a fresh placement attempted.

The original Pagh–Rodler formulation uses an iteration limit that grows logarithmically with the table size under its chosen parameterization [1]. The existence of such a bound is important because it prevents an insertion attempt from wandering indefinitely. It does not imply that the cost of a classical insertion is independent of table size. Rather, it bounds the search sufficiently to obtain the desired probability of successful construction or to trigger the recovery mechanism. Subsequent analyses have studied the construction cost, insertion heuristics, and probability of problematic configurations in substantially greater detail [3]–[5].

The systems-level concern is therefore best phrased as follows: a lookup can be logically bounded while a successful online insertion can require a displacement sequence whose worst-case search budget grows with the size of the structure, and an unsuccessful attempt can invoke a recovery action that touches much more state than an ordinary update. This is the phenomenon that motivates bounded-displacement designs.

It is also useful to distinguish *rehashing* from *reorganization*. A global rehash changes the placement functions or the effective table geometry and may require all resident keys to be considered again. A local reorganization can instead attempt to place a small subset of entries using the existing table state. The distinction becomes critical when a proposed stash mechanism is described as an “incremental rehash”. A routine that retains the global hash functions and merely retries the placement of stash contents is not equivalent to a global rehash with fresh functions. It is more accurately described as *stash recovery*, *local reinsertion*, or *bounded reorganization*. The terminology used in this manuscript follows the latter interpretation because that is what the supplied implementation actually does.

F. Stashes as an Overflow Mechanism

Kirsch, Mitzenmacher, and Wieder introduced the stash as a small auxiliary area for entries that cannot be placed in the ordinary cuckoo table without encountering an undesirable configuration [6]. The core idea is to exchange a very small amount of additional storage for a substantial reduction in the probability of catastrophic construction failure. The resulting data structure retains the small candidate set of the main cuckoo table while giving the implementation a dedicated location for exceptional entries.

The stash should not be interpreted as a proof that every bounded displacement attempt succeeds. A finite stash converts one type of exceptional event into another: instead of im-

mediately requiring the main table to be reconstructed, the implementation may defer resolution by storing the exceptional key in the auxiliary structure. If the stash is itself bounded, a second failure condition remains: the exceptional population may exceed the available stash capacity. The correct theoretical question is then the probability of a configuration requiring more than s exceptional placements, rather than merely the probability of one displacement chain exceeding a chosen limit.

Theoretical analyses of cuckoo hashing with a stash have established strong failure-probability reductions under specific random-graph models and specific hash-function assumptions [6]–[9]. However, these results must not be imported into a different algorithm without checking that the model is the same. In particular, generalized cuckoo-hashing analyses may use d to denote the number of items that a bucket can hold, while a d -ary cuckoo table may use d to denote the number of candidate locations per key. These parameters have different algorithmic meanings. The revised manuscript keeps these notions separate and does not transfer a failure exponent from one formulation to another merely because both are written using the symbols d and s [10].

This distinction is not cosmetic. If a theoretical result assumes independent uniformly random candidate sets, but the implementation uses a particular deterministic hash function and a state-dependent randomized eviction policy, then the result may not apply directly. A correct paper must therefore state whether an assertion is a theorem about an idealized random model, a theorem about the implemented hash family, an empirical observation, or a heuristic engineering interpretation.

G. d -ary Cuckoo Hashing and Load Thresholds

The move from two choices to $d \geq 3$ candidate locations changes the occupancy landscape substantially. In the standard one-slot-per-location model, the feasible placement problem can be represented as an orientation problem on a random hypergraph: each key corresponds to a hyperedge whose vertices are the candidate table positions, and a successful placement corresponds to assigning each key to one incident position without assigning more than one key to a position. This representation leads to sharp load thresholds for the existence of a valid placement [11], [12].

For example, a conventional three-choice cuckoo model with one item per location has a load threshold near 0.918 in the asymptotic random-graph model. The number is useful as a property of that model, but it is not a universal “92% limit” for every three-way cuckoo implementation. The threshold refers to the existence of a valid assignment under a particular random model; it does not automatically state that an online insertion algorithm can reach the threshold with bounded or constant work, nor does it imply that a finite system at a finite size will remain failure-free at the same numerical load.

This separation between *feasibility* and *online construction cost* is central to the present study. A table can possess a valid placement while a particular insertion heuristic fails to discover that placement within its allowed search budget.

Conversely, an insertion routine may find a placement at a load below the theoretical threshold while occasionally using an unusually long sequence of evictions. These are different failure modes and should be measured separately.

Recent work has made the online insertion problem substantially more precise. Random-walk d -ary cuckoo hashing has been shown to admit constant expected insertion time for suitable numbers of choices and loads below the corresponding threshold, extending earlier polylogarithmic results [5], [13], [14]. Other work has developed the bubble-up construction, achieving high load factors with expected insertion bounds that depend on the distance to full occupancy and constant expected positive-query time under its stated model [15]. These results are directly relevant because they demonstrate that the insertion-time question is richer than the classical two-table construction alone.

Consequently, the novelty claim in a bounded-displacement engineering paper must be narrow. It is not sufficient to state that bounded or de-amortized update paths are absent from the literature. There are prior constructions that explicitly address worst-case or de-amortized cuckoo updates [16], and there are recent theoretical constructions with stronger expected insertion guarantees under particular random models [13], [15]. A more defensible research question is whether a simple, implementation-oriented policy consisting of a fixed eviction budget plus a small stash yields a useful systems trade-off for exact-match tables, and under which loads and parameter choices the trade-off remains practical.

H. Networking Implementations and the Placement of the Research Problem

Cuckoo hashing has been used repeatedly in networking because the primary lookup path can be made short and predictable. Networking-oriented work has considered pipelined implementations, flow-monitoring structures, cache-aware layouts, and optimizations for batch lookup [17]–[19]. Such work demonstrates that the algorithmic structure of cuckoo hashing interacts closely with the memory system. The number of logical candidate locations is only one component of the actual latency; layout, bucket geometry, prefetching, SIMD operations, cache-line utilization, and the ratio of positive to negative lookups also matter.

For example, the Cuckoo++ study of networking-oriented hash tables emphasizes that connection-tracking workloads may contain millions of entries and that lookup cost is strongly affected by cache and memory behavior [19]. The work also illustrates a broader methodological principle: the same abstract hash table can have different performance characteristics depending on whether the implementation is optimized for mostly-positive lookup workloads, mostly-negative workloads, batched processing, or scalar access. A research paper should consequently avoid treating a single throughput number as an implementation-independent property of the data structure.

More recent FPGA work has likewise focused on insertion time as a first-class concern in networking applications. Megías et al. study alternative cuckoo insertion methods for

FPGAs and explicitly treat increasing load as a driver of additional relocation iterations [20]. This establishes the practical relevance of insertion-time engineering without implying that the same hardware conclusions hold for a CPU implementation or for a software-defined switch with a different memory architecture.

The present work targets a narrower but complementary problem. The proposed mechanism is deliberately simple: maintain a fixed number of candidate locations, cap the number of eviction rounds performed by the primary insertion procedure, and use a bounded stash for keys that remain displaced after the cap is reached. The objective is to obtain a strong local statement about the normal insertion path while making explicit what happens when that path cannot complete. The design is therefore closer to a latency-bounding policy than to a claim that the stochastic placement process itself has been eliminated.

I. Motivation for Bounded Displacement

The bounded-displacement idea can be stated in a single sentence: an insertion should not be permitted to consume an unbounded amount of work merely because a collision graph contains an unusually long path or cycle. In a d -ary table, the implementation examines the d candidate locations for the current key. If all are occupied, one candidate is selected for eviction and the displaced key becomes the new current key. Repeating this process can eventually find an empty position, but the number of repetitions is implementation-dependent. Introducing a fixed parameter $K = \text{MaxKicks}$ changes the policy by enforcing a hard cutoff on this primary path.

The immediate consequence is easy to state: the primary displacement loop performs at most a fixed number of eviction rounds. The more difficult question is what this implies for the overall insertion operation. If the loop reaches K without finding a free slot, a stash insertion may still complete the operation. If the stash is full, a recovery operation is needed. If recovery repeatedly fails, insertion may fail. Therefore, the true property of the implementation is not “every insertion succeeds in constant time”, but rather a more precise conditional statement: *the normal displacement path is bounded in the number of eviction rounds, while successful completion depends on stash availability and recovery behavior.*

This distinction is the principal correction made to the original manuscript. It prevents the algorithm from being described as a deterministic constant-time successful dictionary when the supplied implementation does not prove such a property. A deterministic bound on the number of loop iterations is useful and potentially important, but it is not sufficient by itself to establish a deterministic bound on the time required to complete every successful insertion, because the fallback mechanism can invoke additional work.

The same reasoning applies to incremental recovery. In the supplied implementation, the global hash functions are initialized once and are retained. When the stash overflows, the contents of the stash are extracted and reinserted using the same candidate hash functions with fresh randomized eviction

choices. This preserves the consistency between the physical table positions and the lookup functions. Redrawing the global hash functions while leaving already placed entries at their old positions would break the lookup invariant unless the entire main table were migrated. The revised implementation and manuscript therefore explicitly avoid that terminology error.

J. Problem Statement

Let U be the key universe, let $S \subseteq U$ denote the active key set, and let the main table consist of d sub-tables, each of size m , giving total primary capacity $r = dm$. For a key x , let

$$H(x) = \{h_1(x), h_2(x), \dots, h_d(x)\} \quad (1)$$

be its candidate locations, where the table index is also part of the location identity. The load factor is

$$\alpha = \frac{|S|}{dm}. \quad (2)$$

The insertion procedure maintains a current key x' , repeatedly attempts the d candidate locations of x' , and performs an eviction when all candidate positions are occupied. Let K be the maximum number of eviction rounds. After K rounds, the displaced key is routed to a stash of capacity s if a free stash position exists. If no stash position is available, a recovery procedure is invoked.

The research problem is to characterize the resulting system along four dimensions. First, what is the logical upper bound on the primary displacement work as a function of d and K ? Second, how does increasing K affect the frequency with which entries reach the stash? Third, how does the finite stash affect the frequency and cost of recovery events? Fourth, how do these choices affect lookup work, memory overhead, and empirical latency under controlled load factors?

The problem is deliberately stated without assuming that a particular stochastic bound is already valid. A theoretical probability bound can only be claimed after the implemented process has been mapped to a mathematical model for which the necessary independence, distributional, and occupancy assumptions hold. The paper therefore treats theoretical analysis and empirical validation as separate layers. The theory establishes what follows from the model; the implementation section states what the program actually does; and the experimental section reports only the measurements that were actually obtained from the corresponding run.

K. Research Questions

The study is organized around the following research questions.

RQ1: Bounded primary insertion work. For fixed d and K , what is the exact upper bound on the number of primary-table candidate checks and eviction operations performed before an insertion enters its fallback path?

RQ2: Stash utilization. As the load factor and displacement bound change, how does the observed rate of stash routing vary, and how large is the resulting distribution of stash occupancy?

RQ3: Recovery behavior. Under what conditions does stash recovery occur, how much work does the supplied implementation perform during recovery, and under what conditions can recovery itself fail?

RQ4: Lookup–update trade-off. What logical lookup cost is incurred by a bounded-displacement table, including the cost of searching the stash, relative to linear probing, chained hashing, and classical two-way cuckoo hashing?

RQ5: Reproducibility and claims. Which performance statements can be supported solely by the supplied implementation, and which previously stated numerical or hardware claims require additional experiments, benchmark output, or external evidence before they can appear in a journal manuscript?

L. Contributions and Claim Boundaries

The intended contribution of this manuscript is an implementation-oriented bounded-displacement variant of d -ary cuckoo hashing accompanied by an explicit statement of the conditions under which its latency bound holds. The contribution should be read as an algorithmic design and evaluation framework rather than as a claim that the design supersedes the theoretical literature on cuckoo insertion.

The first contribution is the explicit separation of the bounded displacement path from the exceptional recovery path. The parameter $K = \text{MaxKicks}$ controls the number of eviction rounds executed by the ordinary insertion routine. This gives a direct, deterministic bound on that portion of the algorithm. The paper then treats stash placement and stash recovery as separate events whose costs and failure conditions must be analyzed independently.

The second contribution is an implementation-consistent description of stash recovery. The supplied code preserves the global candidate hash functions after the main table begins to contain entries and uses fresh randomized eviction choices when attempting to reintegrate stash contents. This choice maintains the placement invariant of keys already resident in the main table. The manuscript does not call this process a global rehash because it does not redraw the global hash functions and does not migrate the entire table.

The third contribution is a parameterized evaluation methodology in which the displacement bound, stash capacity, and load factor are treated as independent experimental variables. This permits the practical trade-off to be studied directly: a smaller K can reduce the maximum primary-path work but may increase stash traffic, whereas a larger K can reduce stash routing at the cost of additional primary insertion work. Neither direction is presumed to be universally preferable.

The fourth contribution is a claim-auditing methodology for the empirical section. Numerical statements are tied to the actual executable benchmark harness rather than to prose inherited from an earlier draft. The current source uses 2,000,000 distinct random 64-bit keys and 2,000,000 lookup operations, evaluates a target load of 0.90 for the proposed $d = 3$, $K = 16$, $s = 4$ configuration, and includes a sensitivity sweep over $K \in \{4, 8, 12, 16, 24\}$ at a target load of 0.88.

These are implementation parameters, not measured results. Until the program is run under a specified environment and its output is preserved, the manuscript should not report a numerical throughput, percentile latency, stash-routing rate, or rehash count as an empirical result.

Several claims are intentionally excluded from the present contribution statement. The paper does not claim a universal deterministic $O(1)$ bound for complete successful insertion, because the supplied fallback procedure can invoke repeated stash recovery and contains a terminal failure condition. It does not claim that a four-entry stash is sufficient for all table sizes or workloads. It does not claim that the supplied `splitmix64` construction is equivalent to an ideal fully independent random hash family. It also does not claim FPGA or ASIC area overheads, DPDK compatibility, CAIDA-trace validation, or a particular host-processor throughput unless those claims are supported by corresponding experimental artifacts.

M. Scope of the Journal-Oriented Revision

The remainder of the paper is intended to be expanded in a modular manner. The next part will formalize the data structure and algorithm, define the cost measures, state the precise invariants that the implementation maintains, and give implementation-level pseudocode corresponding to the C++ source. A subsequent part will distinguish what can be proved directly from the bounded-loop structure from what requires a stochastic model of the placement process. The experimental part will then reconstruct the benchmark methodology from the source itself and reserve numerical results for executed runs.

This modular structure is intentional. A journal article on an algorithmic data structure should not rely on a single narrative in which implementation details, asymptotic bounds, and benchmark values are implicitly substituted for one another. Each class of statement must be traceable to its evidence. Established properties of classical cuckoo hashing are supported by the literature; properties of the proposed algorithm are established from its implementation and proofs; empirical observations are established from a documented run; and system-level implications are presented as interpretations conditioned on explicit assumptions.

II. RELATED WORK

A. Classical Cuckoo Hashing

Pagh and Rodler’s cuckoo hashing construction established a practical dictionary with worst-case constant lookup by associating each key with a small number of candidate locations and moving resident keys during insertion [1]. The fundamental abstraction is an assignment problem: each key must be assigned to one of its candidate positions, subject to the capacity constraint that a slot stores at most one key. The dynamic insertion procedure discovers such an assignment incrementally rather than solving the complete placement problem offline.

The key systems consequence is a deliberate asymmetry. A successful lookup does not need to search a long collision chain because the key has only a small number of legal locations. Insertion, by contrast, may alter the location of multiple already stored keys. The table therefore shifts complexity from a read-heavy path to a write-heavy path. This property has made cuckoo hashing attractive in dictionaries for which the lookup-to-update ratio is large.

The original construction also established the basic recovery mechanism for failed displacement sequences: after a bounded number of attempts, the implementation may discard the current placement functions and rebuild the table. Subsequent analyses refined the expected construction cost and the probability of problematic configurations [3], [4]. The literature consequently provides a strong foundation for treating lookup complexity and insertion complexity as distinct performance objectives.

B. Stash-Enhanced Cuckoo Hashing

The stash mechanism introduced by Kirsch, Mitzenmacher, and Wieder augments the primary cuckoo table with a small auxiliary area for exceptional items [6]. Rather than requiring the primary table to be perfect for every possible random placement graph, the design allows a small number of difficult items to remain outside the ordinary placement structure. This converts a rare global reconstruction event into a small amount of auxiliary storage, thereby changing the failure probability of the construction.

Kutznigg’s subsequent analysis examined the stash construction from a random-graph perspective and showed that the relevant asymptotic bounds can be obtained without requiring an additional search step of the kind used in some earlier formulations [7]. Aumüller, Dietzfelbinger, and Woelfel further investigated explicit and efficient hash families, showing that strong stash-related guarantees can be retained under substantially weaker hash-family assumptions than full randomness [8]. These results are important to implementation studies because an engineering realization must eventually replace idealized random functions with concrete functions that are efficiently evaluable.

The recent literature has also emphasized that stash analyses are sensitive to the exact cuckoo model. Minaud and Pamanthou revisited generalized cuckoo hashing with buckets of capacity larger than one and identified an error in an earlier asymptotic analysis, providing corrected bounds for the generalized setting [10]. The result is directly relevant to scientific writing because it illustrates the danger of reusing a probability exponent without verifying that the underlying parameters have the same meaning. In a one-slot-per-location d -ary table, d typically denotes the number of candidate locations. In a bucketized formulation, a separate parameter may denote the number of items a bucket can contain. The two must not be conflated.

C. Insertion-Time Analyses

The theoretical literature on cuckoo hashing has progressively sharpened the understanding of online insertion. Early results established that some insertion procedures can achieve favorable expected or amortized construction costs under suitable load conditions, while random-walk analyses addressed the practical process in which a displaced item repeatedly chooses a new location [5]. Arbritman, Naor, and Segev subsequently studied de-amortization techniques intended to convert attractive amortized properties into stronger worst-case operation bounds, explicitly motivating applications such as high-performance routing where occasional long updates are undesirable [16].

More recent results strengthen the theoretical picture for d -ary schemes. Walzer analyzed random-walk insertion below the peeling threshold and established a constant expected amortized insertion guarantee in that regime [14]. Bell and Frieze subsequently obtained an $O(1)$ expected insertion-time result for random-walk d -ary cuckoo hashing up to the load threshold under their stated assumptions [13]. These results make it particularly important for a new manuscript to specify whether its own contribution is a deterministic bound, an expected bound, an amortized bound, a high-probability bound, or simply an implementation heuristic.

Kusmaul and Mitzenmacher’s bubble-up cuckoo hashing provides an additional recent point of comparison. Their construction is designed for high load factors and achieves expected insertion times that depend on the remaining free capacity while retaining strong query behavior under the stated model [15]. It is therefore not scientifically accurate to position a simple bounded-kick implementation as though the literature contains no constant or near-constant insertion-time results. The meaningful question is instead what is gained by using a simpler implementation-level bound and a small stash in an exact-match networking context.

D. Load Thresholds and Feasible Placements

The analysis of cuckoo capacity is naturally connected to random hypergraphs. Each key can be viewed as a hyperedge connecting its candidate locations. A valid table configuration exists when the hypergraph admits an appropriate orientation subject to the slot capacities. This framework leads to sharp asymptotic load thresholds for standard k -ary cuckoo hashing [11], [12].

The threshold is a feasibility statement. It identifies a phase transition in the existence of a legal assignment in a specified random model. It is not, by itself, a statement about the runtime of a particular online insertion algorithm. An online heuristic can fail to discover a valid placement within a bounded search path even when an assignment exists. For the proposed bounded-displacement structure, this distinction is central: a stash-routing event may indicate either a genuinely infeasible local configuration or merely that the prescribed search budget was insufficient to discover an available assignment.

Accordingly, the revised paper uses three separate terms: *placement feasibility*, *algorithmic insertion success*, and *operational recovery*. Placement feasibility refers to the existence of a global assignment for the current set of keys under the candidate-location model. Algorithmic insertion success refers to whether the implementation can obtain such an assignment using its actual insertion procedure. Operational recovery refers to what the implementation does after the primary procedure does not complete. Only the first of these is characterized by the classical random-hypergraph threshold alone.

E. Networking-Oriented Cuckoo Implementations

Networking systems have repeatedly adopted cuckoo-based structures because of their short lookup path and compatibility with high-performance memory access strategies. Pontarelli et al. investigated a pipelined cuckoo implementation intended to increase throughput, while related work applied cuckoo tables to flow monitoring and other networking workloads [17], [18]. These studies demonstrate that a theoretically attractive collision-resolution scheme can require careful architecture-level treatment to obtain predictable throughput.

Le Scouarnec’s Cuckoo++ work provides a particularly useful systems reference because it studies memory layout, cache behavior, prefetching, and the distinction between positive and negative lookups in a networking context [19]. Its results underline a point that is directly applicable to the present paper: logical probe count and wall-clock lookup rate are correlated but not interchangeable. A design with fewer logical accesses can perform worse on a particular processor if those accesses have poor locality, while a design with more nominal work can benefit from batching or prefetching.

The present manuscript adopts this systems perspective but does not reproduce claims from prior hardware or software implementations without their corresponding experimental environment. In particular, results obtained using DPDK, custom packet-processing pipelines, FPGA fabrics, or a particular processor are treated as evidence about those systems rather than universal constants of cuckoo hashing [19], [20].

F. Hash Functions and Model Assumptions

The mathematical analysis of cuckoo hashing often assumes that candidate locations are independent and uniformly distributed, or that the hash functions belong to a family with explicitly quantified independence properties. Practical implementations rarely evaluate truly random functions; they use deterministic pseudorandom mixing functions or families chosen for computational efficiency. Aumüller et al. demonstrated that strong results can survive under restricted independence in appropriate constructions [8], but the existence of such results does not imply that every fast mixing function automatically inherits the same guarantees.

The supplied implementation uses a seeded `splitmix64`-style mixing function to generate candidate positions. This is a pragmatic engineering choice, and it is suitable for a benchmark that seeks a repeatable mapping from keys to slots.

It should not be described as though the benchmark itself proves all assumptions of a fully random hash model. The appropriate scientific procedure is to state the actual hash function, use the theoretical assumptions only where they have been established for that function or an equivalent family, and characterize any remaining gap as an implementation limitation or empirical question.

G. Positioning of the Present Study

The literature establishes the principal ingredients that underlie the present design: constant-size candidate sets and worst-case constant lookup in classical cuckoo hashing [1]; stash-based reduction of global failure probability [6]; refined hash-function analyses [8]; hypergraph-based load thresholds [11], [12]; de-amortized or theoretically improved insertion procedures [13], [14], [16]; high-load constructions [15]; and network-specific implementations [17]–[20].

Within this landscape, the role of the present study is intentionally narrower. It investigates whether a straightforward engineering policy—fixed candidate count, bounded eviction depth, and a small stash—provides a useful and analyzable compromise for exact-match tables. The paper does not claim that bounded displacement is the first technique to limit cuckoo update work, that the stash mechanism itself is novel, or that the resulting structure achieves a new asymptotic bound that supersedes existing theoretical constructions. Instead, the contribution is the combination of a concrete implementation, an explicit separation between normal-path and recovery cost, and a reproducible methodology for studying the resulting parameter trade-offs.

This positioning also changes how empirical results should be interpreted. A measurement showing that a particular configuration produces few stash events at a given load would constitute evidence about that configuration and workload. It would not by itself establish the asymptotic probability of stash overflow. Conversely, a theoretical bound on the placement graph would not establish that the particular C++ implementation achieves a specific nanosecond latency. The manuscript therefore treats empirical measurements, asymptotic results, and systems interpretations as complementary but distinct forms of evidence.

III. EXPERIMENTAL STATUS AND CLAIM DISCIPLINE

The supplied benchmark source is incorporated into this manuscript as an auditable implementation artifact. Its purpose is to provide a concrete mapping from the abstract design to executable data structures and to define the quantities that can subsequently be measured. The source contains implementations of linear probing, chained hashing, classical two-way cuckoo hashing, and the proposed bounded-displacement cuckoo structure. It also contains a fixed-size benchmark configuration and a sensitivity experiment over the primary displacement bound.

The present revision deliberately avoids presenting any benchmark output as fact. The program’s presence establishes what the experiment is configured to measure; it does not

establish that the configured experiment was executed on a particular machine, compiler, operating system, CPU frequency, cache state, or workload trace. A journal submission should report such environmental information together with a timestamped output artifact or another reproducible record of execution.

The current harness generates distinct pseudorandom 64-bit keys rather than parsing real packet traces. Accordingly, the phrase “synthetic flow identifiers” is used here only as an abstraction for benchmark keys. The keys are not literal IPv4 or IPv6 five-tuples, and the benchmark should not be described as a CAIDA-trace experiment or a DPDK experiment. Likewise, the code uses a `splitmix64`-style hash function rather than MurmurHash3 or an AES-derived hash. These distinctions are necessary because benchmark provenance is part of the scientific result.

The timing methodology also requires care. The harness measures individual operations using `std::chrono::steady_clock`. Such measurements can be useful for comparative microbenchmarking, but when the operations are very short, timer invocation and measurement overhead can become a non-negligible fraction of the reported value. A rigorous follow-up experiment should therefore characterize timing overhead, warm up the relevant code paths, specify processor affinity and frequency behavior, and report an appropriate number of independent repetitions. These are methodological requirements, not claims about the results of the current unexecuted program.

The manuscript likewise separates logical counts from timed latency. For example, the proposed implementation can report a bounded number of eviction rounds on its primary path, while a timer captures the complete software cost of the operation. The two measurements answer different questions and should not be substituted for one another. A later experimental section will therefore report both when the underlying run provides both.

IV. ORGANIZATION OF THE REMAINING MANUSCRIPT

The next manuscript part formalizes the proposed dictionary as a bounded-displacement d -ary cuckoo table. It defines the key universe, table geometry, candidate hash functions, stash state, displacement state, insertion invariants, and lookup semantics. It also translates the supplied C++ implementation into mathematical pseudocode without silently introducing behavior that the program does not implement.

The subsequent theoretical part distinguishes statements that follow directly from the bounded-loop structure from probabilistic statements that require additional assumptions about the hash family and insertion process. It also analyzes the complete stash-recovery recursion, the baseline rehash path, placement feasibility, and the source-level terminal recovery condition. In particular, the analysis avoids applying classical stash failure exponents to bounded search events unless the correspondence between the models has been established.

The implementation part will document the C++ data structures and benchmark harness, including the baseline imple-

mentations and the parameterization of the proposed scheme. The experimental methodology will then define the load-factor sweep, displacement-bound sensitivity, stash occupancy metrics, logical lookup costs, and wall-clock latency measurements. Numerical results will be inserted only when supported by an executed run and the corresponding recorded output.

The final technical discussion will address limitations, including finite-table effects, the distinction between a bounded primary path and a bounded complete operation, the dependence of probabilistic claims on hash-function assumptions, and the limitations of extrapolating software microbenchmarks to ASIC or FPGA packet-processing pipelines. The conclusion will summarize the validated properties rather than repeating aspirational performance claims.

V. SYSTEM MODEL AND DESIGN REQUIREMENTS

The preceding sections established the scope of the study and separated claims that can be inferred from the supplied implementation from claims that require executed measurements or additional theoretical assumptions. This section formalizes the dictionary represented by the proposed implementation. The purpose is not to retrofit the source code into a stronger abstract construction, but to identify precisely which mathematical object is implemented, what state it maintains, which operations are bounded by construction, and which exceptional paths remain relevant.

A central distinction is maintained throughout this section. The term *main table* denotes the collection of d fixed-size arrays used by the implementation. The term *stash* denotes the auxiliary array used only after the bounded eviction process has reached its configured depth. The main table and stash together represent the current dictionary state, but they do not have identical access semantics or memory organization. In particular, a key located in the main table is recoverable from its candidate hash positions, whereas a key located in the stash is found only by an explicit scan of the stash entries. This distinction is important for both lookup complexity and the interpretation of benchmark counters.

A. Dictionary State and Workload Model

Let $U = \{0, 1\}^w$ denote a universe of fixed-width keys. The experimental program instantiates this model using pseudorandom 64-bit keys generated with a Mersenne Twister engine and filtered through an unordered set so that the inserted benchmark keys are distinct. The implementation therefore realizes a finite synthetic-key workload rather than a packet-trace parser. The benchmark source configures $N = 2,000,000$ inserted keys and 2,000,000 subsequent lookup operations. These quantities define the benchmark configuration and are not, by themselves, empirical results.

At logical level, the dictionary stores a set

$$S = \{x_1, x_2, \dots, x_n\} \subseteq U,$$

where every key may be associated with an application value $v(x)$. The supplied benchmark stores keys only, because the measured operations concern placement and membership. A

production exact-match flow table would normally store both a flow identifier and associated state, but introducing such payload state into the formal model would not change the placement mechanism considered here.

The implementation partitions its primary storage into d sub-tables,

$$T_0, T_1, \dots, T_{d-1},$$

where every sub-table contains exactly m logical slots. The total number of main-table slots is therefore

$$r = dm. \quad (3)$$

Each slot stores either one key or the empty marker $\perp$. Let n_m denote the number of keys currently resident in the main tables and let n_s denote the number of keys resident in the stash. The logical population is

$$n = n_m + n_s. \quad (4)$$

The supplied C++ implementation uses its variable ‘ n ’ for this total population, including stash entries. This detail matters because a load measurement formed as ‘ $n/(d*m)$ ’ is not strictly the main-table occupancy whenever $n_s > 0$.

For analytical clarity, two load factors are therefore distinguished. The *main-table load factor* is

$$\alpha_m = \frac{n_m}{dm}, \quad (5)$$

and the *logical load factor relative to main capacity* is

$$\alpha = \frac{n_m + n_s}{dm}. \quad (6)$$

When the stash is empty, the two quantities coincide. When the stash contains entries, $\alpha > \alpha_m$. The distinction prevents a common but consequential interpretive error in which a table with a nonempty stash is described as having exactly the same physical occupancy as its logical population.

B. Candidate Locations

Let

$$h_i : U \rightarrow \{0, 1, \dots, m-1\}, \quad i \in \{0, 1, \dots, d-1\},$$

denote the candidate hash functions. For a key x , the implementation defines its candidate set as

$$C(x) = \{(i, h_i(x)) \mid 0 \leq i < d\}. \quad (7)$$

A main-table placement of x is valid exactly when x occupies one of the locations in $C(x)$. Consequently, the placement invariant for every key resident in the main table is

$$T_i[h_i(x)] = x \quad \text{for some } i \in \{0, \dots, d-1\}. \quad (8)$$

This formulation is deliberately weaker than assuming that the candidate locations are independent uniform samples. The latter is a mathematical property of an idealized hash model and is not established merely by observing that the implementation uses a high-quality pseudorandom mixing function. The supplied program defines `splitmix64` as a deterministic 64-bit mixing transformation and constructs different seeded

instances for the candidate functions. The resulting functions are repeatable and computationally efficient; however, the benchmark source does not establish the degree of independence needed to transfer every theorem proved for fully random hash functions. This paper consequently treats the hash family as an implementation parameter unless a particular theorem is invoked under its explicit assumptions.

The distinction is especially relevant because several published analyses of cuckoo hashing are asymptotic results for random or limited-independence hash families. Aumüller, Dietzfelbinger, and Woelfel, for example, established favorable stash guarantees for specific hash families with limited independence; such a result cannot be transferred automatically to an unrelated deterministic mixing function merely because both functions are fast to evaluate [8]. The present work therefore states the implementation’s actual hash mechanism and leaves stronger distributional claims conditional on independent verification.

C. The Auxiliary Stash

The proposed structure maintains a stash containing at most s entries. The implementation represents the stash as two parallel arrays: one array of keys and one occupancy array. A key is placed in the stash only after the bounded displacement procedure has performed the configured maximum number of kicks without finding an empty candidate slot.

Let S_{stash} denote the set of keys currently stored in the stash. The basic invariant is

$$S_{\text{stash}} \subseteq S, \quad |S_{\text{stash}}| \leq s, \quad (9)$$

and no key counted in S_{stash} is simultaneously required to occupy a main-table slot. The implementation’s stash lookup scans the entire allocated stash array until either a matching key is found or all s entries have been examined.

This arrangement gives a bounded logical lookup cost when s is fixed. The cost is not, however, equivalent to the cost of a main-table probe. A main-table candidate access consists of computing one candidate index, accessing one slot, and comparing one resident key. A stash access consists of examining an auxiliary slot and may require scanning multiple entries. The difference should be preserved when reporting both logical probe counts and wall-clock latency.

Stashes have a substantial theoretical literature. Kirsch, Mitzenmacher, and Wieder introduced the stash mechanism as a means of reducing the probability that an entire cuckoo placement must be rebuilt; later work refined the probability analysis and hash-function assumptions [6], [8]. More recent analyses have also shown that generalized stash failure bounds require careful attention to the exact bucket and choice model, with one commonly cited generalized bound having been corrected by Minaud and Papamanthou [10]. These results motivate the use of a stash, but they do not establish a particular numerical overflow probability for the implementation studied here.

D. Displacement State and the Bounded Primary Path

For an insertion of a key x , the implementation maintains a temporary key variable cur . Initially,

$$cur \leftarrow x.$$

At each displacement round, all d candidate positions of cur are examined in increasing sub-table order. If any candidate is unoccupied, cur is placed there and the insertion terminates. If all candidate locations are occupied, one candidate index is selected by the program’s pseudorandom generator, the resident key is evicted, and that evicted key becomes the new value of cur .

Let K denote the parameter `maxKicks`. The primary path contains at most K eviction rounds. Consequently, independent of the stochastic behavior of the placement process, the number of rounds executed by this path is bounded by

$$0 \leq K_{\text{obs}} \leq K. \quad (10)$$

The corresponding number of eviction events is at most K . Because each round examines at most d candidate positions before selecting an occupied position, the number of candidate-slot checks performed on the primary path is at most dK . Additional constant-time work is required for candidate selection and replacement.

The distinction between an *eviction bound* and a *complete insertion bound* is crucial. Equation (10) says only that the primary bounded-kick loop terminates after K rounds. It does not, by itself, imply that the complete insertion operation has completed successfully in bounded work. After the loop, the implementation may perform stash insertion. If the stash is full, it may invoke stash reintegration and recursively attempt the insertion again. The complete path therefore contains a recovery component not captured by K alone.

The implementation also does not perform explicit cycle detection. An eviction sequence may revisit the same slot or key state, but the supplied source does not maintain a visited-set structure or detect graph cycles. The stopping rule is purely depth based: after K rounds the current displaced key is routed to the stash if a free stash entry is available. Accordingly, statements in an earlier draft describing “cycle detection” as a component of the proposed algorithm are not retained in this revision.

E. Recovery from Stash Overflow

Stash overflow occurs when the bounded displacement procedure reaches its kick limit and every stash entry is already occupied. The supplied implementation responds by invoking `incrementalRehashStash`, which extracts the current stash contents, clears the stash occupancy flags, decreases the population counter by the number of extracted entries, and reinserts those entries using the same global candidate hash functions.

The term *incremental rehash* is potentially misleading in this context. Classical rehashing normally means changing one or more hash functions and reconstructing the entire

placement under the new mapping. The supplied routine does neither. It keeps the candidate hash functions fixed and changes only the sequence of randomized eviction choices used while attempting to reintegrate the small set of stashed keys. In this manuscript the operation is therefore described as *stash reintegration* or *stash recovery*; the original routine name `incrementalRehashStash` is retained only when referring directly to the C++ implementation.

This design choice is a direct correctness requirement. Once keys are resident in the main table under fixed functions $h_0, \dots, h_{d-1}$, changing those functions without relocating every resident key invalidates the placement invariant in Equation (8). A subsequent lookup would compute candidate addresses using the new functions while the resident keys would remain at positions generated by the old functions. The supplied source comments explicitly document this issue and maintain the candidate functions for the lifetime of the table. Thus, the present paper does not describe stash recovery as a hidden global rehash.

After the stash is cleared, the implementation reinserts the extracted stash keys. If reinsertion again exhausts its kick budget, a key can be returned to the stash. The process may therefore invoke additional stash recovery recursively. The recursion is explicitly limited by the source through `retryDepth`. If `retryDepth` reaches 20, the implementation increments `fullRehashesFromStuckStash` and returns ‘-4’.

The counter name `fullRehashesFromStuckStash` should not be interpreted literally. The code path associated with this counter does not rebuild the main table and does not redraw the global hash functions. It represents a terminal recovery condition within the current implementation. The journal manuscript therefore treats it as an implementation-level insertion failure indicator, not as evidence of a full-table rehash.

F. Load Monitoring and Capacity Selection

The original manuscript described a proactive load-factor monitor that would resize the table before reaching a theoretical d -ary threshold. The supplied benchmark source does not implement such a monitor. Instead, the benchmark computes the primary-table size once for each experiment using a target load factor and then constructs a table with that fixed capacity. For the principal proposed configuration, the source sets $d = 3$, $K = 16$, $s = 4$, and selects

$$m = \left\lfloor \frac{N}{\alpha_{\text{target}} d} \right\rfloor, \quad (11)$$

with $\alpha_{\text{target}} = 0.90$. The sensitivity experiment uses $\alpha_{\text{target}} = 0.88$ and evaluates $K \in \{4, 8, 12, 16, 24\}$.

Equation (11) is an experimental capacity-sizing rule, not a resizing algorithm. No operation in the proposed C++ class monitors the current population and allocates a larger table while the structure is live. Any statement asserting that the current implementation performs automatic resizing must therefore be removed. A future implementation may add

such a mechanism, but doing so would introduce a distinct state transition with its own migration cost and correctness conditions.

The theoretical literature on cuckoo hashing does provide sharp load thresholds for idealized random placement models. For one-slot-per-location k -ary cuckoo hashing, those thresholds can be characterized via random hypergraph orientability and related core phenomena [11], [12]. These thresholds are relevant for interpreting capacity choices, but they should not be confused with an implementation guarantee that every finite table below a quoted asymptotic threshold will admit a successful bounded-displacement placement under a particular hash function and random walk policy.

G. Design Requirements

The formal model leads to the following operational requirements for the proposed scheme.

First, every key resident in the main table must satisfy the candidate-location invariant in Equation (8). Second, every key resident in the stash must remain discoverable by the stash scan and must count exactly once toward the logical dictionary population. Third, the ordinary displacement path must execute no more than K eviction rounds. Fourth, stash recovery must not alter the hash functions defining the candidate mapping of the already populated main table. Fifth, any terminal failure condition must be represented explicitly rather than being silently treated as successful insertion.

These requirements are deliberately implementation-centric. They are sufficient to reason about correctness of the supplied C++ state transitions without assuming stronger properties such as universal hashing, independence between displacement events, or guaranteed successful placement at a particular load factor. Such stronger properties can be added as assumptions for a separate probabilistic analysis, but they are not intrinsic consequences of the data structure’s control flow.

VI. ALGORITHMIC SPECIFICATION OF THE PROPOSED STRUCTURE

This section gives an implementation-consistent specification of the principal operations. The specification is intentionally narrower than the earlier manuscript’s pseudocode. It does not introduce a resize operation, cycle detection, deletion, or global hash-function regeneration because none of those operations appears in the supplied proposed C++ benchmark. This avoids a common source of methodological drift in algorithm papers: presenting a mathematically elegant procedure that differs materially from the executable system used for evaluation.

A. Lookup

For a query key x , lookup first evaluates the d candidate positions and checks each corresponding main-table slot. If a slot contains x , lookup returns success immediately. If none of the candidate slots contains x , the stash entries are scanned sequentially. A successful stash lookup returns only after a

matching stash slot is found; a negative lookup terminates after all stash entries have been inspected.

A faithful high-level specification is given in Listing 1. The exact loop bounds correspond to the current C++ implementation.

```

1 lookup(x):
2   for i = 0 .. d-1:
3     p = h[i](x) mod m
4     if occ[i][p] and T[i][p] == x:
5       return FOUND
6
7   for j = 0 .. s-1:
8     if stashOcc[j] and stash[j] == x:
9       return FOUND
10
11  return NOT_FOUND

```

Listing 1. Implementation-consistent pseudocode for proposed lookup

Let $Q_m(x)$ denote the number of main-table candidate accesses and $Q_s(x)$ the number of stash entries examined. Then

$$\begin{aligned}
 Q_m(x) &\leq d, \\
 Q_s(x) &\leq s, \\
 Q_{\text{lookup}}(x) &= Q_m(x) + Q_s(x) \leq d + s.
 \end{aligned} \tag{12}$$

For fixed d and s , this is $O(1)$ logical work. Equation (12), however, is a bound on logical slot examinations rather than a statement about processor cycles. Cache misses, memory-system behavior, branch prediction, allocator effects, and the cost of evaluating the hash functions can change wall-clock latency substantially.

The code returns a distinct value when a stash hit occurs: a main-table hit reports a value in $\{1, \dots, d\}$, whereas a stash hit reports $d + 1$. The benchmark’s current lookup aggregation converts these return codes into a bounded probe statistic for comparison. A later experimental revision should retain separate counters for main-table hits, stash hits, and negative queries so that the logical cost distribution is not compressed into one scalar.

B. Insertion on the Primary Path

The proposed insertion operation begins with a duplicate test. If the key is already present in either the main table or stash, no structural modification is necessary. Otherwise the key becomes the current displaced object cur and enters the bounded displacement process.

For each round $r_k \in \{1, \dots, K\}$, the implementation evaluates the candidate positions of cur . The first free candidate slot immediately receives cur , after which the operation terminates. If all candidates are occupied, one of the d candidate indices is chosen uniformly from the generator state and the resident key at that location is evicted. The current key is written into the selected slot, and the evicted key becomes the new cur .

```

1 insertInner(x, retryDepth):
2   if lookup(x) == FOUND:
3     return 0
4
5   cur = x
6   kicks = 0

```

```

8   for round = 0 .. K-1:
9     for i = 0 .. d-1:
10      p = h[i](cur) mod m
11      if not occ[i][p]:
12        occ[i][p] = true
13        T[i][p] = cur
14        n = n + 1
15        return kicks
16
17    i = random_integer(0, d-1)
18    p = h[i](cur) mod m
19    evicted = T[i][p]
20    T[i][p] = cur
21    cur = evicted
22    kicks = kicks + 1
23
24  for j = 0 .. s-1:
25    if not stashOcc[j]:
26      stashOcc[j] = true
27      stash[j] = cur
28      n = n + 1
29      return STASH_ROUTED
30
31  return RECOVERY_REQUIRED

```

Listing 2. Implementation-consistent pseudocode for bounded displacement

The key structural property of Listing 2 is that the variable cur carries the displaced key across rounds. After each eviction, the resident key that was overwritten becomes the object whose candidate locations will be examined in the next round. This is the same displacement semantics as conventional cuckoo insertion, but the search is terminated after a fixed number of rounds rather than being allowed to continue until a graph-theoretic stopping rule or implementation-specific rehash threshold is reached.

The number of candidate checks on the primary path is at most dK . If the loop exits through an empty slot, the insertion’s primary displacement work is therefore bounded by a function of d and K only. This observation is purely structural and does not require any probabilistic assumption.

C. Stash Routing

When the bounded loop terminates without finding an empty candidate slot, the implementation searches the stash for an unoccupied entry. If one is available, the current displaced key is inserted into the stash and the insertion returns success. Because the stash itself is fixed-size, stash placement requires at most s occupancy checks in the current implementation.

This behavior should be interpreted as a deliberate error-containment policy. The stash does not repair the primary table in the same operation; instead, it converts an unresolved displacement event into a bounded auxiliary state. The corresponding trade-off is immediate and measurable: reducing K limits primary displacement work but can increase the rate at which keys are diverted to the stash. Increasing K can reduce stash routing but expands the primary insertion work budget.

No claim is made here that the routing probability is a function only of (K, α, d, s) in a universal closed form. The probability also depends on the table history, hash-function behavior, eviction-choice distribution, and finite-size effects. The literature on cuckoo hashing provides probabilistic results for related models, including constant-time expected insertion under certain random-walk policies below appropriate load thresholds, but such results concern specific algorithms and

assumptions and are not automatically equivalent to the present fixed-depth policy [5], [13], [14].

D. Stash Reintegration

When the stash has no free entry, the current implementation invokes stash reintegration. Let the stash contents before recovery be

$$S_R = \{y_1, \dots, y_q\}, \quad q \leq s.$$

The recovery procedure copies these keys into a temporary collection, clears the stash occupancy flags, adjusts the population counter so that the temporary copies are no longer counted as resident stash entries, and then calls the same bounded insertion routine for each y_j .

Importantly, the candidate hash functions remain fixed. The random generator state used to select eviction candidates changes as insertions proceed, but the mapping $x \mapsto h_i(x)$ does not. Consequently, if a previously resident main-table key is not moved during recovery, its physical location remains a valid candidate position under the same hash functions. The recovery procedure therefore preserves the main placement invariant by construction, provided that the insertion routine itself preserves it.

The recovery procedure is more accurately described as a local reintegration attempt than as a rehash. The work it performs depends on the number of stash entries and on the number of recursive recovery levels reached. With s fixed, each reinsertion has a bounded primary path; however, repeated stash overflow can cause several batches of stash reinsertion to occur. This observation is the reason the complete insertion operation is not stated here as an unconditional deterministic $O(1)$ successful-insertion theorem.

E. Complete Insertion and Terminal Failure

The outer insertion wrapper simply invokes ‘insertInner’. The latter accepts a `retryDepth` parameter and increments it whenever stash recovery is required. The supplied implementation permits at most 20 levels of this recovery recursion. Once that bound is reached, the operation returns ‘-4’ and records an implementation-level failure event.

This terminal condition has two methodological implications. First, the implementation is a finite-state bounded procedure in the practical sense that no insertion can recurse indefinitely. Second, the bounded procedure does not guarantee successful insertion for every possible state of the table. A proof claiming deterministic $O(1)$ successful insertion for all inputs would therefore be stronger than the behavior actually implemented.

For complexity analysis, let W_t denote a worst-case upper bound on the work of one ‘insertInner’ invocation when at most t additional recovery levels remain. There exists a constant $A = A(d, K, s)$ bounding the ordinary work of one invocation that either succeeds on the main path, succeeds in the stash, or reaches the recovery step. The recovery step can reinsert at most s extracted keys and then retries the triggering

key, so the source induces a recurrence with branching factor $s + 1$:

$$W_t \leq A + (s + 1)W_{t-1}, \quad t > 0, \quad (13)$$

with $W_0 \leq A$. Hence

$$W_t \leq A \sum_{j=0}^t (s + 1)^j. \quad (14)$$

Because the implementation fixes the recovery depth at 20, s is fixed, and d and K are fixed for a particular configuration, Equation (14) is finite with respect to table cardinality. This is a termination-work statement only. It should not be promoted to a deterministic successful-insertion theorem because the terminal path can return failure, and because a failed stash reinsertion can violate population preservation as discussed later.

F. Correctness Invariants

The proposed implementation can be checked against a compact set of invariants. These invariants are stronger and more useful for implementation auditing than an unsupported complexity theorem because they can be inspected directly in the source code.

Invariant 1: Unique residency. Each logical key belongs either to one main-table slot or to one stash entry, but not both. The duplicate check at the beginning of insertion establishes that an already resident key is not intentionally inserted a second time.

Invariant 2: Main placement validity. If a key x resides in a main-table slot $T_i[p]$, then $p = h_i(x)$. Every write performed by the bounded eviction loop uses a slot computed from the current key and one of the fixed candidate functions, so each eviction preserves this invariant for the newly written key.

Invariant 3: Stash discoverability. Every key transferred to the stash occupies an entry whose occupancy bit is set, and stash lookup scans precisely those entries. Reintegration clears the occupancy bits before reinserting the extracted keys, preventing stale occupancy flags from making an entry inaccessible.

Invariant 4: Population accounting. A successful insertion into the main table or stash increments the population counter, whereas stash reintegration subtracts the number of extracted stash keys before reinserting them. The population counter consequently denotes the number of logical resident keys rather than the number of main-table slots occupied.

Invariant 5: Stable candidate mapping. The global hash functions are fixed after construction. Stash reintegration changes the eviction randomization but does not alter $h_0, \dots, h_{d-1}$. This invariant is essential for continued correctness of lookup.

Invariant 6: Bounded primary displacement. Between two visits to a stash-recovery level, at most K eviction rounds occur. Therefore no single bounded-displacement invocation can remain indefinitely inside its primary loop.

The last invariant is the direct source of the proposed architecture’s primary latency control. The other five are what

make that bound meaningful as a data-structure property rather than merely a loop counter.

G. Comparison with Classical Two-Way Cuckoo Hashing

The classical Pagh–Rodler construction associates each key with a small number of possible locations and achieves worst-case constant lookup time [1]. Its principal insertion mechanism transfers work from lookup to updates by moving resident keys among their candidate positions. Later theoretical work studied the insertion-time behavior of random-walk variants, including polylogarithmic high-probability bounds and, under certain regimes and algorithms, constant expected insertion time [5], [13], [14].

The supplied baseline implementation is not a literal transcription of every detail of the theoretical Pagh–Rodler algorithm. It is a benchmark implementation that uses two arrays of equal size, a logarithmically derived ‘maxLoop’ parameter, and full-table rehashing after loop exhaustion. Consequently, comparisons in this manuscript use the terms *classical baseline implementation* and *proposed bounded implementation* rather than treating the benchmark code as a formal proof object for every theorem in the literature.

The conceptual contrast nevertheless remains clear. In the baseline, the displacement budget depends on the table capacity through a logarithmic expression. In the proposed implementation, the ordinary displacement budget is the explicit parameter K . The former emphasizes making failure sufficiently rare by allowing an n -dependent search budget; the latter emphasizes making the primary search budget structurally independent of n and accepting a nonzero amount of auxiliary-state traffic as the cost of that decision.

This trade-off is related to, but distinct from, de-amortized cuckoo hashing. Arbitman, Naor, and Segev developed a different construction that provably achieves constant worst-case operations with high probability by reorganizing the update process [16]. The existence of such a construction is important context: a bounded-displacement architecture should not be presented as the first or only route to worst-case bounded updates. The present study instead evaluates a simpler implementation policy and its practical behavior.

H. Relationship to High-Load d -ary Cuckoo Hashing

The choice $d = 3$ in the supplied benchmark is motivated by the well-known advantage of multiple candidate positions for increasing feasible load relative to two-choice cuckoo hashing. Sharp threshold work on random hypergraph orientability establishes critical densities for d -ary cuckoo placement under an idealized random model [11], [12]. More recent work has addressed the online insertion-time problem directly. In particular, Bell and Frieze proved constant expected insertion time for random-walk d -ary cuckoo hashing in a specified regime below the corresponding load threshold, while Kuzmaul and Mitzenmacher developed the bubble-up construction for high-load operation with a carefully controlled number of hashes [13], [15].

These results sharpen the conceptual position of the proposed structure. The present implementation should not be described as a replacement for these more sophisticated constructions. Its parameter K serves a different purpose: it imposes a software-level maximum on the number of primary eviction rounds before the auxiliary recovery mechanism is used. The paper’s experimental question is therefore not whether a small fixed K can reproduce the asymptotic guarantees of a sophisticated high-load construction. Rather, it asks how the finite control parameter changes the observable balance among displacement work, stash occupancy, and insertion completion behavior in a concrete implementation.

I. Summary of the Algorithmic Contract

The proposed data structure can now be characterized by a concise contract. A query examines at most d candidate main-table slots and at most s stash entries. An insertion performs an initial duplicate lookup, executes no more than K primary displacement rounds, routes the final displaced key to a free stash entry when one exists, and otherwise performs bounded-depth stash recovery using the same candidate hash functions. The primary displacement and lookup bounds are deterministic with respect to the configured d , K , and s . The probability of reaching the stash, the distribution of recovery events, and the frequency of terminal failures remain empirical or probabilistic questions that depend on the workload, hash behavior, and table state.

This contract is the basis for the remaining experimental analysis. It identifies exactly which quantities can be measured from the source, which statements follow immediately from the control flow, and which claims require an execution trace or a separate probabilistic proof. In particular, it removes the unsupported implication that bounded primary displacement alone establishes deterministic successful insertion or that a finite stash makes global failure impossible.

VII. THEORETICAL ANALYSIS OF THE IMPLEMENTED STRUCTURE

This section develops a formal analysis of the data structure and recovery procedure represented by the supplied C++ implementation. The objective is deliberately narrower than a conventional asymptotic analysis of an idealized cuckoo-hashing family. The analysis is instead source-faithful: every unconditional statement is derived from the control flow, state representation, or fixed parameters visible in the implementation, while probabilistic conclusions are stated only when an explicit probabilistic model is introduced. This distinction is essential for a journal submission because the existence of a known theorem for a related cuckoo-hashing construction does not imply that the theorem applies to a different insertion rule, a different hash family, or a different recovery mechanism.

Three classes of statements are distinguished throughout. A *structural statement* follows directly from the source code and therefore does not require randomization assumptions. A *conditional probabilistic statement* follows after imposing a specified random-hash or random-graph model. An *empirical*

statement depends on execution of the benchmark and cannot be established from source inspection alone. Only the first two classes are considered in the present section; empirical measurements are deferred to the experimental part and are not inferred here.

The analysis also exposes a subtle but important limitation of the present implementation. The bounded primary displacement loop is deterministically limited by `MaxKicks`, and the recovery recursion is deterministically limited by `retryDepth`. Those bounds constrain the amount of logical work performed by the implementation. They do not establish that every requested insertion succeeds. Moreover, the terminal recovery path in the supplied source can remove a stashed key from the accounted population if its reinsertion reaches the terminal depth and returns ‘-4’. Consequently, the current source should be treated as a benchmark implementation whose recovery semantics require one further correctness revision before being presented as a production dictionary with a total insertion guarantee.

A. Analytic Model and Notation

Let $U = \{0, 1\}^w$ be the key universe and let $S \subseteq U$ be the set of logically resident keys. The implementation represents the primary storage using d arrays of length m . The total number of primary slots is

$$R = dm. \quad (15)$$

The auxiliary stash has capacity s . Let S_m and S_s denote the subsets of keys resident in the main table and stash, respectively. Under a successful state transition,

$$S = S_m \dot{\cup} S_s, \quad |S_s| \leq s, \quad (16)$$

where $\dot{\cup}$ denotes a disjoint union.

Each key x has a candidate vector

$$H(x) = (h_0(x), h_1(x), \dots, h_{d-1}(x)), \quad (17)$$

where $h_i(x) \in \{0, \dots, m-1\}$. The physical candidate locations are pairs $(i, h_i(x))$, so candidate equality is defined on the pair rather than on the integer index alone. This avoids ambiguity when different sub-tables assign the same numerical offset to a key.

For a finite execution prefix, let $n_m = |S_m|$ and $n_s = |S_s|$. The main-table and logical load factors are then

$$\alpha_m = \frac{n_m}{R}, \quad \alpha = \frac{n_m + n_s}{R}. \quad (18)$$

The implementation’s population counter ‘ n ’ is intended to equal $n_m + n_s$. Hence, whenever the stash is nonempty, the quantity printed as ‘ $n/(d \cdot m)$ ’ is the logical load relative to primary capacity rather than the fraction of physically occupied primary slots. This distinction remains relevant in all load-related statements below.

The benchmark’s principal configuration uses $N = 2,000,000$, $d = 3$, and a target logical load of 0.90. The source computes

$$m = \left\lceil \frac{N}{0.90 d} \right\rceil = 740740, \quad (19)$$

so the corresponding primary capacity is $R = 2,222,220$. If all N inserted keys remain resident, the nominal logical load relative to this capacity is therefore

$$\alpha = \frac{2,000,000}{2,222,220} \approx 0.9000009. \quad (20)$$

This arithmetic describes the configured experiment, not an observed run result.

B. Deterministic Lookup Bound

The lookup procedure is structurally simple. It evaluates at most one slot for each candidate hash function and then scans the stash if no candidate matches. No loop in the lookup routine depends on the current number of keys other than through the fixed table dimensions, and the stash scan is limited by s .

Theorem 1 (Deterministic logical lookup bound). *For fixed d and stash capacity s , every invocation of the supplied lookup routine examines at most $d + s$ logical storage locations. Consequently, its logical access complexity is $O(d + s)$ and is $O(1)$ when d and s are constants independent of n .*

Proof. The first loop in ‘lookup’ iterates exactly d times, with one candidate slot examined in each iteration. If a key is found, execution terminates immediately; otherwise all d candidate slots have been examined. The second loop iterates over at most s stash positions. Therefore no execution examines more than $d + s$ logical locations. No other loop is present in the lookup routine. The complexity statement follows directly. $\square$

The theorem is a deterministic statement about the implementation, not a statement about processor cycles. In the present C++ source, each candidate is obtained by evaluating a deterministic seeded hash function and indexing one vector, while the stash is represented by parallel vectors and scanned linearly. The number of hash evaluations and comparisons is bounded, but the time of those evaluations remains machine dependent.

The distinction between positive and negative queries is also material. A positive main-table lookup may terminate after one candidate; a positive stash lookup can require several stash comparisons; and a negative lookup can require all d candidate checks and all s stash checks. Thus, while the worst-case logical bound is $d + s$, the distribution of logical work is workload dependent. A later experimental section should therefore report separate positive-main, positive-stash, and negative-query populations rather than only a single mean probe count.

C. Deterministic Bound on the Primary Displacement Path

The primary insertion path maintains the temporary key ‘`cur`’ and a counter ‘`kicks`’. In each round, all d candidate positions of ‘`cur`’ are inspected. If none is free, one candidate is selected and its resident key is evicted. The outer loop terminates after at most $K = \text{maxKicks}$ rounds.

Let $E(x)$ denote the number of eviction events experienced by the insertion attempt before it leaves the primary loop. The source guarantees

$$0 \leq E(x) \leq K. \quad (21)$$

No probability statement is involved in Equation (21); the bound follows from the syntactic loop limit.

The primary path performs at most dK candidate checks in the worst case. In addition, every round in which all candidates are occupied performs one extra hash evaluation for the randomly selected eviction target because the source recomputes the selected candidate index. Therefore the number of main-table hash evaluations performed by the primary loop is at most

$$K(d + 1). \quad (22)$$

The initial duplicate lookup adds at most d further candidate-hash evaluations. Thus an implementation-level upper bound on hash evaluations before leaving the primary path is

$$H_{\text{primary}}(x) \leq d + K(d + 1). \quad (23)$$

The source also performs at most K displacement writes, followed either by one final primary-table write or one stash write if the displacement limit is reached. These counts exclude the auxiliary work caused by stash recovery.

Theorem 2 (Primary-path work bound). *For fixed d and K , the portion of an insertion from the initial duplicate test through completion of the bounded eviction loop requires $O(dK + d + s)$ logical work, and at most K eviction events occur.*

Proof. The duplicate test contributes at most $d + s$ logical checks. The bounded loop contains at most K rounds, each performing at most d candidate inspections and one additional selected-candidate access. Hence the primary loop contributes $O(K(d + 1))$ logical operations. Summing the terms yields $O(dK + d + s)$ for the pre-recovery portion of the insertion. $\square$

This result is the principal deterministic property of the proposed policy. Importantly, it bounds the *primary displacement path*, not necessarily the complete insertion operation. A manuscript that states simply “insertion is $O(1)$ ” without qualifying the recovery path would therefore conflate two different claims.

D. Preservation of Main-Table Placement During Eviction

The placement invariant for main-table residents is

$$T_i[h_i(x)] = x \quad \text{for some } i \in \{0, \dots, d - 1\}. \quad (24)$$

Suppose a key x occupies the selected target slot $(i, h_i(y))$ for the current displaced key y . The insertion operation writes y into that slot and moves the old resident x into the temporary variable ‘cur’. The old resident is no longer considered committed to that physical location, while the new resident satisfies its own candidate-location condition by construction. All other committed main-table residents are untouched by that write.

Lemma 1 (Local preservation of placement). *Assume all committed keys satisfy Equation (24) before a single eviction step. After overwriting one candidate slot with the current temporary key, every committed main-table resident other than the evicted key still satisfies the placement invariant, and the new occupant satisfies the invariant at the written slot.*

Proof. Let the current temporary key be y and let the selected candidate be (i, p) with $p = h_i(y)$. By the insertion code, the value y is written to $T_i[p]$, so y occupies one of its candidate locations. Exactly one previous main-table resident, say x , is overwritten and moved to ‘cur’; that key is temporarily not committed to the main table. Every other main-table slot is unchanged, so every other committed resident remains at its previously valid candidate location. $\square$

The lemma explains why a temporary displacement variable is sufficient: the algorithm need not maintain a complete graph representation during insertion because the displaced key itself carries the unresolved part of the placement problem. The lemma does not imply that the displaced key will eventually be placed successfully. It only states that the already committed portion of the structure is not invalidated by a legal eviction write.

E. Stash Routing as a Conditional Completion Event

After K unsuccessful displacement rounds, the source scans the stash for an unoccupied entry. If at least one exists, the current key is inserted into the stash and the insertion returns a successful status. This provides a deterministic conditional statement.

Proposition 1 (Conditional stash completion). *Suppose an insertion reaches the end of its K -round primary path, and suppose the stash contains at least one free entry. Then the current insertion invocation terminates with the current displaced key stored in the stash after at most s stash-slot examinations.*

Proof. The implementation scans stash entries sequentially. Because at least one entry is unoccupied, the first such entry is eventually encountered no later than the s -th iteration. The key is written to that entry, its occupancy flag is set, the population counter is incremented, and the routine returns. $\square$

The proposition should not be read as saying that stash routing is rare. Its frequency depends on the workload and the state of the table. In particular, reducing K can make the stash a more frequently used escape path, while increasing K may reduce the rate of routing but increases the deterministic upper bound on primary insertion work. This establishes a tunable trade-off, but it does not yield a universal closed-form routing probability.

A useful conditional event is

$$\mathcal{R}_K(x) = \{\text{the primary path of } x \text{ exhausts } K \text{ rounds}\}. \quad (25)$$

Stash routing occurs on $\mathcal{R}_K(x)$ only if the stash is not already full. Consequently,

$$\Pr[\text{stash route for } x] \leq \Pr[\mathcal{R}_K(x)], \quad (26)$$

for any probability model under which the probability is defined. Equality need not hold because an exhausted primary path can be followed by a full stash, leading instead to recovery. Equation (26) is therefore a logical event inclusion, not a numerical estimate.

F. Why a Finite Stash Does Not Imply a Negligible Failure Probability

The classical stash literature establishes strong asymptotic failure reductions for specified random cuckoo-hashing models [6]–[8]. Those results concern the probability that a valid placement cannot be obtained, or that the stash would need to exceed a specified capacity, under a mathematical model of candidate locations. They do not imply that a fixed-depth online heuristic will discover every feasible placement.

For the present implementation, two distinct failure events must be separated:

- 1) a *search failure*, in which the bounded eviction procedure does not find a free candidate within K rounds; and
- 2) a *recovery failure*, in which stash reintegration is invoked but the implementation eventually reaches its terminal recovery depth without restoring all affected keys.

The first event can occur even if a globally valid assignment exists. The second event is a property of the chosen recovery policy rather than solely of the underlying placement graph.

This distinction can be expressed by defining $\mathcal{F}_{\text{feas}}$ as the event that no legal placement exists for the current key set under the candidate mapping, and $\mathcal{F}_{\text{alg}}$ as the event that the current implementation fails to complete its insertion procedure. In general,

$$\mathcal{F}_{\text{feas}} \subseteq \mathcal{F}_{\text{alg}} \quad (27)$$

need not hold, because an algorithmic failure can occur while a legal placement still exists. Conversely, a particular implementation could also enter a failure path because the hash family or recovery policy differs from the model used to define $\mathcal{F}_{\text{feas}}$.

The paper therefore avoids converting a theoretical stash theorem into a numerical claim for the supplied program. In particular, no assertion of the form “a stash of size four makes failure negligible” is justified merely by citing the general stash literature. The actual probability depends on the candidate-location distribution, the insertion dynamics, the finite table size, the workload order, and the specific recovery process.

G. Complete-Insertion Recovery Recurrence

The complete insertion routine contains an additional layer of recursion. When the stash is full, ‘incrementalRehashStash’ extracts up to s stash entries and attempts to reinsert them, after which the original insertion is retried at the next recovery

depth. This produces a branching factor of at most $s + 1$ at each recovery level: up to s extracted stash keys are reinserted, and the triggering key is then attempted again.

Let $A(d, K, s)$ be an upper bound on the ordinary logical work of one ‘insertInner’ invocation excluding recursive recovery calls. Such a bound exists for fixed d , K , and s because the duplicate lookup, bounded primary path, and stash scan are all bounded. Let W_t be the maximum work of one invocation with at most t recovery levels available. The implementation gives the recurrence

$$W_t \leq A + (s + 1)W_{t-1}, \quad t > 0, \quad (28)$$

with

$$W_0 \leq A. \quad (29)$$

Theorem 3 (Finite logical-work bound of the supplied recovery procedure). *For a fixed recovery-depth limit T , the complete logical work of an ‘insertInner’ invocation is bounded by*

$$W_T \leq A \sum_{j=0}^T (s + 1)^j = A \frac{(s + 1)^{T+1} - 1}{s} \quad (30)$$

for $s > 0$. Thus, for fixed d , K , s , and T , the source code has a finite bound with respect to the number of logical keys n .

Proof. Equation (28) follows because one invocation can perform at most A nonrecursive work, then trigger a recovery that reinserts at most s extracted stash entries and finally retries the triggering key. Each of these at most $s + 1$ calls uses one fewer remaining recovery level. Repeated substitution gives

$$W_T \leq A + (s + 1)A + (s + 1)^2 A + \cdots + (s + 1)^T A,$$

which is the stated geometric sum. $\square$

The recurrence in Equation (28) is stronger and more faithful to the source than a recurrence with branching factor s because the triggering key is explicitly retried after stash reintegration. This correction matters when reasoning about worst-case work, even though it does not change the qualitative conclusion that the primary loop alone is not a complete insertion bound.

For the benchmark configuration $s = 4$ and the implementation’s maximum recovery depth $T = 20$, the geometric factor is

$$\sum_{j=0}^{20} 5^j = 119,209,289,550,781. \quad (31)$$

This value is finite, but it is clearly not a practically meaningful latency constant. Its purpose in the analysis is to demonstrate why merely observing that all loops are bounded by hard-coded constants does not constitute an engineering-quality worst-case latency guarantee. An asymptotic statement may technically classify this quantity as independent of n , yet the resulting constant is so large that such a classification would obscure rather than clarify the operational behavior of the implementation.

H. Terminal Recovery and a Correctness Defect in the Supplied Source

The source contains an additional condition at recovery depth 20:

```

1  if (retryDepth >= 20) {
2      fullRehashesFromStuckStash++;
3      return -4;
4  }

```

The label ‘fullRehashesFromStuckStash’ is semantically misleading because no full main-table rehash occurs on this path. More importantly, the preceding recovery function has already removed the stash entries from their original storage and decremented the population counter before attempting to reinsert them. If one of those reinsertion calls reaches the terminal ‘-4’ result, the corresponding logical key is not restored to either the main table or the stash.

This observation yields a source-level correctness condition.

Proposition 2 (Conditional population preservation). *Population accounting is preserved across stash reintegration only if every extracted stash key is successfully reinserted into either the main table or the stash. If any extracted key reaches the terminal ‘-4’ path after extraction, the population counter can undercount the logical key set and the implementation can lose a previously resident key.*

Proof. Immediately before reintegration, the source constructs a temporary vector containing all occupied stash entries and executes ‘ $n = S.size()$ ’. Hence the extracted keys are removed from the population count. A successful reinsertion increments ‘ n ’ again when the key enters the main table or stash. If an extracted key instead returns ‘-4’, neither a main-table insertion nor a stash insertion increments ‘ n ’, so the decremented population count is not restored for that key. The key is also absent from both storage regions after the failed reinsertion. $\square$

The proposition is not merely a stylistic concern. A dictionary insertion failure is normally expected to report failure while preserving all previously committed keys. The present terminal path can violate that expectation because stash recovery is not transactional. The manuscript therefore treats the implementation as *not yet insertion-atomic under terminal recovery*. This statement supersedes any earlier wording that implied unconditional dictionary correctness under all recovery outcomes.

A production-quality repair can be implemented in several ways. One approach is to keep a separate transaction-local collection containing all affected keys and to commit their removal from the stash only after all reinsertion attempts have succeeded. Another is to retain a secondary recovery stash and restore failed keys on the terminal path. A third approach is to perform a true global rebuild under fresh candidate functions, but that would change the algorithmic contract and would need to update all main-table residents atomically. The present benchmark does not implement any

of these transactional mechanisms, so no repaired behavior is attributed to the current source.

I. No Deterministic Success Theorem Follows from the Loop Bound

A common inference in informal descriptions of bounded displacement is that a fixed maximum number of kicks establishes an $O(1)$ insertion guarantee. The correct implication is weaker. Let K be fixed. Then the number of primary eviction events is bounded by K . If the path exits through a free primary slot, insertion succeeds after bounded work. If it exits through an available stash entry, insertion also succeeds after bounded additional work. If the stash is full, recovery begins, and successful completion becomes conditional on the recovery sequence.

Thus the strongest unconditional statement supported by the current source is of the form

$$\text{primary work} \leq C_1(d, K, s), \quad (32)$$

while the complete insertion satisfies

$$\text{complete work} \leq C_2(d, K, s, T) \quad (33)$$

only in the sense that the implementation eventually returns either a success code or a terminal failure code after bounded recovery depth. The second bound is a *termination bound*, not a *successful insertion bound*.

This distinction is important when comparing the proposed structure with published constructions. The literature contains several results establishing expected, amortized, high-probability, or worst-case operation bounds for carefully defined cuckoo-hashing variants [5], [13], [14], [16]. Those properties arise from the corresponding algorithms and analyses; a hard-coded depth counter in a different implementation cannot inherit the same theorem without a proof establishing equivalence.

J. Placement Feasibility and the d -Ary Threshold Model

The feasibility question can be expressed independently of the online insertion procedure. Construct a bipartite incidence graph with one vertex for each key and one vertex for each main-table slot, and connect key x to its d candidate slots. A feasible one-slot-per-location cuckoo assignment corresponds to a matching that covers all key vertices.

Under a fully random d -choice model with one key per slot, asymptotic threshold results characterize a critical load c_d^* such that the probability of a feasible placement changes sharply around that value as the table size grows [11], [12]. For $d = 3$, the classical threshold is approximately

$$c_3^* \approx 0.9179352767 \quad (34)$$

for the idealized one-slot-per-location model. The configured principal benchmark load of approximately 0.90 is below this asymptotic value.

The interpretation must nevertheless remain conditional. The threshold is a statement about *existence of a legal assignment* in a random model. It is not a guarantee that the supplied

online algorithm will find the assignment within $K = 16$ eviction rounds. Nor is it a statement about the four-entry stash, because the benchmark permits keys to remain outside the primary matching structure.

The most useful conceptual decomposition is therefore

feasibility $\rightarrow$ does a valid assignment exist?, (35)

search $\rightarrow$ can the insertion policy find one?, (36)

recovery $\rightarrow$ what is done when bounded search fails? (37)

The threshold literature primarily addresses the first question. The present implementation introduces explicit policies for the second and third.

K. Relation Between K and the Feasibility Margin

For a fixed load factor below the placement threshold, increasing K enlarges the search budget of the online insertion rule but does not alter the underlying candidate graph. This can only change the behavior of the algorithmic search process; it does not change whether a global assignment exists for the current set of hash edges.

Consequently, K should be interpreted as a *search-depth parameter*. It is not a capacity parameter in the same sense as m , and it does not shift the mathematical placement threshold c_d^* of the underlying random graph model. For example, changing K from 4 to 24 in the supplied sensitivity experiment changes the number of allowed displacement rounds but leaves $d = 3$, the table capacity rule, and the candidate hash functions structurally unchanged.

This distinction suggests an appropriate experimental interpretation of a K sweep. If increasing K reduces the stash-routing rate, that observation indicates that a larger local search budget is allowing more displaced paths to terminate in the primary table. It does not demonstrate that the asymptotic threshold has changed. If increasing K has little effect, this may indicate that the dominant difficulty arises from the structure of the displacement graph rather than from the finite truncation of the search. Such interpretations require actual benchmark outputs; the present section only identifies the variables and their causal meaning.

L. A Conditional Probabilistic Model for Bounded Search

To discuss stash routing probabilistically, one must specify a model for candidate occupancy and displacement choices. A tractable but idealized model can be obtained by assuming that candidate hashes are uniformly distributed and that, for a sufficiently mixing state, each candidate independently encounters an occupied primary slot with probability approximately α_m . Under this approximation, the probability that a fresh key has all d candidate positions occupied in its first candidate scan would be

$$p_1 \approx \alpha_m^d. \quad (38)$$

Equation (38) is a *mean-field heuristic*, not a theorem for cuckoo insertion. The candidate events are not independent after the table has evolved through correlated evictions, and the occupancy distribution is constrained by the placement

process. The expression is therefore useful only as a rough diagnostic for how load and candidate count interact at the first scan.

No equally simple expression can be asserted for the probability of exhausting K displacement rounds. The random walk changes the state after every eviction, and the next candidate locations are correlated with the state created by earlier evictions. Existing theoretical analyses of random-walk cuckoo hashing employ substantially more detailed probabilistic machinery for precisely this reason [5], [13], [14].

For the same reason, a statement such as

$$\Pr[\mathcal{R}_K] = \alpha_m^{dK} \quad (39)$$

would be unjustified for the current algorithm. Such a formula would implicitly assume independence between successive candidate scans, which the displacement process does not provide. The journal version therefore does not use a closed-form stash-routing probability unless a separate proof establishes the required dependence structure.

M. Why the Supplied Hash Function Does Not Automatically Inherit Random-Hash Theorems

The benchmark defines a deterministic seeded ‘splitmix64’-style mixing function and forms different candidate functions by changing the seed. This gives a reproducible family of concrete functions. It does not, solely from source inspection, establish that the resulting candidate vectors have the same distribution as independent uniformly random functions.

Published results demonstrate that some cuckoo-hashing guarantees survive under restricted-independence hash families [8]. Such results are valuable because they show that full randomness is not always necessary. Nevertheless, the assumption must match the theorem. In the present manuscript, the appropriate logic is therefore:

- 1) state the actual deterministic hash implementation;
- 2) state the idealized random-hash model when using a theoretical threshold or probability result;
- 3) explain whether a theorem has been proved for the actual hash family; and
- 4) otherwise treat the difference as an experimental limitation.

This prevents a common category error in which pseudorandom implementation behavior is treated as a mathematical guarantee.

N. Baseline Analysis: Classical Two-Way Cuckoo Implementation

The supplied classical baseline maintains two tables of size m and two candidate functions. Its loop parameter is calculated as

$$L = \left\lceil \frac{3 \ln(2m)}{\ln(1 + \varepsilon)} \right\rceil, \quad (40)$$

subject to a lower limit of four. Each outer-loop iteration can perform an eviction in the first table and then an eviction in the second table. Consequently, the maximum number of eviction events before the rehash path is not L but as much as $2L$.

This distinction is important when interpreting the benchmark field ‘maxLoopParam’. The variable represents the number of outer iterations in the source code, whereas ‘maxKicksObserved’ represents the number of actual eviction events returned by ‘insert’. The two quantities are therefore not interchangeable.

When the loop is exhausted, the baseline gathers the resident keys from both tables, adds the current displaced key, clears the occupancy arrays, redraws the candidate hash functions, and recursively reinserts the collected keys. The full-rehash path therefore has cost that grows with the number of resident keys. The value returned by the outer insertion function after a rehash is simply ‘maxLoop’; it is not the actual wall-clock cost of the rehash.

This observation provides another source-level correction to the interpretation of benchmark output. A returned integer used as a displacement statistic is an algorithmic status surrogate; it should not be assumed to equal elapsed time or the number of memory accesses performed during a rehash. The timer surrounding the insertion is the appropriate source for wall-clock latency, while the kick counter measures displacement activity.

O. Comparison with Established Online Insertion Results

The theoretical status of online cuckoo insertion is now sufficiently mature that the manuscript should frame the proposed implementation as a bounded-policy study rather than as the first construction to obtain controlled insertion time. Random-walk analyses established nontrivial upper bounds on insertion effort, including polylogarithmic high-probability results under appropriate conditions [5]. Walzer later established constant expected amortized insertion time for random-walk cuckoo hashing below the peeling threshold, while Bell and Frieze established an $O(1)$ expected insertion-time result up to the load threshold for their random-walk d -ary model under the stated assumptions [13], [14].

Kuszmaul and Mitzenmacher’s bubble-up cuckoo hashing provides another distinct construction aimed at high-load operation. Their analysis controls both the number of hash locations and the expected insertion cost as a function of remaining free capacity [15]. This work is relevant because it demonstrates that high-load d -ary cuckoo hashing admits sophisticated insertion procedures with theoretically characterized behavior. It does not imply that the present bounded-kick implementation has the same guarantees.

The appropriate claim for the current structure is therefore narrower: it provides a configurable hard upper limit on the ordinary displacement path and uses a small stash as an explicit overflow mechanism. Whether this policy offers an attractive systems trade-off remains an empirical question.

P. Deterministic Bound Versus Probabilistic Reliability

A central design objective is to separate latency control from reliability. The parameter K supplies a deterministic upper bound on one local search path. The stash supplies a bounded auxiliary repository for keys whose current search

path does not terminate in the main table. Reliability, however, is governed by the frequency with which the algorithm reaches these exceptional states and whether recovery can restore them.

This creates a three-axis design space:

$$(K, s, d) \mapsto (W_{\text{prim}}, O_{\text{stash}}, F_{\text{rec}}). \quad (41)$$

Here W_{prim} denotes primary-path work, O_{stash} denotes stash occupancy, and F_{mrec} denotes recovery frequency. Increasing K expands primary work and may reduce stash pressure. Increasing s expands lookup work and auxiliary memory while allowing more unresolved keys to be retained without immediate recovery. Increasing d expands the lookup candidate set and table metadata while generally increasing the combinatorial flexibility of the placement problem.

No single asymptotic parameter dominates this trade-off in a networking setting. A flow table with lookup-heavy workloads may value a small candidate count and stable negative-query latency, whereas an insertion-heavy control-plane workload may tolerate additional lookup work to obtain more placement flexibility. The present implementation offers a controlled experimental surface for these choices, but it does not mathematically select one configuration as universally optimal.

Q. Theoretical Interpretation of the $d = 3$, $K = 16$, $s = 4$ Configuration

The benchmark’s principal proposed configuration uses three candidate tables, sixteen bounded eviction rounds, and a four-entry stash. These values should be regarded as experimental design parameters rather than theoretically optimal constants. In particular, the choice $d = 3$ is consistent with the broad d -ary cuckoo literature and gives a placement threshold substantially above the two-way case under the idealized one-slot model [11], [12]. The value $K = 16$ supplies a finite local-search budget. The value $s = 4$ supplies a very small recovery region.

At the configured nominal load of approximately 0.90, the idealized three-choice threshold of approximately 0.9179 provides a nonzero asymptotic feasibility margin. That margin is not a guarantee that the finite implementation never routes to the stash, nor is it a guarantee that a particular key ordering will result in a successful bounded displacement. It merely indicates that the target load lies within the asymptotic feasible region of one related random placement model.

This is precisely the situation in which the proposed experiment is scientifically useful. The gap between a global feasibility threshold and the finite search budget can be studied empirically: how frequently does a bounded insertion policy fail to find the feasible placement before consuming its budget, and how much auxiliary state is needed to absorb such events? Those are implementation questions that are not answered by the threshold theorem alone.

R. A Formal Claim Hierarchy for the Manuscript

The theoretical analysis supports the following hierarchy of statements.

TABLE I
CLAIM HIERARCHY SUPPORTED BY THE SOURCE-FAITHFUL ANALYSIS.

Claim class		Status in the present manuscript
Worst-case lookup bound	logical	Established directly from the lookup loops: at most $d + s$ storage locations.
Primary eviction bound		Established directly from the bounded loop: at most K eviction events.
Primary-path complexity	logical	Established as a function of fixed d , K , and s .
Stash-routing frequency		Not established analytically; requires an execution trace or a separate probabilistic proof.
Stash-overflow probability	probability	Not established analytically for the supplied hash family and recovery policy.
Successful insertion for every finite input		Not established; terminal recovery can return failure.
Deterministic complete insertion	$O(1)$ successful	Not established by the present implementation.
Asymptotic placement threshold	placement	Established only for the corresponding idealized random cuckoo model, not for the concrete hash implementation.
Wall-clock latency		Empirical quantity; cannot be inferred without executing the benchmark.

The table should be treated as a governing rule for the remainder of the manuscript. Any later numerical result must be attached to the corresponding executed experiment, and any theorem-level statement must identify the assumptions under which it holds. This claim hierarchy is intended to prevent a common failure mode in systems papers, in which an implementation detail, a theoretical result from a related model, and an unexecuted benchmark configuration become conflated into a single unsupported performance claim.

S. Implications for Reproducibility

A rigorous theoretical section also constrains the reproducibility protocol. For a benchmark result to be interpretable, the following state variables must be recorded together: N , d , m , K , s , the hash-function seeds, the insertion order, and the recovery-depth limit. The supplied source fixes the principal key-generation seed, candidate-hash seeds, and several configuration constants directly in code. Future experimental runs should preserve these values in the result metadata or expose them through command-line arguments so that repeated runs can be compared without source-code inspection.

The insertion order is especially relevant. The same set of keys can produce different online displacement behavior under a different ordering, even when the candidate hash functions are unchanged. Consequently, a statement such as “the table succeeds at load 0.90” is incomplete unless the workload ordering and hash-state initialization are specified.

The benchmark also uses a single pseudorandom generator instance for eviction choices. If multiple independent repetitions are performed, the seed controlling this process should be varied independently from the seed controlling key generation. Otherwise repeated runs may merely reproduce the same displacement trajectory.

T. Summary of Theoretical Findings

The analysis establishes several properties without executing the benchmark. First, lookup is deterministically bounded by $d + s$ logical storage examinations. Second, the primary displacement path is deterministically bounded by K eviction events and by a corresponding function of d and K in candidate checks and hash evaluations. Third, every main-table write performed by the eviction mechanism is made at a valid candidate location for the newly written key, so the committed placement invariant is locally preserved during normal insertion.

The analysis also establishes what cannot be claimed from the source alone. A fixed K does not establish successful insertion; a finite stash does not establish negligible failure probability for the concrete hash family; a classical d -ary load threshold does not establish bounded online search; and a finite recovery depth gives termination but not insertion atomicity. The supplied recovery implementation can lose a previously stashed key on the terminal ‘-4’ path because extraction and reinsertion are not transactional. This is a concrete correctness limitation rather than a probabilistic qualification.

Finally, the analysis shows why the proposed structure is best characterized as an *engineering policy for bounding the normal insertion path* rather than as a new asymptotic cuckoo-hashing theorem. That characterization is fully compatible with the existing theoretical literature, which already contains stronger insertion-time guarantees for other algorithms and models [13]–[16]. The scientific question left for the experiments is whether the simpler bounded-displacement policy provides a favorable finite-table trade-off among lookup cost, primary insertion work, stash utilization, and recovery frequency in an exact-match workload.

VIII. COST MODEL AND MEASURABLE QUANTITIES

The proposed structure is intended for environments in which wall-clock latency and logical operation counts are both relevant. A rigorous evaluation therefore requires a cost model that distinguishes abstract data-structure work from machine-level timing. This distinction is especially important in a two-column academic paper because a single scalar such as “insertion latency” can conceal the mechanism responsible for a result.

A. Logical Lookup Cost

For a lookup of key x , define $Q_m(x)$ as the number of main-table candidate slots examined and $Q_s(x)$ as the number of stash entries examined. The total logical lookup cost is

$$Q_{\text{lookup}}(x) = Q_m(x) + Q_s(x). \quad (42)$$

The worst case satisfies

$$Q_{\text{lookup}}^{\max} \leq d + s. \quad (43)$$

For positive main-table queries, $Q_s(x) = 0$. For positive stash queries, $Q_s(x) > 0$ unless the first examined stash entry matches. For negative queries, the complete stash scan is required whenever all d main candidates miss.

This decomposition is preferable to a single “probe” count when evaluating stash-based structures. A main-table probe involves indexed memory access, whereas a stash scan introduces a sequential auxiliary search. On modern processors the latter may be cheap for a four-entry array and expensive for a larger structure, depending on layout and vectorization. The academic analysis should therefore report both components whenever possible.

B. Logical Insertion Cost

Define $K_{\text{obs}}(x)$ as the number of eviction events performed for key x before successful main-table placement or stash routing. The implementation enforces

$$K_{\text{obs}}(x) \leq K \quad (44)$$

for every primary insertion attempt. Let $C_{\text{cand}}(x)$ denote the total number of candidate checks made on the primary path. Because all d candidates are inspected whenever the current key cannot be placed immediately,

$$C_{\text{cand}}(x) \leq d(K + 1), \quad (45)$$

where the extra factor accounts for the terminating round in which a free slot may be found. The implementation may terminate earlier, so Equation (45) is an upper bound rather than a typical count.

The number of actual writes is also bounded on the primary path by a constant multiple of K . This metric is useful because an eviction is not merely a read: it modifies a table location and transfers the displaced key into the temporary state. For hardware implementations, these writes can interact with port availability; for software implementations, they can affect cache-line ownership and store-buffer behavior. The present benchmark does not measure these hardware-level events directly, so such consequences remain design considerations rather than measured findings.

C. Stash Traffic

Three distinct stash metrics are relevant. The first is the number of keys routed to the stash, denoted R_s . The source maintains `stashRoutedCount`, which records successful stash-routing events. The second is the maximum instantaneous stash occupancy, denoted

$$P_s = \max_t |S_{\text{stash}}(t)|. \quad (46)$$

The implementation tracks this quantity in `peakStashOcc`. The third is the amount of work associated with stash recovery, recorded indirectly through the number of `incrementalRehashes` and `stashOverflowEvents` counters.

An important distinction is that `stashRoutedCount` does not equal the number of stash overflows. Many keys may be routed to an empty stash entry without any recovery taking place. Conversely, one recovery event may reintegrate several previously routed keys. The two counters consequently describe different phases of the system’s behavior and should not be collapsed into one failure rate.

D. Wall-Clock Insertion Latency

The benchmark records per-operation insertion latency using `std::chrono::steady_clock`. The resulting sample sequence

$$L_1, L_2, \dots, L_N$$

can be summarized by median, 99th percentile, 99.9th percentile, and maximum values. The supplied benchmark contains a generic percentile reporting function that sorts the latency vector and selects the sample at the requested empirical quantile.

The timing values should be interpreted as end-to-end software measurements of the instrumented function, not as pure algorithmic operation counts. Each sample includes timer invocation overhead, branch behavior, hash-function execution, vector indexing, and the memory-system effects present during the benchmark. A rigorous study should therefore report the measurement environment and, ideally, quantify the timer and instrumentation overhead through a control experiment.

The difference between logical and wall-clock costs can be written abstractly as

$$L(x) = \tau_0 + \tau_h H(x) + \tau_m M(x) + \tau_w W(x) + \tau_b B(x), \quad (47)$$

where H , M , W , and B represent hash evaluations, memory operations, writes, and branch-related work, respectively, and the τ coefficients summarize the machine-dependent cost of those events. Equation (47) is a conceptual decomposition, not a calibrated processor model. Its purpose is to explain why two algorithms with similar logical probe counts can exhibit different timing behavior.

E. Memory Overhead

For a key width of w bits and a main table containing dm slots, the idealized key-storage requirement of the primary table is

$$M_{\text{keys}} = dm w \quad (48)$$

bits, excluding occupancy metadata and allocator overhead. A stash of s entries adds approximately sw bits for keys plus occupancy state. The C++ implementation additionally uses ‘`std::vector`’ objects for each sub-table and occupancy array, and the actual memory footprint therefore exceeds the idealized lower-level expression in Equation (48).

The experimental paper should consequently distinguish *logical storage overhead*, measured as the number of allocated key slots relative to the number of logical keys, from *process memory footprint*, which includes metadata and library allocation structures. The former is a data-structure property; the latter is an implementation measurement.

F. Recommended Result Tables

The formalization suggests three principal empirical tables. The first should compare lookup cost, primary displacement count, and stash traffic among the baselines and proposed structure. The second should sweep K at fixed d , s , and target load. The third should vary the load factor at a fixed K and stash size. Each table should report the number of inserted

keys, number of lookup operations, logical load definition, and the benchmark seed. Wall-clock percentile values should appear only after the corresponding run has been archived.

This presentation avoids the unsupported values removed from the original manuscript. In particular, it prevents a configured load target from being silently relabeled as a measured maximum load, and it prevents an intended number of updates from being represented as an executed operation count without an output record. The distinction is necessary for a journal pre-submission in which reproducibility is itself part of the scientific contribution.

IX. DISCUSSION, LIMITATIONS, AND IMPLICATIONS

The purpose of this section is to interpret the algorithmic and experimental record established in the preceding sections without introducing claims that are not supported by the implemented source, the declared experimental protocol, or the cited theoretical literature. The distinction is particularly important for this study because the proposed data structure combines a standard d -ary cuckoo-hashing placement rule with an explicitly bounded primary displacement budget and a finite stash. Each of these components has well-established conceptual precedents, but their combination in the present implementation does not inherit every guarantee proved for a different algorithm. The discussion therefore treats the contribution as an implementation-specific bounded-work design whose empirical effectiveness remains an experimental question.

A. Principal Interpretation of the Proposed Design

The central architectural decision in the proposed structure is to impose an explicit bound K on the number of primary displacement iterations and to route the unresolved displaced key into a stash rather than extending the primary eviction process indefinitely. This transforms an unbounded or probabilistically long primary insertion path into a path whose iteration count is specified a priori. The source implementation realizes this policy with d candidate positions per key, a randomized choice of one candidate among the occupied positions for each eviction, and a fixed maximum number of displacement iterations. The principal configuration uses $d = 3$, $K = 16$, and a stash of four entries.

The significance of the construction is consequently not that cuckoo hashing has been made inherently constant-time in every operational sense. Classical cuckoo hashing already provides worst-case constant-time lookup under its standard two-choice model, while a substantial literature has studied ways to control insertion cost through random walks, de-amortization, or auxiliary storage [1], [5], [16]. The narrower contribution considered here is the explicit engineering policy that the primary path is never permitted to continue beyond a declared budget. An unresolved insertion is converted into a separately measurable exceptional state. This makes the boundary between the common path and the recovery path explicit in a way that is straightforward to instrument in

software and potentially useful in systems where tail work, rather than mean work, is a dominant design concern.

This interpretation also clarifies what the bounded displacement policy does not establish. A finite K does not imply that a suitable location must be found within K evictions, that a valid cuckoo assignment always exists, or that all recovery operations complete in a fixed amount of time. The first statement is a direct property of the implementation; the latter statements require probabilistic or structural arguments that depend on the input workload, the hash distribution, table occupancy, stash size, and recovery procedure. The cleaned manuscript deliberately avoids conflating these levels of assertion.

B. Relationship to Classical Cuckoo Hashing

Pagh and Rodler introduced cuckoo hashing as a dictionary with worst-case constant lookup time by assigning each key a small set of candidate locations and moving colliding keys among those locations during insertion [1]. This separation of concerns is fundamental: lookup is made structurally simple by restricting a key to a constant number of candidate locations, whereas insertion absorbs the complexity of collision resolution. The present implementation preserves that fundamental principle.

The difference is in how the insertion complexity is exposed to the system. In the source benchmark, the classical two-way implementation continues through a bounded loop and triggers a complete rehash when the loop limit is reached. The proposed implementation also has a bounded primary loop, but the unresolved displaced key is first directed to a finite stash. The two designs therefore differ in the location of exceptional work rather than in the basic cuckoo invariant. Classical cuckoo hashing uses rehashing as the principal mechanism for restoring a feasible placement after a difficult insertion; the proposed implementation attempts to absorb a bounded number of such exceptional keys in auxiliary storage before invoking a recovery procedure.

This distinction is important for interpreting any measured reduction in tail latency. A reduction in the number of primary displacement steps cannot by itself establish that the total cost of an insertion is lower. The cost may have migrated into stash insertion, stash scanning, stash reintegration, or repeated retry. A meaningful comparison must therefore partition insertion work into at least primary displacement, stash routing, recovery, and terminal failure. The experimental protocol in Section X was designed to expose these populations separately.

The comparison is also asymmetric in another respect. The classical benchmark contains a full-rehash operation that changes its two hash functions and rebuilds the table. Such an operation is qualitatively different from a local stash operation because it has work proportional to the number of resident keys. Any summary statistic that averages ordinary insertion times together with a small number of rehash events can therefore produce an apparently benign mean even when the extreme tail is dominated by reconstruction. This is one reason why percentiles and explicit exceptional-event counters are more informative than a single mean insertion latency.

C. Relationship to Stash-Based Cuckoo Hashing

A stash is not a novel concept in cuckoo hashing. Kirsch, Mitzenmacher, and Wieder introduced cuckoo hashing with a stash to reduce the probability of failure by allowing a small number of difficult keys to remain outside the main placement structure [6]. Subsequent analyses refined failure bounds and considered simpler hash families and generalized bucket models [7]–[10]. The present work should therefore be described as an implementation study of a bounded-displacement scheme with stash-assisted recovery, not as the first use of a stash in cuckoo hashing.

The source-level behavior nevertheless differs in an experimentally relevant way. The proposed program does not simply treat the stash as a passive failure buffer. When the stash is full, it extracts the current stash contents, clears the stash occupancy markers, and attempts to reintegrate those keys using the same global hash functions while drawing fresh randomized eviction choices. The hash functions are deliberately kept fixed. This is necessary for correctness because changing the global hash mapping while leaving already-placed main-table entries at their old physical positions would make those entries inaccessible through the new candidate locations. The corrected source comments explicitly identify this invariant.

This recovery rule has two consequences. First, the proposed design does not have the same semantics as a conventional global rehash. The recovery step reuses the existing candidate-location relation and changes only the path through that relation. Second, the stash cannot be treated as an independent cache whose occupancy is unrelated to the main table. Its behavior is coupled to the current placement state. An insertion that reaches the stash indicates that the bounded search policy failed to find a primary placement within the configured budget; it does not imply that no valid placement exists.

The distinction is especially important near a structural load threshold. A valid assignment may exist in the underlying cuckoo graph even when the particular bounded randomized walk followed by the implementation does not find one within K iterations. Conversely, a stash can postpone visible failure after a difficult placement event without changing the underlying feasibility of the main table. Therefore, stash-routing probability is an algorithm-dependent observable rather than a direct estimator of the mathematical threshold for cuckoo-hash orientability.

D. Relationship to d -Ary Load-Threshold Results

The literature on d -ary cuckoo hashing establishes sharp threshold phenomena for the existence of valid placements under random hash-location models. For the offline model with one key per location, the threshold depends on the number of choices and can be characterized using random-hypergraph cores and related formulations [11], [12]. These thresholds are properties of an assignment problem: they describe when a placement exists with high probability as the table size grows under the specified random model. They are not, by themselves, guarantees about the runtime of a particular online insertion heuristic.

This distinction is central for the present study. The principal configuration uses three candidate positions and targets an aggregate occupancy of approximately 0.90. The fact that the three-choice one-slot cuckoo model has a theoretical threshold in the vicinity of 0.918 is therefore a useful reference point, but it does not imply that the implemented bounded-displacement algorithm will successfully maintain an arbitrarily large finite table at 0.90, nor does it imply a particular stash-routing probability. Finite-size effects, the exact candidate-generation function, insertion order, bounded search policy, randomized eviction decisions, and the finite stash all affect observed behavior.

The distinction between the structural threshold and the algorithmic operating point also avoids an invalid inference from a successful finite benchmark. If a run completes at 0.90 load without a terminal failure, the correct interpretation is that the tested instance was successfully handled by the implemented procedure. It is not evidence that every random instance below 0.90 succeeds, nor is it evidence that the asymptotic threshold is exactly 0.90. Conversely, an observed stash event below the asymptotic threshold is not evidence that the mathematical placement problem is infeasible. It may indicate only that the bounded search budget was insufficient for that local configuration.

The relationship becomes even more important as d increases. Recent analytical work has established strong insertion-time results for particular random-walk d -ary cuckoo-hashing procedures under appropriate load conditions. Walzer showed constant expected amortized insertion time below the peeling threshold, and Bell and Frieze subsequently established an $O(1)$ expected insertion-time bound up to the load threshold for a broad class of random-walk d -ary schemes [13], [14]. These results are highly relevant context, but they analyze a particular random-walk process rather than the finite-budget, stash-recovery process implemented here. The present manuscript therefore treats them as neighboring theoretical results rather than as premises for a theorem about the proposed algorithm.

E. Relation to High-Load Cuckoo-Hashing Research

High-load cuckoo hashing illustrates why a simple load-factor argument is insufficient. Kuzmaul and Mitzenmacher developed bubble-up cuckoo hashing to obtain strong expected insertion guarantees at load factors approaching the high-load regime while controlling the number of hash locations used [15]. Their construction demonstrates that insertion behavior near a high occupancy threshold can be improved through careful management of where and when candidate positions are explored. It therefore provides a useful conceptual comparison for the present bounded-displacement policy.

The similarity is that both designs recognize that insertion cost is not completely characterized by the existence of a free candidate location. The sequence of candidate evaluations and evictions matters. The difference is that bubble-up cuckoo hashing derives its guarantee from a specialized algorithmic structure and probabilistic analysis, whereas the present im-

plementation explicitly truncates the primary eviction path and exposes a stash as a recovery mechanism. The bounded implementation is consequently simpler to reason about at the local control-flow level, but it provides a weaker basis for broad asymptotic claims unless an additional stochastic analysis is supplied.

This comparison suggests a useful research interpretation. The bounded algorithm can be viewed as a systems-oriented mechanism for limiting the amount of sequential eviction work that any one insertion is allowed to consume. The high-load literature, in contrast, studies how the placement process itself can be designed so that difficult configurations are unlikely or quickly resolvable. The two objectives are complementary. A future theoretical study could ask whether an appropriately randomized bounded policy inherits a nontrivial failure bound as a function of d , K , s , and the load margin. The present paper should not claim that result without proving it.

F. Interpretation for Exact-Match Flow Tables

The intended application domain motivates the emphasis on bounded exceptional work. Exact-match networking tables are commonly used for connection tracking, firewall state, NAT mappings, flow monitoring, and related per-flow state. Cuckoo-based structures have been investigated specifically for such workloads because they can keep positive lookup work small while avoiding the pointer chasing associated with chained hashing [18], [19]. Hardware-oriented studies similarly show that insertion behavior becomes an important implementation consideration as table occupancy increases [20].

For such applications, a bounded primary displacement policy has an interpretable systems advantage: the common insertion path has a declared upper limit on sequential eviction iterations. This may simplify resource allocation in software, pipeline design in specialized hardware, or admission control in a control-plane process that must avoid unexpectedly long critical sections. However, the bound is meaningful only when the exceptional paths are also provisioned. If the stash is full and reintegration repeatedly fails, the true service time can exceed the nominal primary budget by an amount that depends on recovery behavior. Therefore, the appropriate systems quantity is not merely K , but the distribution of total insertion work conditioned on normal and exceptional paths.

The networking literature further emphasizes the importance of memory behavior. Cuckoo++ demonstrated that cacheline organization, metadata locality, batched lookup, and selective prefetching can materially influence throughput in software packet-processing systems [19]. The present benchmark is not a packet-processing benchmark and therefore cannot establish line-rate packet-processing performance. Its logical probe counts are useful as algorithmic measurements, but they should not be translated into millions of packets per second without controlled hardware experiments.

This limitation is not merely procedural. The supplied benchmark uses scalar C++ containers and generic dynamic-memory abstractions. Such a program does not model an

SRAM-backed pipeline, an FPGA memory architecture, an ASIC hash unit, or a DPDK-optimized packet-processing loop. It also does not measure concurrent access, NUMA effects, cache-line contention, packet batching, or NIC queueing. The networking relevance of the design should therefore be stated as a motivation and architectural hypothesis until a platform-specific prototype is evaluated.

G. Interpretation of the 90% Target Load

The use of a target load near 0.90 is intended to explore a regime in which memory utilization is substantially greater than that of the classical two-way cuckoo configuration while remaining below the known three-choice asymptotic placement threshold. As an experimental target, this is reasonable because it creates a nontrivial stress point for bounded insertion. It should not, however, be described as an intrinsic capacity of the proposed structure.

There are at least four distinct quantities that can be confused here. The first is the aggregate ratio of resident keys to all primary slots. The second is the occupancy of a particular sub-table. The third is the fraction of keys that can be assigned under the random candidate-location model. The fourth is the empirical fraction of insertions that the finite-budget algorithm can complete without using the stash. These quantities coincide only under additional assumptions. The benchmark code explicitly computes load as the number of resident keys divided by dm , which is an aggregate occupancy over all d sub-tables. It is this quantity that should be used consistently in the manuscript.

The finite stash also changes the interpretation of utilization. If s keys are resident in the stash, the data structure is effectively storing more than its main-table placement state, even though the benchmark’s load calculation includes those keys in the numerator. This is appropriate for measuring total resident occupancy, but it means that the observed load can slightly exceed the fraction represented by main-table slots alone. A complete analysis should therefore report both aggregate load and stash occupancy rather than treating load as a single scalar descriptor.

H. Interpretation of the Displacement Bound K

The displacement parameter K is the most direct control variable in the proposed design. Increasing K permits the insertion procedure to explore a larger portion of the local cuckoo graph before routing the unresolved key to the stash. Decreasing K produces a more aggressively bounded primary path but should generally increase the fraction of operations that require fallback, all else equal. This trade-off is a structural hypothesis of the algorithm and does not require a benchmark result to state.

The sensitivity experiment uses $K \in \{4, 8, 12, 16, 24\}$ at a declared target load of 0.88 with $d = 3$ and a four-entry stash. The purpose of this sweep is not to identify a universal optimal value. Rather, it characterizes how the local control parameter changes the observed division between ordinary primary placement and exceptional stash activity. A value

of $K = 16$ should therefore be presented as the principal experimental configuration selected for evaluation, not as a theoretically optimal constant.

A particularly important consideration is that the implementation counts an eviction as a “kick” only after an occupied candidate position has been selected and replaced. Before that choice, each iteration checks up to all d candidate locations for an empty slot. Consequently, the relation between K and actual memory accesses is not one-to-one. A nominal bound of sixteen iterations can involve as many as dK candidate-position checks in the primary search, in addition to the selected evictions and associated metadata access. This distinction should be retained in any performance model or hardware mapping.

I. The Stash as an Exception-Handling Mechanism

The stash should be understood operationally as an exception queue rather than as an ordinary extension of the main table. Under normal insertion, the algorithm attempts to preserve the principal invariant that every resident main-table key occupies one of its candidate positions. Only after the bounded displacement budget has been consumed does the currently displaced key become a stash candidate. This structure naturally partitions insertions into a high-volume normal population and a low-volume exceptional population.

This partitioning is useful for systems design because it makes the frequency of exceptional work observable. A stash-routing counter measures how often the primary policy reaches its configured limit. A stash-overflow counter measures how often all stash entries are occupied when an additional fallback is required. An incremental-recovery counter measures how often the auxiliary recovery mechanism is invoked. These quantities answer different questions and should not be collapsed.

The four-entry stash used here is intentionally small. Its value is therefore not that it can absorb arbitrary placement failures, but that it creates a finite buffer between a bounded primary path and a more expensive exceptional path. The design is analogous to a bounded queue in a real-time system: queue capacity can reduce the probability that an isolated burst immediately propagates into a global recovery action, but queue capacity does not eliminate the possibility of overflow under sustained pressure.

This observation also suggests a more complete future control policy. Instead of treating stash overflow as a rare pathological event, an implementation could monitor a moving estimate of stash occupancy and initiate maintenance before the stash becomes full. Such a policy would change the algorithm and would therefore require a separate correctness and complexity analysis; it is not part of the present source and is mentioned only as a future direction.

J. Hash-Function Assumptions and Their Consequences

The theoretical cuckoo-hashing literature often assumes random or suitably independent hash functions because the

associated placement graph is then amenable to probabilistic analysis [8], [12]. The supplied implementation uses a splitmix64-style mixing function with seeded instances. This is a practical deterministic hash construction, not a literal implementation of independent random functions drawn afresh for each key. It may be empirically adequate for the intended synthetic workload, but that adequacy cannot be converted into a theorem without analyzing the hash family actually used.

This point has two implications. First, the paper should distinguish “random seed” from “random hash function.” A pseudorandom generator seeded with a fixed value produces a deterministic sequence. Second, reproducibility and probabilistic independence are different properties. A fixed seed is desirable for reproducibility, whereas multiple independent seeds are desirable for estimating how sensitive the measured statistics are to the realized hash mapping.

The source also contains a useful correctness invariant: the proposed global hash functions are kept fixed over the table’s lifetime during stash recovery. This is not an abstract theoretical requirement of cuckoo hashing in general; it is a requirement induced by this particular representation because physical positions are not tagged with the hash-function version that produced them. If the global hash functions were changed without reassigning every resident main-table key, lookups would no longer probe the locations containing those keys. A future implementation that supports true incremental hash migration would therefore require versioned placement, dual-hash lookup, or a complete migration protocol.

K. Interpretation of Timing Measurements

Wall-clock latency measurements should be interpreted as properties of the program and machine used for the experiment rather than as universal properties of the abstract data structure. Even when the algorithm executes a bounded number of source-level operations, memory latency, cache state, branch prediction, compiler scheduling, operating-system interruptions, frequency scaling, and background activity can change elapsed time substantially.

This distinction is particularly important for a claim involving “worst-case latency.” A source-level bound on the number of eviction iterations is a deterministic algorithmic statement. It is not a deterministic nanosecond bound on a commodity CPU. A hardware implementation can, in principle, convert a bounded number of accesses into a cycle bound if the memory architecture and pipeline schedule are themselves bounded, but that requires a hardware model or implementation artifact that is outside the current study.

The benchmark measures insertion latency by starting a high-resolution steady-clock timer around each insertion. This provides per-operation samples, which are useful for percentiles, but it also introduces measurement overhead. For a rigorous microbenchmark, the overhead of the timer pair should either be estimated independently or suppressed through batching and amortization. The present paper therefore treats measured nanoseconds as experimental quantities whose

interpretation depends on the final benchmark environment and does not use them to support an algorithmic theorem.

L. Statistical Interpretation and Reproducibility

The benchmark uses a large finite workload, but a large sample count does not eliminate experimental uncertainty. The input keys are generated once and inserted in a fixed order. A result obtained from one key sequence and one hash seed characterizes that instance of the algorithm. It does not, by itself, characterize the distribution of all possible key sets or insertion orders.

For this reason, seed replication should be treated as part of the final study rather than as an optional cosmetic extension. The quantities of greatest interest for replication are stash-routing count, stash-overflow count, recovery count, terminal-failure count, maximum primary displacement, and latency percentiles. Seed-to-seed variation is itself scientifically relevant: large variation may indicate operation near a structural or algorithmic transition and may reveal sensitivity that an average alone would conceal.

The same principle applies to confidence intervals. Percentiles such as the 99th and 99.9th percentile describe the empirical sample and should not be reported with unjustified asymptotic certainty. A bootstrap interval or a seed-level interval can be supplied once repeated runs are available. For rare-event counters, zero observations should be reported as zero observations in the tested sample, not as evidence that the underlying probability is zero. This is especially important for claims such as “no rehashes occurred.”

M. Threats to Construct Validity

The first construct-validity threat is the use of logical probe counts as a surrogate for latency. Probe count is a useful architecture-independent measure, but a probe is not a constant-cost event on a modern memory hierarchy. A second threat is the use of generic C++ containers as a proxy for a networking hash table. Vector-of-vectors, occupancy arrays, and dynamic allocation expose the benchmark to memory-management behavior that may not exist in a tightly packed hardware or packet-processing implementation.

The third threat is the synthetic workload. The benchmark describes its random 64-bit keys as “synthetic 5-tuple-like flow IDs,” but the generated values are simply distinct uniformly produced 64-bit words; no packet header semantics, flow-size distribution, temporal locality, or burst structure are represented. Therefore, the workload should be described as synthetic rather than as a trace-derived networking workload. Claims concerning real flow-table traffic require actual trace evaluation.

A fourth threat is the positive-only lookup phase in the supplied benchmark. Because every lookup key is selected from the previously inserted key set, the measured lookup distribution emphasizes successful searches. Negative lookups have a different cost profile because they must exhaust the candidate set and, for the proposed implementation, the stash. A complete systems evaluation must therefore include both positive and negative queries.

N. Threats to Internal Validity

One internal-validity issue is the presence of multiple benchmark structures with different capacity targets. Linear probing is configured at approximately 0.70 load, chained hashing at approximately 1.00, classical two-way cuckoo at approximately 0.495, and the proposed three-way structure at approximately 0.90. These are intentionally different operating points, but a direct row-by-row statement that one structure “outperforms” another would be difficult to interpret without acknowledging the unequal occupancy conditions. The cleaned manuscript therefore treats these configurations as comparative stress points rather than as a single controlled-factor experiment.

A second issue is the use of different seeds for some classical-cuckoo experiments and the common seed for the principal configurations. Seed changes are legitimate experimental factors, but they should not be hidden inside a single table. The final numerical study should disclose seed choice in the caption or methods and use a declared seed set for comparative claims.

A third issue is instrumentation allocation. The benchmark stores per-insertion latency vectors, generated keys, and auxiliary state. These allocations can change memory pressure and cache behavior. If the objective is to estimate steady-state data-structure latency, the benchmark should separate an instrumented correctness run from a production-style throughput run or use a sampling strategy that limits instrumentation overhead.

O. Threats to External Validity

The proposed results, once measured, should not be generalized automatically to arbitrary networking workloads. Flow tables in production systems differ in key width, value width, aging policy, deletion frequency, concurrency model, access locality, table size, and memory technology. A data structure that behaves well on uniformly random 64-bit keys may behave differently under structured or adversarial key distributions.

The same caution applies across hardware platforms. A CPU benchmark cannot be used to infer FPGA frequency, ASIC area, SRAM bandwidth, or packet-processing throughput. Networking-specific prior work demonstrates that cache layout, SIMD, batching, and prefetching can materially alter the performance of cuckoo hash tables [19]. The present work should therefore frame platform portability as a future evaluation requirement.

P. Security and Adversarial Considerations

Although the intended application is networking, the present implementation should not be characterized as adversarially robust. An attacker who can infer or influence the hash mapping may be able to construct keys that produce localized contention, increasing stash use or forcing repeated recovery. The literature on robust cuckoo hashing explicitly studies stronger adversarial models and shows that robustness may require additional query or storage trade-offs [21].

The synthetic random-key benchmark does not test this scenario. In particular, its key-generation procedure is independent of the fixed hash seeds used by the hash-table instances. Therefore, successful behavior under this workload says nothing about resilience to a workload selected after observing the hash function. A networking deployment that exposes the data structure to untrusted traffic would need a separate adversarial evaluation and, potentially, a secret or periodically rotated hash-family construction. Such a modification would also need to respect the placement-version invariant discussed above.

Q. What the Present Study Can and Cannot Claim

The cleaned manuscript supports a compact hierarchy of claims. At the lowest level, it can state deterministic facts about the source code: the number of candidate positions is d , the primary displacement loop is bounded by K , and the stash capacity is s . At the next level, it can derive deterministic logical work bounds for the primary path. At the theoretical level, it can cite published threshold and insertion-time results for related cuckoo-hashing models. At the empirical level, it can report only observations generated by an executed benchmark with a declared environment and workload.

The study cannot, without additional analysis, claim that the proposed structure has deterministic successful $O(1)$ insertion time for an unbounded sequence of operations. It also cannot claim a negligible stash-failure probability solely from a four-entry stash, because stash size and failure probability depend on the hashing model and algorithmic process. The same principle applies to memory overhead, hardware area, and line-rate suitability: those quantities require the corresponding implementation or measurement artifact.

This hierarchy is not merely a stylistic restriction. It changes the scientific question from “Does the algorithm achieve all advertised properties?” to “Which properties are guaranteed by the implementation, which are supported by prior theory, and which are observed under a stated experimental protocol?” The latter is a falsifiable formulation and permits reviewers to distinguish an implementation contribution from a theorem.

R. Implications for Future Algorithmic Development

The present design suggests several directions for a subsequent theoretical paper. The first is to model the bounded-randomized eviction process as a stopped walk on the cuckoo graph and derive a bound on the probability that a key is not placed within K iterations. Such a bound would directly connect K to stash-routing probability and would be more informative than importing a generic cuckoo threshold.

The second direction is to analyze the interaction between stash capacity and recovery. A finite stash introduces a two-level process: the main table is updated continuously, while the stash is periodically emptied and its keys are reinserted. A rigorous model would need to account for dependence between stash keys because they are not independent samples after having participated in previous eviction paths.

The third direction is to replace the empirical splitmix64 hash family with an explicitly characterized limited-independence family. Prior work demonstrates that strong

cuckoo-hashing guarantees can sometimes be obtained with simple hash families [8]. Establishing an analogous result for the bounded-displacement policy would materially strengthen the theoretical basis of the construction.

The fourth direction is to study adaptive control of K . Rather than selecting a fixed displacement budget, a system could choose K based on observed stash pressure or the distance to a load threshold. Such an adaptive controller would create a tunable trade-off between normal-path work and exceptional-path frequency. The controller itself would need stability and adversarial analyses, which are outside the current paper.

S. Implications for Hardware and System Design

The bounded primary path has a natural correspondence to finite hardware resources. A hardware designer can reserve storage and control resources for a fixed maximum number of eviction stages, while a stash can be implemented as a small auxiliary structure. The resulting architecture can be reasoned about using a finite-state controller rather than an unbounded loop. This is potentially attractive in FPGA or ASIC contexts where variable-latency control paths complicate timing closure.

However, the present software implementation does not establish such a hardware design. In particular, the source uses dynamically sized C++ vectors, scalar modulo operations, and pseudo-random eviction selection. A hardware prototype would need a concrete hash circuit, a specified multiported or banked-memory organization, a mechanism for detecting and resolving bank conflicts, and a defined interface for stash access. The insertion scheduler would also need to specify whether a displaced key can be examined against all d candidates in parallel or must be serialized.

Consequently, the appropriate hardware implication is architectural: the algorithm exposes a finite primary control budget that could be mapped to a finite pipeline or bounded microsequence. Quantitative claims about clock frequency, throughput, latency, or resource utilization require hardware implementation and should not be inferred from the software benchmark.

T. Interpretation of Potential Experimental Outcomes

Several qualitatively different experimental outcomes are possible, and each would support a different interpretation. If the proposed structure exhibits a low stash-routing rate with a sharply reduced primary-work tail, that would support the hypothesis that bounded displacement can isolate exceptional work without frequently invoking recovery. If stash routing is frequent but recovery is rare, the result would suggest that the stash is acting effectively as a small exception buffer even though the primary budget is relatively aggressive. If recovery is frequent, then the apparent boundedness of the primary path may be offset by auxiliary work, and the architecture would need to be evaluated using total insertion cost rather than K alone.

If the stash fills repeatedly or terminal failure occurs well below the asymptotic three-choice threshold, several explanations would remain possible: finite-table effects, the specific hash family, insertion order, the bounded walk policy, an implementation defect, or the chosen parameter combination. The correct response would be to isolate these factors experimentally rather than attributing the result immediately to a failure of the underlying cuckoo model.

Likewise, if the proposed structure exhibits higher mean insertion latency but a lower extreme tail than classical cuckoo hashing, the conclusion should be stated as a trade-off rather than a universal performance improvement. The design objective is explicitly tail-oriented: it sacrifices some exceptional placement freedom in exchange for an explicit primary work bound. Whether that trade is useful depends on the target system’s sensitivity to mean versus tail latency and on the cost of stash maintenance.

U. Journal-Pre-Submission Positioning

For journal submission, the paper is strongest when positioned as a careful algorithm-engineering study with a clearly delimited theoretical contribution, rather than as a universal replacement for cuckoo hashing. The original draft contained several statements that exceeded the evidence available from the implementation. The revised manuscript instead makes the implementation itself reproducible, identifies a deterministic primary-work bound, places the construction within the existing stash and d -ary cuckoo literature, and specifies the experiments needed to establish its empirical behavior.

A reviewer should be able to answer four questions from the manuscript: what exact algorithm was implemented, what properties follow mechanically from the source, what properties are imported from prior theory, and which quantities remain empirical. The current structure is designed around that separation. This makes a future benchmark result more valuable because the result can be inserted into a framework whose claims are already clearly bounded.

For a journal version, the study would be materially strengthened by adding at least the following evidence before final acceptance of quantitative claims: multiple independent hash seeds, explicit negative-lookup measurements, a genuine load-factor sweep, raw benchmark logs, a controlled machine configuration, and an independent verification that all resident keys remain reachable after every recovery event. These additions would address the main sources of construct, internal, and external validity identified above.

V. Summary of Discussion

The proposed bounded-displacement structure should be understood as a mechanism for controlling the primary insertion path, not as a proof that all cuckoo-hashing insertion complexity has been eliminated. Its main-table lookup structure remains simple and bounded by construction. The finite stash provides a second level of storage for keys that exceed the primary displacement budget, while the recovery procedure

attempts to reintegrate stash entries without changing the global candidate-location functions.

The theoretical literature provides strong reference points: classical cuckoo hashing supplies constant worst-case lookup; stash-based analyses establish that small auxiliary storage can dramatically alter construction-failure probabilities under specified random models; random-walk and high-load d -ary constructions provide increasingly strong insertion-time results; and networking-oriented implementations demonstrate that memory behavior and insertion work matter in practical flow-table systems [1], [6], [8], [13]–[15], [19], [20]. None of those results, however, substitutes for analysis or measurement of the exact bounded-displacement program studied here.

The principal scientific value of the cleaned formulation is therefore its claim discipline. A deterministic source-level bound is presented as such; a probabilistic threshold is attributed to the appropriate model; a measured latency is tied to an execution environment; and a finite-run absence of failure is not elevated into a theorem. This distinction should be preserved through the final conclusion and through any future experimental revision.

X. EXPERIMENTAL METHODOLOGY

The experimental component of this study is designed to evaluate the implemented bounded-displacement structure without conflating configured parameters with observed behavior. This distinction is necessary because the source program contains both deterministic configuration choices and measurements that depend on the execution environment, key sequence, hash-state evolution, allocation behavior, and the occurrence of exceptional insertion paths. Consequently, an experimental result is admissible as a manuscript claim only when the corresponding operation has actually been executed, the execution parameters have been recorded, and the reported statistic can be reconstructed from the stored output.

The methodology is deliberately organized around four layers. The first layer specifies the data-structure configuration and workload. The second specifies the logical quantities counted by the implementation. The third specifies the wall-clock measurement procedure. The fourth specifies the acceptance criteria for a run, including functional correctness checks and provenance of the resulting measurements. This structure permits the paper to distinguish what is known from theory, what is fixed by source code, and what is observed in a particular experiment.

A. Experimental Questions

The evaluation addresses four primary questions. First, how does a fixed bound on the number of eviction events change the distribution of insertion work relative to classical two-way cuckoo hashing? Second, how does increasing the number of candidate locations from two to three affect the amount of auxiliary stash traffic when the primary table is operated at a high logical load? Third, how sensitive are stash-routing frequency and insertion cost to the parameter K , the maximum number of displacement events permitted on the primary path?

Fourth, to what extent do logical probe counts correspond to measured software latency on a conventional processor?

These questions are intentionally narrower than a claim that the proposed structure is universally superior to all other dictionary implementations. Classical cuckoo hashing is known for constant worst-case lookup, while research on random-walk, de-amortized, and high-load d -ary constructions has established several distinct insertion-time guarantees under different models and assumptions [1], [5], [13]–[16]. The present experiment therefore evaluates the behavior of one concrete implementation rather than attempting to reproduce or subsume those results.

B. Benchmark Implementations

The comparative benchmark contains four logical data structures. The first is linear probing, included as a representative open-addressing baseline whose probe sequence is contiguous. The second is separate chaining, included to provide a baseline in which collision resolution is represented by a per-bucket sequence rather than by relocation. The third is classical two-way cuckoo hashing, using two tables with one candidate location in each table. The fourth is the proposed bounded-displacement d -ary structure with a finite stash.

The classical cuckoo implementation uses the two-table representation in the source harness, with a configurable displacement-loop bound followed by global rehashing when that bound is exhausted. This implementation should be interpreted as the benchmark’s concrete realization of a two-choice cuckoo baseline; it is not a claim that every implementation of classical cuckoo hashing uses the identical loop-control rule. The proposed implementation uses three independently seeded candidate functions, a primary eviction bound, a four-entry stash in its principal configuration, and randomized victim selection among the candidate tables. The comparison is consequently an algorithmic microbenchmark rather than a byte-for-byte comparison of production networking libraries.

The networking motivation is justified by prior work showing that cuckoo-based structures are used in flow tracking and related packet-processing systems, where a small number of candidate memory locations can be beneficial for lookup performance [18], [19]. The present benchmark, however, runs in user-space C++ and does not emulate an ASIC, FPGA, DPDK polling loop, or packet-processing pipeline. Hardware-specific throughput claims therefore fall outside the scope of the experiment.

C. Workload Construction

The supplied harness generates $N = 2,000,000$ distinct unsigned 64-bit keys from a pseudorandom generator initialized with seed 42. The keys are created by repeatedly drawing from `std::mt19937_64` and rejecting duplicates. This makes the benchmark workload reproducible at the sequence level, subject to the C++ implementation and standard-library behavior of the execution platform. The generated keys are described in the source as synthetic “5-tuple-like” flow identifiers; they

are not literal network five-tuples and should not be reported as traces collected from a network.

The benchmark defines 2,000,000 subsequent lookup operations for the principal comparative experiment. Those lookups are drawn cyclically from the inserted key sequence and therefore constitute positive-query measurements. A separate negative-query workload is not presently generated by the source program. Consequently, negative-query latency and false-miss behavior should not be presented as measured results until an explicit negative-query workload has been added and executed.

The absence of a real traffic trace is methodologically important. A uniformly random 64-bit key sequence does not reproduce temporal locality, heavy-hitter behavior, burstiness, correlated flow arrivals, skewed source/destination prefixes, or churn characteristics found in operational network traces. Such properties can materially affect cache behavior and collision structure. Accordingly, the present workload should be characterized as a synthetic microbenchmark, and any claim concerning production traffic must be deferred until a trace-based experiment is performed.

D. Capacity and Load-Factor Definitions

The benchmark uses different capacity rules for its baselines, reflecting the different representations of the structures. Linear probing is sized so that $N/C \approx 0.70$, where C is its single-table capacity. Chained hashing uses $C = N$, so that its nominal bucket-array load is approximately one. Classical two-way cuckoo hashing uses two arrays of size m , and its reported logical load is

$$\alpha_2 = \frac{n}{2m}. \quad (49)$$

The proposed d -ary implementation uses d arrays of size m , giving

$$\alpha_d = \frac{n}{dm}. \quad (50)$$

The source sizes the principal proposed experiment accordingly to

$$m = \left\lceil \frac{N}{\alpha_{\text{target}} d} \right\rceil, \quad \alpha_{\text{target}} = 0.90, \quad d = 3. \quad (51)$$

Because integer truncation is applied, the realized ratio need not equal the target value exactly. More importantly, the realized final ratio must be computed from the number of keys successfully represented by the structure, not from the requested insertion count alone. A reported “90% load” is therefore an experimental observation only when the measured final occupancy and allocated capacity have been recorded.

The sensitivity experiment fixes $d = 3$, stash capacity $s = 4$, and target load $\alpha = 0.88$, and varies K over $\{4, 8, 12, 16, 24\}$. These are configured experimental points, not measured results [13], [14]. The load threshold literature provides a useful theoretical reference because valid cuckoo assignments exhibit sharp transitions in random-hypergraph models, but a threshold is a feasibility statement and cannot be substituted for a measured success rate of a particular online insertion heuristic [11], [12].

E. Primary Experimental Configurations

The principal proposed configuration is

$$(d, K, s, \alpha_{\text{target}}) = (3, 16, 4, 0.90). \quad (52)$$

This point provides a concrete reference configuration for the remaining experiments. It should not be interpreted as an analytically optimal parameter choice. The purpose of fixing one configuration is to permit repeatable comparisons before a larger parameter sweep is attempted.

For the classical two-way baseline, the source contains two distinct operating points. One attempts a logical load near 0.495 and uses $\varepsilon = 1$ in the loop-bound formula embedded in the implementation. The other uses a nominal load near one third and sets the corresponding parameter to 2.03. These settings are useful for exposing different portions of the displacement-cost distribution, but the manuscript should not label either setting as a universal “recommended” operating point unless that statement is tied to a particular source. The purpose of the experiment is to compare the implemented behaviors at documented configurations, not to establish a new optimal load for classical cuckoo hashing.

The sensitivity experiment changes only K while keeping d , the stash size, and the target load fixed. This produces an interpretable parameter sweep: smaller values emphasize hard per-attempt bounds at the cost of potentially more frequent stash routing, whereas larger values permit longer primary searches before the stash is consulted. The expected direction of that tradeoff can be stated as a hypothesis, but the magnitude and even the monotonicity of observed wall-clock latency require measurement.

F. Correctness Validation Before Timing Analysis

A timing result is scientifically useful only if the data structure remains functionally correct during the measured run. The source program currently checks lookup behavior indirectly but does not establish a complete end-to-end insertion-success certificate for every run. Therefore, an accepted benchmark run should satisfy the following validation conditions.

First, every key submitted for insertion must be represented exactly once at the end of the insertion phase. Second, a lookup of every inserted key must return success. Third, a count of successful representations must agree with the expected number of distinct keys. Fourth, stash occupancy and the total number of stash-routed items must be internally consistent. Fifth, any terminal failure code such as -4 must be reported explicitly rather than silently mapped to a successful insertion.

The fifth condition is particularly important for the proposed structure. The stash-reintegration routine extracts the current stash, clears its occupancy markers, subtracts the extracted population from the internal count, and then attempts reinsertion. If recursive recovery reaches the terminal retry bound, the source can return a failure code after that count adjustment. Such an event would require the run to be classified as functionally unsuccessful, even if most latency samples remain numerically available. A failure cannot be treated as merely a

high-latency sample because the corresponding data-structure state may no longer represent the requested set.

The validation phase should therefore be logically separate from the timing phase. A failed correctness check invalidates the corresponding performance summary for claims about successful operations. The raw timing samples may still be retained for diagnostic analysis, but they should not be mixed with successful-run statistics.

G. Insertion-Latency Measurement

The benchmark instruments each insertion by recording timestamps immediately before and after the call to the relevant insertion routine using `std::chrono::steady_clock`. For a sequence of N insertions, let L_i denote the measured duration of insertion i . The empirical distribution is summarized by the median, 99th percentile, 99.9th percentile, and maximum. These quantiles are intended to expose the upper tail that is hidden by the arithmetic mean.

The use of a monotonic clock avoids dependence on wall-clock adjustments, but it does not remove measurement overhead. Each sample contains clock invocation cost in addition to the actual insertion operation. The benchmark also performs per-operation vector insertion into the latency arrays and therefore introduces memory traffic associated with recording the measurement. These effects are acceptable for a first microbenchmark but should be treated as instrumentation rather than algorithmic work.

A stronger experimental protocol should estimate the timer overhead using an empty control region with the same timestamp calls and should record the result as part of the benchmark metadata. Let Δ_{timer} denote the median overhead of that control. An adjusted quantity can then be reported for diagnostic purposes as

$$L'_i = \max(0, L_i - \Delta_{\text{timer}}). \quad (53)$$

The adjusted series should not silently replace the raw observations; both should be retained so that readers can reconstruct the analysis.

For journal-quality measurements, each configuration should be executed for multiple independent trials after the binary and machine have been fixed. The trial mean, median, and tail quantiles should be reported across operations, while variability across trials should be summarized separately. Combining all operations from many trials into a single giant sample can make confidence intervals appear artificially narrow because operations from the same run are not necessarily independent.

H. Warm-Up, Repetition, and Random Seeds

The supplied program creates a deterministic key sequence but uses separate hash seeds for the data-structure instances. The principal seed is `123456789`, while the sensitivity configurations derive additional seeds from the value of K . This makes a single run reproducible but does not constitute a statistical sample over the randomness of the hashing process.

A rigorous extension should therefore distinguish three reproducibility axes: (1) a fixed workload seed, (2) an independently varied hash-instance seed, and (3) a repeated execution on the same machine. These sources of variability measure different phenomena. Changing the workload alters the key sequence; changing the hash seed alters the collision graph; changing the machine or run can alter timing through cache state, frequency scaling, operating-system activity, and allocator behavior.

A practical protocol is to preserve a fixed workload seed for direct reproducibility while running several independent hash seeds for the statistical analysis. The exact seed list should be recorded in the result artifact. This makes it possible to distinguish a statement such as “under seed s the measured stash-routing rate was r ” from the stronger statement “the mean routing rate across independently generated hash instances was $r \pm \sigma$ ”. Only the latter supports a claim about variation across randomized instances.

Warm-up should be treated carefully because the benchmark is an insertion-only program. There is no stable steady-state after all insertions have completed; the table occupancy itself changes throughout the run. Consequently, discarding the first arbitrary number of insertion samples as a conventional cache warm-up would also discard a systematically different load regime. A better procedure is to report latency as a function of insertion index or current load and, where appropriate, analyze separately defined load windows.

I. Lookup Measurements

The lookup benchmark currently measures positive lookups using keys that are known to be in the table. For the proposed structure, a main-table hit returns a logical position in $\{1, \dots, d\}$, whereas a stash hit returns $d + 1$ and a miss returns -1 . This return convention permits the benchmark to distinguish main-table and stash-resident hits without requiring additional instrumentation.

For a positive-query experiment, three quantities should be reported separately: main-table candidate count, stash entries examined, and total logical accesses. A stash-resident key should not be counted as though it were found within the first d candidate locations. This would systematically understate the cost of the exceptional path. The average lookup quantity should consequently be formed from the actual returned access count or, preferably, from separate counters for the main and stash components.

The logical lookup model predicts a worst-case bound of $d + s$ candidate or stash entries for the implemented linear stash scan. That bound should not be interpreted as a processor-cycle bound. Sequential access to four cache-resident stash entries and random access to a main-table slot are different machine-level events. Earlier networking-oriented work likewise demonstrates that cache layout and prefetching can materially change the relation between nominal memory accesses and observed throughput [19].

A complete lookup study should contain negative queries as well. Negative queries are important because the implemen-

tation must establish that no candidate contains the requested key and then scan the stash. A synthetic negative workload can be created by drawing keys from a separate random stream and rejecting accidental matches. Until such a workload has been executed, the manuscript should not make any claim about negative-query latency.

J. Stash-Routing and Recovery Measurements

The source exposes three directly relevant counters: `stashRoutedCount`, `incrementalRehashes`, and `stashOverflowEvents`. These counters should be reported separately because they describe different events. `stashRoutedCount` counts successful assignments to a free stash entry. `stashOverflowEvents` counts attempts made when the stash is already full. `incrementalRehashes` counts invocations of the stash-reintegration routine. One overflow can therefore cause more than one reinsertion operation, and one reintegration can process multiple stash residents.

The principal stash-routing probability for a completed run is naturally

$$\hat{p}_{\text{stash}} = \frac{R_s}{N_{\text{ins}}}, \quad (54)$$

where R_s is the number of successfully routed keys and N_{ins} is the number of successful logical insertions represented by the final table and stash state. When a run terminates with functional failure, the denominator must be handled explicitly rather than silently assuming $N_{\text{ins}} = N$.

The peak stash occupancy is

$$P_s = \max_t |S_t|, \quad (55)$$

and should be reported in absolute entries and, optionally, normalized by the stash capacity as P_s/s . The average occupancy can be summarized either over time samples, over insertion events, or over routing events. These definitions are not equivalent and therefore should not be interchanged. The supplied program accumulates occupancy specifically at successful routing events, which is best described as “average stash occupancy at routing” rather than a continuous-time average.

K. Sensitivity to the Displacement Bound

The displacement-bound study evaluates $K \in \{4, 8, 12, 16, 24\}$. The response variables should include at least the stash-routing rate, peak stash occupancy, number of stash overflows, number of recovery invocations, distribution of observed eviction counts on successful primary placements, and insertion latency percentiles if timing is enabled for that sweep.

The principal statistical comparison is not whether one K value produces the smallest single scalar metric. Instead, the experiment characterizes the trade-off surface induced by K . A smaller K directly limits primary search work but can expose more operations to the stash path. A larger K permits more relocation work before exceptional handling is invoked.

Because stash traffic and insertion latency depend on both load and collision structure, the effect of K must be interpreted at the fixed load and hash-seed conditions of the sweep.

A useful normalized measure is the number of primary eviction events per successful insertion,

$$\bar{K} = \frac{1}{N_{\text{ins}}} \sum_{i=1}^{N_{\text{ins}}} K_i, \quad (56)$$

with the upper-tail quantiles of K_i reported alongside the mean. The maximum cannot exceed the configured primary bound for an individual primary attempt, but this does not imply that complete insertion latency is bounded by the same constant, because stash recovery may invoke additional insertion attempts.

L. Load-Factor Sweep

The next stage of evaluation should vary the target load while holding the algorithmic configuration fixed. For the proposed structure, a suitable grid is a sequence of target loads below the nominal feasibility region, followed by points progressively closer to the high-load regime of interest. The exact values should be chosen after preliminary correctness runs establish that the benchmark completes without violating memory constraints.

For each target load, the table capacity must be derived from the same capacity equation and the realized occupancy must be measured after insertion. The resulting experiment should plot at least three response families: insertion latency versus load, stash-routing probability versus load, and lookup access count versus load. The same hash-seed protocol should be used at each load to separate systematic load effects from seed-to-seed variation.

The experiment should not identify the point at which the implementation first fails as the theoretical cuckoo load threshold. The asymptotic threshold is a property of a random assignment model and is associated with global placement feasibility. A bounded online insertion heuristic can fail below that threshold because its finite displacement search does not find a feasible augmenting sequence. Conversely, a finite benchmark can complete at a load at which an asymptotic failure probability is nonzero simply because the particular finite instance contains no obstructing subgraph. The two observations are compatible and should be reported separately.

M. Comparison With Baseline Structures

Cross-structure comparisons should use the same key sequence and workload size whenever the configuration permits it. However, equal nominal load does not imply equal memory efficiency. Linear probing allocates one contiguous table; classical cuckoo hashing allocates two tables; the proposed structure allocates d tables plus occupancy metadata and a stash. The paper should therefore report both normalized logical occupancy and an estimate of actual allocated bytes.

Lookup comparisons should likewise distinguish logical accesses from wall-clock latency. Linear probing may inspect several adjacent slots but enjoy strong spatial locality. Separate

chaining may involve pointer-like or nested-vector accesses, while the proposed structure performs several indexed candidate checks and may subsequently scan a small stash. Consequently, a simple ordering by logical probe count would not constitute a valid performance ranking.

The experiment should also avoid treating unrelated table implementations as if they share identical optimization levels. A fair software comparison requires a common compiler, common language standard, common optimization flags, common key representation, common measurement clock, and identical workload sequence. Production libraries with custom SIMD, batching, memory prefetching, or hardware instructions can be included as additional baselines, but they should be identified as distinct implementation classes rather than merged with the minimal reference implementations.

N. Statistical Reporting

For each configuration and independent run, the manuscript should preserve the raw operation count and summarize the distribution rather than reporting only a single mean. Let $X_{r,j}$ denote metric X for trial r and operation j . For latency, the primary summaries should include the median, p_{99} , $p_{99.9}$, and maximum within each trial. Across trials, the median of trial medians or a confidence interval around a suitable location estimator can be reported. Tail quantiles should not be assigned a conventional Gaussian error bar without justification because their sampling distributions are generally skewed.

For event probabilities such as stash routing, the observed proportion should be accompanied by the number of Bernoulli trials underlying it. If the study uses multiple independent hash seeds, an interval across seeds may be more informative than an interval computed as though every insertion were an independent draw from a common distribution. The paper should state which experimental unit defines the uncertainty estimate.

When comparing two configurations, effect sizes should be reported in addition to raw values. Suitable examples include the difference in median latency, ratio of p_{99} latency, difference in stash-routing rate, and difference in mean eviction count. Statistical significance tests are optional and should not replace the underlying effect-size description.

O. Measurement of Memory Footprint

The source implementation stores keys and occupancy flags separately. In the proposed structure, each of the d tables contains an array of 64-bit keys and a corresponding array of `char` occupancy markers. The stash adds another key array and occupancy array. The resulting process memory footprint is therefore larger than the idealized key-capacity formula because it includes metadata and allocator state.

Two memory quantities should be reported separately. The first is logical capacity overhead,

$$O_{\text{logical}} = \frac{dm + s}{N}, \quad (57)$$

which describes allocated key positions relative to logical keys. The second is process memory consumed by the benchmark, which should be measured by a repeatable external mechanism and reported in bytes. The latter depends on the standard library, allocator, page size, address-space behavior, and operating system and therefore should not be inferred from Equation (57).

The same principle applies to comparisons with networking hardware. A software process’s DRAM footprint cannot be translated directly into SRAM, TCAM, or FPGA block-RAM usage. Prior networking research explicitly treats memory organization as an architectural design variable [19], [20]. The present study reports process-level and logical memory quantities only.

P. Result Provenance and Artifact Retention

Every reported table entry should be associated with an immutable run record. The minimum record contains the source revision, compiler version, compiler flags, operating system, processor model, number of hardware threads available, workload seed, hash seed, N , lookup count, algorithm name, d , K , stash capacity, target load, realized load, and termination status.

The complete standard error stream emitted by the benchmark should be retained because the program prints percentile and event counters there. Raw key data may be omitted from the publication artifact if the deterministic generation rule and seed are sufficient for reproduction, but the exact source revision must be preserved. If the workload generation method is subsequently modified, the change should create a new artifact version rather than silently overwriting a previous data set.

A useful result identifier can be defined as a hash over the source revision and configuration tuple. For example,

$$I = H(\text{source} \parallel \text{compiler} \parallel \text{flags} \parallel N \parallel K \parallel d \parallel s \parallel \alpha \parallel \text{seed}), \quad (58)$$

where H denotes a collision-resistant identifier function used for artifact management, not as a statistical assumption about the benchmark. The purpose of such an identifier is to make it possible to trace a manuscript table back to a concrete source configuration.

Q. Acceptance Criteria for Publication-Grade Results

A benchmark result should be admitted into the quantitative results section only when all of the following conditions are met: the exact source revision is known; the program terminated normally; the final representation passes the functional validation described above; the configured and realized capacities are recorded; the seed tuple is retained; and the raw output corresponding to the reported statistic exists. A plot or table generated from missing raw output should be marked as unavailable rather than reconstructed from memory or from an expected trend.

This rule applies particularly strongly to the present manuscript because the original submission contained numerical values that were not traceable to a recorded execution.

The corrected paper therefore adopts a conservative reporting policy: configured parameters are stated as parameters, theoretical values are cited to their source literature, and empirical values are stated only after their execution record has been archived.

R. Threats to Validity

The benchmark has several external-validity limitations. The workload consists of synthetic 64-bit keys rather than captured flow records. The implementation is single-threaded. The data structures are written in C++ using standard library containers rather than optimized packet-processing layouts. No FPGA or ASIC implementation is measured. Consequently, the experiment does not by itself establish line-rate performance, hardware timing closure, or production network throughput.

There are also construct-validity limitations. A bounded eviction count is a logical operation constraint, not a direct measurement of CPU cycles. A stash routing event is not necessarily a table-feasibility failure. A theoretical load threshold is not an insertion-time guarantee. Finally, the use of a seeded `splitmix64`-style mixing function does not automatically place the benchmark within every fully random hashing model used in theoretical proofs. These limitations do not invalidate the microbenchmark; they define the questions that its results can legitimately answer.

Internal validity additionally depends on the correctness of the benchmark instrumentation. In particular, recursive stash recovery and exceptional returns must be included in the logical accounting of insertion work. A timing measurement that records a bounded primary-kick value while omitting subsequent recovery work would understate complete insertion cost. The result tables must therefore distinguish primary-path eviction counts from end-to-end insertion latency.

S. Experimental Hypotheses

The methodology supports the following falsifiable hypotheses without assuming their outcomes. First, reducing K should reduce the maximum primary-path eviction count by construction. Second, at a fixed load, reducing K is expected to expose a greater fraction of difficult placements to the stash path, although the measured routing probability depends on the collision graph and eviction policy. Third, increasing d from two to three candidate locations should alter the feasible-load regime and collision structure, but the effect on wall-clock latency depends on the additional hash evaluations and memory accesses. Fourth, stash occupancy should remain small in regimes where exceptional bounded paths are sufficiently rare, but the experiment is required to determine whether a stash of four entries is adequate for the selected finite workload.

These hypotheses are intentionally phrased without an assumed winner among the algorithms. Their purpose is to establish experimental tests that can either support or contradict the design rationale.

T. Planned Presentation of Results

The quantitative results section should begin with a table documenting the exact machine and software environment.

It should then present the comparative microbenchmark, followed by the K -sensitivity experiment, followed by the load-factor sweep. Each result table should include the number of successful insertions, realized load, stash-routing count, recovery count, and at least one upper-tail latency statistic where timing is available.

Figures should emphasize distributions rather than only means. A cumulative distribution of insertion latency can reveal whether a bound primarily removes the extreme tail or simply shifts the median. A plot of stash-routing probability against K can show whether increasing the primary budget merely moves exceptional work from the stash into the main insertion path. A load sweep can reveal threshold-like behavior without asserting that the first observed failure point is a mathematical threshold.

No quantitative interpretation should be made from the shapes of these figures until the underlying runs have been executed and validated. In the present revision, therefore, this section specifies the experiment rather than claiming its outcome.

U. Summary of the Experimental Protocol

The experimental protocol can be summarized as follows. A deterministic synthetic workload of distinct 64-bit keys is generated. The same sequence is used for the comparative structures where capacity constraints permit. The proposed structure is evaluated at a principal configuration of $d = 3$, $K = 16$, and $s = 4$, followed by a sensitivity analysis over the displacement bound and a separate sweep over target load. Each run records logical operation counts and wall-clock timing, and every completed run is subjected to a functional correctness check before its numerical results are accepted.

The protocol intentionally separates the claims that can be made before execution from those that require observation. Before execution, the manuscript can establish the algorithmic bounds on its primary displacement loop and the maximum number of candidate locations examined by a lookup. After execution, it can quantify the actual incidence of stash routing, recovery, insertion latency, and memory consumption for the specified implementation. This separation forms the basis for the subsequent Results and Discussion sections.

XI. RESULTS AND QUANTITATIVE ANALYSIS

The purpose of this section is to define, organize, and interpret the quantitative evidence admissible for evaluating the proposed bounded-displacement data structure. A central requirement of the present revision is that the Results section must not convert configured experimental parameters, theoretical reference values, or claims inherited from the original submission into observations. In particular, a numerical quantity is treated as an experimental result only when it is produced by an executed benchmark run whose configuration, implementation revision, workload, and measurement procedure are retained and can be reconstructed. This distinction is essential in the present study because the supplied

manuscript originally contained numerical latency, routing-probability, stash-utilization, and rehash-frequency values that are not accompanied by corresponding benchmark output.

The source code supplied with the study does contain a reproducible experimental harness. It constructs a finite synthetic workload of distinct 64-bit keys, evaluates linear probing, chained hashing, classical two-way cuckoo hashing, and the proposed bounded cuckoo structure, and records insertion and lookup measurements for several configurations. The harness therefore determines the set of quantities that can be observed from the software artifact. It does not, by itself, establish the values of those quantities before execution. Consequently, all numerical result cells in this revision that depend on execution are marked explicitly as requiring benchmark output.

This presentation also distinguishes three logically different classes of quantitative statements. The first class contains *deterministic implementation properties*, such as the configured upper bound on the number of iterations in the primary bounded-displacement loop. These can be established by inspection of the source. The second class contains *theoretical reference quantities*, such as asymptotic load thresholds established in the literature under specified random-hashing models. Such quantities provide a mathematical context for the experiments but are not measurements of the present implementation. The third class contains *observed experimental quantities*, such as wall-clock latency, realized stash-routing frequency, or the number of rehash events. These can only be stated after executing the exact benchmark under a specified environment.

A. Evidence Status and Result Admissibility

The quantitative claims of the original submission are not all of the same evidentiary status. Some are direct consequences of program structure, whereas others were presented as empirical findings without an accompanying execution record. For the cleaned manuscript, the admissibility rule is therefore as follows: a result is included as an experimental observation only if the benchmark output establishes it, while an unexecuted result is represented by a placeholder or by a derived expression.

This rule has an important consequence for interpretation. The statement that the proposed structure is configured with $d = 3$, $K = 16$, and $s = 4$ is admissible without execution because those are source-level parameters. Likewise, the fact that the primary displacement loop is bounded by K iterations follows from the source. By contrast, a statement such as “the 99.9th-percentile insertion latency is 32 ns” would not be admissible merely because the benchmark contains a percentile-reporting routine. The corresponding value must be generated by an execution of that routine on a defined platform and workload.

Table II summarizes the evidentiary status of the principal quantities used in the study.

The distinction between a *maximum configured count* and a *maximum observed count* is especially important. In the proposed benchmark driver, the reported “maxInsertKicks”

TABLE II
ADMISSIBILITY OF QUANTITATIVE CLAIMS IN THE PRESENT REVISION

Quantity	Status	Basis for inclusion
Number of inserted keys	Configured	Source-level workload parameter
Number of lookup operations	Configured	Source-level workload parameter
d , K , and s	Configured	Source-level data-structure parameters
Maximum primary displacement	Deterministic	Implied by the bounded loop
iterations		
Maximum candidate checks on one primary insertion path	Derived	Follows from d and K
Lookup candidate count	Deterministic	d primary positions plus stash scan
Stash-routing probability	Observed only	Requires execution on a finite workload
Stash-overflow count	Observed only	Requires execution
Insertion latency percentiles	Observed only	Requires wall-clock measurement
Maximum observed kick count	Observed only	Requires execution; distinct from configured K
Full/incremental hash counts	Observed only	Requires execution of exceptional paths
Load threshold of the present implementation	Not established	Requires a threshold experiment and workload scaling study

field is populated from the configuration value K , not from a maximum over all observed successful insertion counts. Therefore, the value shown by the program in that field would establish only the configured cap. An empirical maximum must instead be calculated from the collected per-insertion samples. The cleaned analysis consequently treats the two quantities separately.

A similar distinction applies to latency. The benchmark records per-operation elapsed times using a steady clock, and it later computes percentiles by sorting the sample vector. The resulting values are measurements of the instrumented program on the execution platform. They are not analytical consequences of the $O(1)$ or bounded-operation argument. The analytical section therefore supplies cost bounds, while the present section supplies the framework for turning executed samples into experimental claims.

B. Verified Experimental Configuration

The supplied benchmark defines a principal synthetic workload containing $N = 2,000,000$ distinct 64-bit keys and performs 2,000,000 subsequent lookups. Keys are generated from a 64-bit pseudorandom generator and stored in an explicit set to enforce distinctness. The workload is therefore synthetic and key-level rather than a replay of a packet capture. The source comments characterize the values as “synthetic 5-tuple-like flow IDs,” but the data are represented internally as uniformly generated 64-bit words rather than explicit protocol, source-address, destination-address, source-port, and destination-port fields.

The principal proposed configuration uses three candidate positions per key, a primary displacement budget of sixteen kicks, and a stash with capacity four. The table capacity is

chosen from the target aggregate load factor according to

$$M_{\text{tot}} = dm, \quad \alpha = \frac{N}{M_{\text{tot}}}, \quad (59)$$

where m is the number of slots in each sub-table and d is the number of sub-tables. For the principal target, the benchmark chooses

$$m = \left\lfloor \frac{N}{0.90d} \right\rfloor, \quad (60)$$

subject to the integer truncation imposed by the implementation. The realized load should always be reported in the results rather than replaced by the requested target value.

The sensitivity experiment varies K over the set

$$K \in \{4, 8, 12, 16, 24\} \quad (61)$$

while retaining $d = 3$, stash capacity $s = 4$, and a nominal target aggregate load of 0.88. This experiment is the principal mechanism for testing the design trade-off between bounded primary work and exceptional-path traffic.

The benchmark also includes classical two-way cuckoo configurations at two different nominal operating points and two non-cuckoo baselines. The comparative structures are useful for contextualizing memory access and collision-resolution behavior, but direct comparison is meaningful only after their actual realized loads, capacity conventions, and measurement definitions are reported. In particular, a two-table cuckoo structure with total capacity $2m$ must not be compared using m as though it represented total storage.

C. Reference Values Versus Experimental Values

The literature provides mathematically meaningful reference points for cuckoo hashing, but these should not be conflated with the present experimental results. The original Pagh–Rodler construction provides worst-case constant-time lookup in the two-choice setting, while its insertion process may require repeated displacement and eventual rehashing. Later analyses and algorithms establish improved insertion-time guarantees under different random-walk, de-amortization, or high-load constructions. These guarantees depend on the precise insertion rule and the probabilistic model.

For one-slot-per-location d -ary cuckoo hashing under the standard random-hypergraph model, the offline load threshold is a function of d . For $d = 3$, the classical threshold literature places the critical aggregate density at approximately 0.918 under that model. This number is useful as a reference when the present implementation is operated near 0.90, but it does not imply that the supplied implementation will succeed with probability approaching one at that load. The present program uses a concrete hash family, a bounded randomized eviction rule, a finite table, finite workload, and a finite stash. These implementation details are not identical to the idealized assumptions of threshold theorems.

The distinction is therefore expressed formally as

$$\alpha < \alpha_3^* \not\Rightarrow \Pr(\text{Succ}_{\text{impl}}) = 1, \quad (62)$$

and likewise

$$\alpha > \alpha_3^* \not\Rightarrow \Pr(\text{FailAtFirstInsertion}_{\text{impl}}) = 1. \quad (63)$$

The threshold is an asymptotic statement about the existence of valid placements in a specified random model. The benchmark measures the behavior of one insertion heuristic on one finite realization.

This separation is particularly important for the proposed stash. The corrected generalized stash literature shows that failure probabilities depend delicately on the exact model, including the number of candidate choices, bucket capacity, and stash size. It is therefore not legitimate to insert a general expression such as $\Theta(n^{-d-s})$ into an empirical table and label it the observed stash-overflow probability. That expression belongs to a theoretical model and requires the corresponding assumptions. The benchmark instead reports the finite-sample ratio

$$\hat{p}_{\text{stash}}(K) = \frac{N_{\text{stash}}(K)}{N_{\text{ins}}}, \quad (64)$$

where $N_{\text{stash}}(K)$ is the number of insertions whose bounded primary path terminates by routing a key into the stash and N_{ins} is the number of attempted insertions. Equation (64) is a descriptive statistic for the executed run; it is not an asymptotic theorem.

D. Comparative Microbenchmark: Corrected Result Structure

The principal comparative experiment evaluates four structures: linear probing, chained hashing, classical two-way cuckoo hashing, and the proposed bounded-displacement cuckoo table. A fifth row may be included for an externally implemented cuckoo-cache or networking-oriented structure only if its implementation and environment are actually available for the same experimental run. The original manuscript included such a row with numerical values, but the supplied benchmark does not instantiate that implementation. The cleaned manuscript therefore does not retain those numbers as observations.

The appropriate comparison dimensions are realized load factor, successful insertion count, insertion-latency distribution, logical lookup probes, primary eviction count, stash-routing count, recovery count, and memory footprint. These dimensions should be interpreted jointly. A structure with a smaller median insertion time but a large exceptional tail may behave differently from one with a slightly larger median and a hard primary-loop cap. Conversely, a logical-probe bound does not establish a wall-clock latency bound because cache misses, branch behavior, memory-level parallelism, and timer overhead affect elapsed time.

Table III provides the publication structure for the principal comparison. The table is intentionally incomplete until the benchmark is executed. Its use prevents the manuscript from presenting placeholders as observations while preserving the exact schema that the final numerical experiment must fill.

The use of “TBD” is deliberate. It does not indicate an incomplete theoretical result; it indicates that the corresponding quantity is empirical and has not been established by a

recorded benchmark execution in the present revision. After execution, the table should be populated from machine-readable benchmark output rather than transcribed manually from recollection.

The comparison should also retain the exact capacity convention. For linear probing, the source sets the capacity such that $N/C \approx 0.70$. For chained hashing, the source chooses $C = N$, so the nominal chain-load parameter is approximately one. For two-way cuckoo hashing, the source creates two tables of size m and targets an aggregate occupancy near 0.495. For the proposed design, the aggregate capacity is dm and the nominal target is 0.90. These are distinct storage models, so a single “capacity” column is inadequate unless it is explicitly defined as the total number of addressable slots.

E. Insertion-Latency Distributions

The benchmark records a wall-clock duration for every timed insertion using `std::chrono::steady_clock`. The sample set for each structure can consequently be represented as

$$L = \{\ell_1, \ell_2, \dots, \ell_N\}, \quad (65)$$

where ℓ_i is an elapsed time in nanoseconds for the i th timed operation. The percentile routine sorts the samples and selects the element at index $\lfloor p(N-1) \rfloor$. The reported p_{50} , p_{99} , $p_{99.9}$, and maximum therefore correspond to order statistics of the measured sample rather than to fitted distribution parameters.

The distribution should be reported in more than one form. At minimum, the final analysis should provide the median, upper-tail percentiles, and maximum. A complementary empirical cumulative distribution function is preferable when space permits because it exposes whether the proposed bound modifies only a small extreme tail or changes the distribution across a wider range. The maximum should be interpreted cautiously when the sample contains millions of operations: it is highly sensitive to one exceptional measurement and should never be described as a universal machine-independent latency guarantee.

The principal analytical question is whether the bounded algorithm exhibits a visibly truncated *primary logical-work* distribution and whether that truncation remains visible in end-to-end wall-clock measurements. A hard limit on the number of primary eviction iterations does not mean that every measured insertion takes the same amount of time. Hash computation, memory latency, branch prediction, cache state, allocator state, and stash recovery can all produce variability below the logical bound.

The appropriate comparison is therefore between two objects:

- 1) the deterministic upper bound on the primary algorithmic path; and
- 2) the empirical upper tail of the complete instrumented operation.

A claim such as “worst-case insertion latency is $O(1)$ ” is too broad unless the scope of the worst case is specified. What can be proved directly for the primary proposed loop is a constant

TABLE III
PUBLICATION-READY COMPARATIVE RESULT SCHEMA

Scheme	Choices	Realized Load	Successful Inserts	Lookup Cost	$p_{99.9}$ Insertion	Max Observed Primary Work	Exceptional Events
Linear probing	N/A	TBD	TBD	TBD	TBD	TBD	Resize/termination
Chained hashing	N/A	TBD	TBD	TBD	TBD	TBD	Allocation/termination
Classical two-way cuckoo	2	TBD	TBD	TBD	TBD	TBD	Full rehash
Proposed bounded cuckoo	3	TBD	TBD	TBD	TBD	$\leq K$	Stash/recovery

bound on the number of loop iterations when K is fixed. The complete software operation additionally includes stash management and recursive recovery. Consequently, the latency table must report whether the timing measurement includes the full recursive call graph; in the supplied benchmark, the timer surrounds the top-level `insert` call, so the measurement includes recursive recovery triggered by that call.

F. Primary Displacement Work

For each insertion of the proposed scheme, the source records the return value of the insertion routine. A nonnegative return value indicates that the operation terminated during the ordinary path and the value records the number of kicks used before successful placement. A value corresponding to stash routing indicates that the key reached the stash path after exhausting the configured primary budget. The logical-work distribution should therefore be partitioned rather than reduced to a single average.

Define

$$N_{0:K} = |\{i : 0 \leq k_i \leq K\}|, \quad (66)$$

$$N_S = |\{i : \text{insertion } i \text{ was routed to stash}\}|, \quad (67)$$

$$N_R = |\{i : \text{insertion } i \text{ triggered recovery}\}|. \quad (68)$$

Then

$$N_{\text{ins}} = N_{0:K} + N_S + N'_R, \quad (69)$$

where N'_R is not necessarily disjoint from N_S because a single top-level operation can trigger stash recovery and eventually terminate by ordinary placement or by another exceptional route. This observation is why the counters in the final results must be interpreted operationally rather than treated as mutually exclusive categories without a trace-based definition.

For the sensitivity experiment, the principal statistic is the observed stash-routing rate as a function of K . The expected qualitative relationship is that increasing K gives the primary displacement mechanism more opportunities to find an empty candidate location before invoking the stash. Under a fixed workload and fixed load, it is therefore reasonable to test whether $\hat{p}_{\text{stash}}(K)$ decreases monotonically. However, monotonicity is an empirical property of the particular randomized eviction policy and finite realization, not a theorem that follows from the source code alone. Changing K also changes the sequence of random-number draws used by subsequent insertions, so two configurations are not merely nested executions of a common deterministic trace.

The results should therefore report both the raw count and the normalized rate. A confidence interval for a binomial proportion is useful as a descriptive uncertainty measure when

stash-routing events are defined at the level of independent insertion trials, although the underlying stateful process is not strictly independent. For a large finite run, Wilson or exact binomial intervals may be reported as a conservative summary, with the dependency caveat stated explicitly.

G. Sensitivity to K

The K -sensitivity experiment is the most direct empirical test of the central engineering trade-off. Let $K_1 < K_2 < \dots < K_r$ denote the tested bounds. For each value, the experiment records realized load, successful insertion count, stash-routing count, stash-overflow count, recovery count, maximum observed primary kick count, and, where enabled, latency.

The expected direction of the primary-work statistic is fixed by construction:

$$\max_i k_i \leq K_j \quad (70)$$

for every configuration K_j . This is an implementation invariant, not an observed performance result. The meaningful empirical question is where the actual maximum falls relative to the configured cap. If the maximum remains substantially smaller than K_j for most runs, increasing K may provide little practical benefit. Conversely, if a non-negligible fraction of operations reach K_j , the cap is acting as an active control on exceptional displacement chains.

A useful normalized quantity is the cap utilization

$$U_K = \frac{1}{N_{\text{ins}}} \sum_{i=1}^{N_{\text{ins}}} \frac{k_i}{K}, \quad (71)$$

computed only for primary-path operations for which a kick count is defined. The fraction of operations that hit the cap,

$$f_{\text{cap}} = \frac{|\{i : k_i = K\}|}{N_{\text{ins}}}, \quad (72)$$

provides additional information because the mean can remain low even when a small but important tail reaches the bound.

For each K , the final analysis should state whether the stash-routing rate, stash-overflow rate, and insertion tail latency change materially. “Materially” should not be defined after observing the data. A reasonable journal analysis can use predeclared absolute differences, relative changes, and confidence intervals, accompanied by effect sizes that do not depend solely on a significance threshold.

The original submission reported values such as 4.12×10^{-2} , 2.84×10^{-3} , and 4.20×10^{-6} for the routing probability at selected values of K . Those numbers are omitted from the cleaned Results because they are not backed by an execution record in the supplied materials. The correct replacement is a

TABLE IV
REQUIRED OUTPUT FOR THE K -SENSITIVITY EXPERIMENT

K	Load	Routing	Overflow	Max Kicks	$p_{99.9}$
4	TBD	TBD	TBD	≤ 4	TBD
8	TBD	TBD	TBD	≤ 8	TBD
12	TBD	TBD	TBD	≤ 12	TBD
16	TBD	TBD	TBD	≤ 16	TBD
24	TBD	TBD	TBD	≤ 24	TBD

table whose cells are populated directly from the benchmark output. The theoretical discussion may state the expectation that a larger primary budget should generally reduce the need for fallback placement, but it must not substitute an invented numerical curve.

The table deliberately leaves the realized load as a measured quantity because integer rounding in the capacity calculation means that the requested target is not necessarily attained exactly. The same principle applies to every percentile value.

H. Stash Occupancy and Recovery Activity

The stash is not merely a binary success/failure mechanism. Its occupancy distribution determines the amount of auxiliary state that remains outside the primary table and, under the current implementation, the amount of work that can be induced by a stash overflow.

Let S_t denote the number of occupied stash entries after event t . The benchmark collects stash occupancy only at particular routing points. Accordingly, the resulting average is an average over those sampled points, not necessarily the time average over every insertion. The final manuscript should use precise language such as “mean occupancy at routing events” rather than “mean stash occupancy” unless the instrumentation has been changed to sample every operation.

Three stash quantities should be distinguished:

$$S_{\max} = \max_t S_t, \quad (73)$$

$$\bar{S}_{\text{route}} = \frac{1}{N_S} \sum_{t \in \mathcal{R}} S_t, \quad (74)$$

$$N_{\text{overflow}} = |\{t : S_t \text{ was full when routing was required}\}|. \quad (75)$$

Here $\mathcal{R}$ denotes the set of routing events recorded by the source.

The occurrence of an overflow does not necessarily mean that the data structure immediately loses all contents. In the supplied implementation, an overflow invokes a stash-recovery routine that extracts the current stash, clears the stash occupancy markers, decrements the population count by the number of extracted entries, and reinserts the extracted keys under the same global hash functions using fresh randomized eviction choices. The triggering key is then retried. This is materially different from replacing the global hash functions for the whole table and therefore must not be described as a conventional full-table rehash.

The result presentation should consequently include both the number of recovery attempts and the number of top-level

operations that caused them. A recovery event can be rare yet expensive relative to a normal insertion, even when its logical number of touched entries is bounded by a constant stash size. The appropriate performance interpretation is thus a two-component tail model: ordinary bounded displacement plus an exceptional recovery path.

For the corrected implementation, the total number of stash entries that can be processed by one recovery invocation is at most the configured stash capacity s , while the triggering key is subsequently retried. However, because the recovery calls the insertion routine recursively, repeated overflow can produce a recursion depth bounded by the source-level retry limit of twenty. This gives a deterministic bound on recursion depth within the implemented control structure, but it does not establish that every possible finite state will recover successfully. In fact, the source contains a terminal exceptional return for a pathological stuck-stash condition, and the empirical absence of that path can only be established by execution.

I. Lookup Analysis and Observed Probe Counts

The primary proposed lookup checks at most one candidate slot in each of the d sub-tables and then scans the stash. The logical candidate count is therefore bounded by

$$C_{\text{lookup}} \leq d + s. \quad (76)$$

For the principal configuration, this evaluates to $3 + 4 = 7$ logical slot comparisons in the worst case, although a positive lookup normally terminates earlier when the key is found in the main table or stash.

The benchmark’s lookup-return convention distinguishes primary-table hits from stash hits. A main-table hit returns an index in $\{1, \dots, d\}$, a stash hit returns $d + 1$, and a miss returns -1 . For aggregate reporting, a positive lookup cost should therefore preserve the stash-hit category rather than silently truncating it to d . The cleaned benchmark uses this interpretation when producing its aggregate lookup statistic.

Observed lookup counts on a synthetic positive-only workload should not be interpreted as a complete characterization of flow-table lookup behavior. Positive lookups are substantially different from negative lookups because a positive search may terminate early, while a negative search must examine all candidate positions and then the stash. A publication-grade experiment should therefore report at least one positive and one negative lookup distribution when the benchmark is extended.

The original numerical claim that the proposed structure has an average lookup cost of 1.28 is removed because no executed benchmark output supporting this quantity is present. The appropriate replacement is the pair of quantities

$$\hat{C}_+ = \frac{1}{N_+} \sum_i C_i^{(+)}, \quad \hat{C}_- = \frac{1}{N_-} \sum_i C_i^{(-)}, \quad (77)$$

for positive and negative searches, together with the observed fraction of queries resolved in each candidate position. This provides a much more informative description of the lookup behavior than one scalar average.

J. Comparison of Logical and Wall-Clock Measurements

An important methodological distinction is required between logical probe counts and elapsed time. The former are architecture-independent measures of algorithmic work; the latter are implementation- and environment-dependent measurements. Two insertions that each perform eight logical candidate checks can have markedly different execution times when one accesses hot cache lines and the other incurs multiple cache misses.

This distinction is especially important for a networking-oriented argument. A deterministic logical bound can support a hardware scheduling argument when the memory system itself is provisioned to meet the corresponding access pattern. On a general-purpose CPU, however, the source code cannot guarantee a nanosecond-scale wall-clock upper bound solely from a fixed loop count. The operating system, scheduler, interrupts, memory subsystem, frequency scaling, and concurrent activity all remain external sources of timing variation.

Accordingly, the cleaned results should use formulations such as “bounded primary displacement work” and “bounded logical lookup work.” A statement about “deterministic insertion latency” should be reserved for a specific execution model in which memory-access costs and recovery-path behavior have also been bounded. This wording is more conservative than the original manuscript and more precisely reflects what can be established from the implementation.

K. Memory-Overhead Analysis

Memory overhead can be reported at two levels. The first is the logical storage requirement of the data structure. For the proposed layout, the main tables require dm key positions plus occupancy metadata, while the stash requires s additional entries. If each key occupies b_k bytes and each occupancy indicator contributes b_o bytes, a simplified logical model is

$$M_{\text{logical}} = dm(b_k + b_o) + s(b_k + b_o), \quad (78)$$

with additional implementation-specific state for hash-function objects, allocators, vectors, and object alignment.

The second is process-level resident memory. The benchmark allocates several large structures sequentially during the same process and may retain auxiliary vectors such as insertion-latency samples and generated keys. Consequently, a resident-set measurement of the entire benchmark process would include workload storage and instrumentation state, not only the hash table.

The original manuscript’s claim that a four-entry stash incurs less than 0.01% hardware area overhead is removed from the empirical Results because no synthesis report, FPGA implementation, ASIC gate estimate, or device-specific resource utilization report is supplied. A hardware-area claim requires a hardware artifact or a documented synthesis flow. The logically stronger statement is simply that a constant-size stash contributes $O(s)$ auxiliary storage and, for fixed s , $O(1)$ logical storage independent of the number of table entries.

L. Load-Factor Sweep and Feasibility Curves

The central performance question is not whether one particular target load can be inserted successfully once, but how the finite implementation behaves as occupancy approaches the region where valid placements become difficult. A load-factor sweep should therefore be conducted by varying capacity while holding the key-generation procedure and insertion order fixed.

For a load target α , the benchmark can select

$$m(\alpha) = \left\lceil \frac{N}{\alpha d} \right\rceil, \quad (79)$$

and then compute the realized load after insertion. The sweep should cover both a clearly subcritical region and a region approaching the theoretical $d = 3$ threshold. Because the present study uses a finite N , the resulting success curve will be a finite-size transition rather than an exact mathematical threshold.

The principal response variables should include completed insertion count, final occupancy, stash-routing frequency, stash-overflow frequency, recovery frequency, and latency percentiles. An informative plot is the probability of a completed insertion sequence against target load. Another is the routing probability against load. A third plot can show the upper tail of insertion latency. These plots reveal whether exceptional work appears gradually or concentrates near a narrow occupancy regime.

The benchmark should not report a “maximum sustainable load” merely as the largest tested value with zero observed failures. Such a quantity depends on the step size of the sweep, workload realization, hash seeds, and the definition of failure. A statistically meaningful threshold estimate requires repeated runs and a declared decision rule. Without those, the correct terminology is “observed success region” or “tested load range.”

M. Finite-Sample Statistical Treatment

A single run over two million keys provides a large sample for describing the selected workload, but it does not eliminate uncertainty arising from the random workload and randomized displacement decisions. At minimum, the final study should execute multiple independent seeds for each principal configuration. The mean of a latency statistic across seeds should not automatically be preferred to pooling all individual operations because the latter gives the largest runs disproportionate weight when variability differs between configurations.

A suitable hierarchical presentation is to report, for each configuration, per-run medians and upper percentiles followed by the distribution of those per-run statistics. For routing probabilities, the per-run proportion and a seed-level confidence interval can be reported. For latency, robust summaries such as median, interquartile range, and upper percentiles are more informative than a simple mean because the distributions are expected to be skewed.

Bootstrap intervals may be used to characterize uncertainty of a sample percentile, but resampling individual operations

treats state-dependent samples as independent. A more conservative bootstrap would resample at the run or seed level once independent runs are available. The present revision does not claim such intervals because the supplied materials contain no multi-seed execution record.

Effect sizes should likewise be stated in the original units. For example, the absolute difference in $p_{99.9}$ insertion latency between two configurations is directly interpretable, whereas a ratio can become unstable when the denominator is close to zero or when timer granularity dominates the measurement. Both can be reported, but neither should be treated as a universal hardware-independent speedup.

N. Interpretation of the Missing Numerical Results

The absence of executed benchmark output is itself a reproducibility result. It means that the supplied source establishes what the experiment *would measure*, but not what the program *does measure*. This distinction is particularly significant because the original manuscript contained a set of precise decimal values that imply a completed experiment. Those values cannot be reconstructed reliably from source inspection alone.

For example, the original manuscript stated a $443\times$ reduction in the 99.9th-percentile insertion latency. Such a ratio is especially sensitive to both numerator and denominator and cannot be justified by the existence of a bounded loop. Likewise, a claim of zero full-table rehashes across 10^7 updates cannot be established by configuring a stash of size four; it requires a run that actually exercises the exceptional path over the stated number of updates.

The cleaned Results section therefore treats the original numerical table as a *schema* rather than evidence. This is not merely editorial caution. Replacing unsupported values with “TBD” makes the manuscript falsifiable: a later execution can either populate the table consistently with the design hypotheses or contradict them. The resulting paper is therefore capable of reporting a negative result without requiring structural changes to the experimental methodology.

O. Expected Qualitative Behavior Before Execution

Theoretical and implementation analysis supports several qualitative expectations that can be tested empirically. First, increasing K should never increase the configured upper bound on primary displacement work; the upper bound is exactly the chosen constant. Second, for a fixed finite workload and load factor, increasing K is expected to reduce the frequency with which an insertion exhausts its primary budget, although the precise monotonicity of measured routing frequency is not guaranteed because the randomized eviction sequence changes with the configuration. Third, increasing d changes both the available candidate set and the number of primary positions that must be examined on lookup, so its effect on latency is inherently a trade-off rather than a one-directional prediction.

The stash should be interpreted as a mechanism for converting a potentially long primary displacement path into a bounded exceptional path. A low routing rate does not

necessarily imply low overall insertion latency if each recovery event is disproportionately expensive. Conversely, a somewhat higher routing rate may be acceptable if recovery is rare and bounded for the target system. The appropriate experimental objective is therefore not to minimize stash routing in isolation, but to characterize the joint distribution of normal and exceptional work.

At higher loads, the literature predicts that the placement problem eventually enters a regime in which valid assignments become unlikely under the corresponding random model. For $d = 3$, the theoretical one-slot-per-location threshold near 0.918 provides a useful context for the principal target near 0.90. The present implementation may exhibit a different finite-size transition because the source uses a specific hash family and insertion heuristic. A measured transition can therefore be compared with, but should not be identified with, the asymptotic threshold without additional analysis.

P. Result Tables Required for the Final Experimental Release

A publication-grade release should include at least the following quantitative tables. The first is the machine and software environment, including compiler version, optimization flags, CPU model, memory capacity, operating-system version, and whether frequency scaling or CPU pinning was enabled. The second is the principal comparative table. The third is the K -sensitivity table. The fourth is the load-factor sweep. A fifth table may summarize memory usage and allocation behavior.

Each result table should identify whether the statistic is per operation, per insertion sequence, or per run. Units must be explicit. Latency should be stated in ns or μ s, probe counts as logical candidate checks or kicks, and rates as dimensionless fractions or percentages. The phrase “maximum insertion probes” should be avoided when the underlying measurement is actually “maximum primary kicks,” because the two are not identical quantities in the proposed implementation.

For reproducibility, the final numerical table should also carry a run identifier or seed set. This enables a later reader to determine whether a surprising result came from a different randomized workload or from a code revision. If the study eventually adopts real packet traces, the trace identifier and extraction procedure should be recorded in the same manner.

Q. Recommended Result Figures

Three figures provide the strongest quantitative narrative. Figure A should show insertion-latency empirical cumulative distribution functions for the principal structures. Figure B should show stash-routing probability versus K at fixed load. Figure C should show routing probability and successful insertion fraction versus load for the proposed structure. These figures together expose the three intended dimensions of the design: tail control, displacement-budget trade-off, and finite-load feasibility.

A fourth optional figure should display the distribution of stash occupancy. A fifth can compare logical lookup cost for positive and negative queries. The figures should use identical axes and normalization across configurations. In particular, the

load-factor plots must distinguish aggregate load across all sub-tables from per-sub-table occupancy.

No figure should contain a theoretical curve drawn through observed data unless the curve represents a clearly defined analytical model. The theoretical threshold near 0.918 for three-choice one-slot cuckoo hashing may be shown as a reference line in a load plot, but the caption must state that it is an asymptotic literature reference and not a measured threshold of the implementation.

R. Interpretation of Exceptional Events

Exceptional events should be analyzed as a separate population. For classical two-way cuckoo hashing, the benchmark increments a full-rehash counter when the displacement loop reaches its prescribed limit and then reconstructs the table under new hash functions. The cost of that event is qualitatively different from a normal insertion, so a single average latency can hide its importance.

For the proposed scheme, the corresponding exceptional events are stash routing and stash-overflow recovery. The design objective is not to prove that such events never occur, but to prevent the primary insertion path from growing without bound. This is a more precise description of the architecture than claiming that full-table rehashing has been eliminated in every possible operational state. The source does contain a global-hash invariant: changing the global hash functions while main-table entries remain resident would invalidate those entries because their physical positions were computed under the old functions. The corrected implementation therefore keeps the global hash functions fixed during stash recovery.

The final Results should include a transition matrix if possible: normal insertion, stash-routed insertion, recovery-triggering insertion, and terminal exceptional insertion. Such a matrix makes it possible to determine whether a given latency tail is caused by ordinary displacement or by auxiliary recovery. Without this decomposition, a measured tail reduction cannot be attributed confidently to the bounded-displacement policy itself.

S. Replication Across Hash Seeds

The benchmark uses deterministic seeds for the workload and hash functions. This is useful for reproducibility, but one seed cannot establish that a result is representative. The final study should repeat the experiment over a declared seed set, preserving the same workload size and capacity policy. A separate seed should be used for the key-generation stream and, when appropriate, for the hash-function and eviction-randomness streams so that the sources of variation can be distinguished.

The most important seed-level outputs are the number of successful insertions, routing count, overflow count, recovery count, and latency percentiles. A result should be considered stable only if these quantities remain qualitatively consistent across seeds. Large seed-to-seed differences are themselves evidence that the structure operates close to a stochastic transition and should be discussed rather than averaged away.

TABLE V
CLAIM CLASSES USED IN THE QUANTITATIVE ANALYSIS

Class	Admissible statement
Deterministic implementation	The configured primary displacement loop executes no more than K iterations.
Derived logical cost	Primary insertion examines at most dK candidate positions before fallback, up to constant-time bookkeeping per step.
Theoretical reference	Random-model cuckoo literature provides asymptotic load thresholds and insertion-time results under explicitly stated assumptions.
Empirical observation	Routing probability, stash occupancy, latency percentiles, and rehash counts for the supplied program require execution.
System-level claim	Line-rate suitability requires hardware or controlled-system measurements beyond the algorithmic bound.

T. Summary of Quantitative Findings Established Before Execution

Before benchmark execution, the study can establish the following quantitative facts about the implementation. The principal configuration has three candidate hash positions, a sixteen-kick primary budget, and a four-entry stash. The main lookup procedure examines at most three primary positions followed by at most four stash positions. The primary displacement loop contains at most sixteen iterations in the principal configuration, although each iteration may perform up to three candidate checks before selecting an eviction target. The recursive stash-recovery mechanism has a source-level retry-depth limit of twenty.

These are implementation facts, not performance claims. They support the design rationale that the primary work is explicitly bounded. They do not establish a numerical nanosecond latency, a success probability, or a load threshold. Those quantities remain empirical or theoretical, respectively.

The experimental outcome should therefore be reported in a manner that permits the reader to see exactly where the evidence changes from proof to measurement. Table V provides the intended separation.

U. Results Reproducibility Checklist

Before the numerical tables are finalized, the following conditions should be satisfied. The exact source revision must be recorded; the workload generation must complete without duplicates; every baseline must successfully account for all attempted operations; the proposed structure must pass a membership check after insertion; stash contents must remain logically reachable; and the number of stored keys must agree with the number of unique inserted keys after accounting for the benchmark's recovery bookkeeping.

The timing harness should be executed under a stable machine configuration. Compiler flags must be recorded rather than inferred from a previous draft. The benchmark should emit machine-readable output rather than relying exclusively on human-readable stderr text. Percentiles should be computed from raw samples and not retyped manually. Every numerical

cell in the manuscript should be traceable to one output record or one explicitly cited theoretical source.

The final release should preserve the raw benchmark logs, the exact source code, the compiler command, the operating-system information, and the seed configuration. A paper that reports only final averages cannot be independently checked for sample-count mismatches, failed insertions, or exceptional recovery paths. Retaining the raw evidence also permits later investigation if a reviewer questions a tail-latency result.

V. Concluding Assessment of the Results Stage

The cleaned Results section does not claim a performance winner because no benchmark execution has been supplied that could support such a conclusion. Instead, it establishes a precise quantitative framework in which the proposed design can be evaluated without conflating theoretical results with measurements. The benchmark is sufficiently concrete to produce directly measurable insertion, lookup, stash, and rehash statistics, but those statistics remain unpopulated until the source is executed on a declared environment.

This is a substantive improvement over the original presentation. Removing unsupported numerical values does not weaken the algorithmic contribution; it makes the empirical argument falsifiable. The principal research question becomes whether the bounded primary displacement rule, combined with a small stash and same-hash recovery, produces the predicted reduction in exceptional primary work while maintaining acceptable end-to-end insertion and lookup behavior. The answer must be determined by the benchmark rather than by the structure of the manuscript.

The subsequent discussion should therefore interpret only the numbers that survive this admissibility test. In particular, a measured absence of full rehashes over a finite test should be reported as a finite-run observation rather than as a proof of impossibility; a measured routing probability should be reported with its workload and seed; and a latency percentile should be tied to the hardware and compiler environment. With these restrictions in place, the empirical component of the study becomes suitable for a journal pre-submission because its claims can be independently reproduced, challenged, and refined.

XII. CONCLUSION

This manuscript has examined a bounded-displacement variant of d -ary cuckoo hashing intended for exact-match flow-table workloads in which lookup regularity is important and unbounded insertion cascades are undesirable. The principal design mechanism is simple: each key is associated with a fixed number d of candidate locations, the ordinary eviction process is limited by a fixed displacement budget K , and an auxiliary stash of finite capacity s receives the displaced key when the primary budget is exhausted. The implementation further retains the global candidate hash functions during stash recovery and randomizes only the eviction choices used while attempting to reintegrate stashed entries. This construction therefore provides a clearly bounded *primary* displacement

path without requiring the stronger claim that every successful insertion, including all exceptional recovery behavior, has a deterministic constant worst-case cost.

The distinction between these two statements is central to the scientific interpretation of the design. The implementation makes the ordinary displacement loop finite by construction. For a fixed configuration, the number of eviction rounds is therefore bounded by K , and the number of candidate-location examinations on that path is bounded by a function of dK . The lookup operation is also explicitly bounded with respect to the configured main-table candidate set and the finite stash. These are structural properties of the implementation and do not depend on observing favorable benchmark behavior.

By contrast, the frequency with which the bounded path reaches the stash, the distribution of stash occupancy, the frequency and cost of stash-overflow recovery, and the probability of terminal recovery failure are stochastic or workload-dependent quantities. The supplied program contains counters for these exceptional events, but the source itself is not evidence that any particular event rate is zero or negligible. The manuscript therefore treats these quantities as empirical variables that must be measured under a declared workload, seed configuration, compiler configuration, and execution environment. This distinction also prevents a finite experimental observation from being promoted into an asymptotic theorem.

The theoretical literature provides a complementary framework for interpreting the implementation. Classical cuckoo hashing establishes worst-case constant lookup under the basic candidate-location model, while subsequent work characterizes insertion time, load thresholds, stash failure probabilities, and de-amortized constructions under more specific stochastic models. In particular, the modern literature demonstrates that several different d -ary insertion procedures can have constant expected insertion behavior below appropriate thresholds, but those results are properties of the corresponding algorithms and assumptions rather than automatic consequences of imposing a hard displacement cap on another implementation. The present manuscript consequently uses such results as theoretical context and comparison points rather than as imported guarantees.

The resulting research claim is deliberately narrower and more testable than the claims in the uncorrected manuscript. The proposed structure should be evaluated as a mechanism for controlling the work performed on the normal insertion path while preserving bounded candidate-based lookup and providing a finite overflow mechanism. Whether that mechanism yields a useful reduction in tail latency, acceptable stash utilization, or a favorable memory–latency trade-off is an empirical question. The manuscript has therefore been rewritten so that the numerical answer to that question is left to the benchmark rather than embedded in the prose in advance.

XIII. SUMMARY OF CONTRIBUTIONS

The first contribution of the study is an implementation-oriented specification of bounded-displacement d -ary cuckoo

hashing for exact-match dictionaries. The design is parameterized by the number of candidate locations d , the maximum number of primary eviction rounds K , and the stash capacity s . This parameterization exposes the principal design trade-off directly: additional displacement budget can reduce the number of entries that reach the stash, while a smaller budget limits the work of the ordinary insertion path more aggressively.

The second contribution is a source-consistent account of the placement invariant. Once entries have been installed under a particular set of candidate hash functions, those functions define the locations that a lookup must inspect. Replacing the global functions while leaving the physical placements unchanged would invalidate this correspondence. The corrected design therefore retains the global hash functions and changes only the randomized eviction trajectory used when reinserting stash contents. This is an implementation-level correctness condition rather than a stylistic choice.

The third contribution is the separation of bounded displacement, stash routing, and stash recovery into distinct analytical events. This decomposition is useful because each event has a different cost model and a different failure interpretation. A primary-path budget can be proven deterministically; stash routing can be measured directly; and recovery can be analyzed as an exceptional control path whose behavior depends on the current occupancy state and the eviction randomness.

The fourth contribution is a claim-auditing methodology for empirical algorithm papers. The cleaned manuscript does not treat a configured experimental parameter as an observed result. For example, the supplied benchmark specifies a workload of 2,000,000 distinct keys and 2,000,000 lookups, and the main proposed configuration uses $d = 3$, $K = 16$, and $s = 4$; these are source-derived settings, not performance measurements. The final paper should therefore report numerical latency, routing frequency, overflow frequency, rehash frequency, and success rate only after the corresponding executable experiment has been run and the raw output retained.

The fifth contribution is a journal-oriented separation of algorithmic evidence, theoretical evidence, and empirical evidence. This separation allows the paper to remain technically useful even before the benchmark dataset is populated: the implementation can be audited, the logical bounds can be proved, the experimental protocol can be reproduced, and the remaining empirical questions can be stated explicitly. The result is a manuscript whose claims can be falsified and refined without rewriting the basic research structure.

XIV. FUTURE WORK

A. Execution of the Controlled Benchmark

The immediate experimental requirement is to execute the supplied benchmark under a documented software and hardware environment. The benchmark already contains a comparative microbenchmark for linear probing, chained hashing, classical two-way cuckoo hashing, and the proposed bounded-displacement structure. It also contains dedicated experiments for the conventional low-load and near-ceiling regimes of two-way cuckoo hashing and a sensitivity experiment over

several values of K . The next release should preserve the raw output of each run, record the compiler and optimization configuration, and attach a unique run identifier to the resulting tables.

The most important methodological improvement is replication. A deterministic seed is valuable for reproducing a particular run but is insufficient for characterizing the variability induced by the random key stream, hash-function seeds, and eviction choices. Multiple independent seeds should therefore be used for each principal configuration. Summary statistics should be computed per run before aggregating across runs so that a single unusually long execution cannot dominate the reported uncertainty.

B. Real Flow-Trace Evaluation

The present source uses synthetic random 64-bit identifiers described as synthetic five-tuple-like flow IDs. This is appropriate for isolating the behavior of the hash table itself, but it does not establish how correlated or skewed network-flow identifiers affect the candidate-location distribution. A subsequent study should incorporate real flow traces with a declared anonymization and extraction procedure. The trace study should report the number of unique active flows, the temporal ordering of first insertion, the persistence distribution, and the lookup-to-update ratio.

A real-trace evaluation should also preserve the distinction between input distribution and hash-function behavior. If trace-derived keys are hashed using the same functions as the synthetic workload, differences can be interpreted as workload effects. If a different hash family is introduced, its cost and statistical properties become an additional experimental variable and should not be confounded with the effect of the trace.

C. Formal Stochastic Analysis of Bounded Eviction

The deterministic bound on the primary displacement path does not by itself yield a closed-form probability for stash routing. A complete probabilistic analysis would require a mathematical model for the occupancy state generated by the implemented eviction policy. In particular, the randomized choice among the d occupied candidate locations and the history-dependent state of the table mean that the process is not automatically equivalent to an offline random assignment problem.

A useful direction is to formulate the insertion process as a Markov or random-graph process whose state captures the local neighborhood of the displaced key. One objective would be to derive bounds on the probability that K consecutive eviction rounds fail to expose a free candidate location, conditioned on the global load factor and the hash-family assumptions. Such a result could then connect the configured values of (d, K, s) to a finite-run failure probability. Until such a derivation is supplied, the empirical routing probability should remain an observed property of the implementation rather than a theorem.

D. Correctness of Stash Recovery

The current implementation contains a terminal recovery depth condition. That condition is useful operationally because it prevents unbounded recursion, but it also creates a correctness obligation: every path that returns a terminal failure code must preserve the representation of all keys that were previously resident. A future implementation should replace recursive stash recovery with an iterative recovery procedure with explicit transactional semantics. Candidate states should be copied or logged before modification, and a recovery attempt should commit only after all affected entries remain reachable from either the main table or the stash.

This redesign would make it possible to state a stronger invariant: after every completed operation, the set of logically stored keys is exactly the union of keys reachable through the main-table candidate functions and keys explicitly stored in the stash. Such an invariant should be validated using randomized differential testing against a reference dictionary.

E. Deletion and Dynamic Table Resizing

The supplied benchmark focuses on insertion and lookup and does not constitute a complete dynamic dictionary with deletion. A production-oriented flow table must eventually accommodate expiration, deletion, replacement, and growth. These operations interact with bounded displacement in non-trivial ways. Deletion can create holes that alter the local insertion landscape, while resizing changes the mapping from keys to physical locations and therefore requires coordinated migration of the affected table.

A future design should therefore specify resizing as an explicit global epoch transition. During migration, a key should remain reachable under its current epoch until its replacement placement has been committed. Incremental resizing can then be studied independently from stash recovery instead of being implicitly conflated with it. This separation is particularly important in systems in which a global resize can dominate the cost of otherwise bounded insertions.

F. Hardware and Parallel Implementations

The motivation for the design includes environments where insertion-time variance can interfere with service guarantees. A subsequent hardware study could map each candidate lookup and eviction step to a fixed-latency pipeline stage and evaluate whether the stash can be implemented in a small on-chip memory. Such a study should report clock frequency, pipeline depth, logic utilization, block-memory utilization, and the timing of exceptional recovery paths rather than inferring hardware suitability from a software latency measurement.

For software packet processing, a separate study should investigate cache locality, batched lookup, vectorized candidate comparisons, and the effect of memory-system contention. Existing networking-oriented cuckoo implementations demonstrate that practical lookup performance depends not only on the number of logical candidate locations but also on bucket organization, prefetching, cache-line layout, and workload composition. Those factors are therefore natural

extensions rather than quantities that can be inferred from the present algorithmic analysis alone.

G. Adversarial and Robustness Analysis

The current evaluation uses pseudorandom keys generated independently of the hash state. This is a useful baseline, but it does not establish robustness when an adversary can choose keys with knowledge of the candidate hash functions. A security-oriented extension should define an adversarial model explicitly and measure whether the bounded displacement rule merely limits the cost of individual insertions or also changes the probability of concentrated stash occupancy.

This question is distinct from ordinary stochastic hashing analysis. A distribution that is benign under independently random keys can behave differently when keys are selected adaptively. Any claim of robustness would therefore require both an explicit adversary model and a proof or experiment covering that model.

H. Parameter Optimization

The parameters d , K , and s provide a compact design space for multi-objective optimization. The appropriate operating point depends on whether the system values lookup cost, insertion tail control, memory efficiency, or exceptional-path frequency most strongly. A future parameter study should therefore report the Pareto surface rather than collapsing the design to a single scalar objective.

The existing benchmark already provides a natural starting point through its K sensitivity experiment. The next step is a factorial or response-surface study over d , K , s , and load factor, with independent seeds and repeated runs. Such a study would permit identification of interactions that cannot be inferred from one-factor-at-a-time experiments.

XV. FINAL EDITORIAL INTEGRATION AND CLAIM AUDIT

The revision strategy adopted throughout this manuscript leads to a final editorial rule: every substantive statement must belong to one of three evidentiary categories. The first category contains statements proved directly from the implementation and its invariants. The second contains statements imported from the published theoretical literature and explicitly attributed to the corresponding model and assumptions. The third contains empirical statements generated by a particular execution of the supplied benchmark. No statement should silently move from one category to another.

This rule resolves several classes of error that appeared in the original submission. A configured displacement limit is not the same thing as an observed latency maximum. A theoretical load threshold is not the same thing as a finite-sample empirical transition. A small stash is not evidence that overflow is impossible. A benchmark data structure labeled as a five-tuple workload is not evidence that a real packet trace was processed. Similarly, a compile-time or logical-cost inspection cannot substitute for an executed timing experiment.

The same rule applies to claims about system deployment. A bounded algorithmic path can be relevant to line-rate systems, but line-rate operation additionally depends on processor

or hardware architecture, cache behavior, synchronization, memory technology, packet batching, and the ratio of reads to writes. Such considerations should therefore be presented as deployment implications conditioned on measured system characteristics, not as properties proved by the hash-table abstraction itself.

A. Disposition of Unsupported Numerical Claims

All numerical claims inherited from the earlier draft that cannot be traced either to an executed benchmark record or to a cited theoretical result are intentionally excluded from the final scientific argument. This includes precise throughput values, nanosecond percentiles, zero-event statements, success rates, and ratios that were previously presented without a preserved execution log. Replacing such values with explicit placeholders or methodological descriptions is preferable to manufacturing a result from source-code inspection.

Where the benchmark output eventually becomes available, each numerical value should be inserted together with its sample count, seed set, parameter configuration, and execution environment. A numerical claim that cannot be reconstructed from the raw output should not be included in the final manuscript. This provides a simple provenance requirement for every table and figure.

B. Disposition of Strong Complexity Claims

The revised manuscript intentionally avoids the unrestricted statement that the complete proposed dictionary has deterministic $O(1)$ insertion in all circumstances. The primary displacement loop is bounded, but the complete operation also includes stash routing, stash scanning, and stash recovery. The supplied implementation has a finite retry-depth guard, and the semantics of an exceptional terminal path must be considered explicitly. The defensible statement is therefore that the *normal primary displacement component* is deterministically bounded for fixed d and K , while complete insertion behavior must be characterized separately.

This wording also avoids confusing a practical latency target with an asymptotic theorem. For a fixed implementation with fixed constants, one may observe a finite upper bound on a particular code path, but a journal claim about asymptotic $O(1)$ insertion requires a precise family of operations and a proof that the exceptional path is controlled in the required model. The manuscript does not make that stronger claim without such a proof.

C. Disposition of Load-Factor Claims

The load factor used by the benchmark is an engineering parameter describing the occupancy of the allocated table. The asymptotic load thresholds studied in the cuckoo-hashing literature describe the existence of valid assignments under random models and are therefore conceptually related but not identical quantities. In particular, the present implementation uses a concrete hash family, finite tables, sequential online insertion, and a bounded eviction heuristic. An observed transition in this implementation can be compared against the

corresponding theoretical threshold as a reference point, but equality should not be asserted without finite-size and model analysis.

For the principal configuration, the benchmark allocates the main storage so that the target aggregate load is approximately 0.90 with $d = 3$. This target is useful for testing behavior near the known three-choice asymptotic threshold, but the manuscript treats it as an experimental operating point rather than as evidence that the implementation can sustain a particular theoretical threshold. The same principle applies to the sensitivity sweep around K and to every future load-factor experiment.

D. Publication Readiness Criteria

Before journal submission, the manuscript should satisfy four minimum conditions. First, every empirical table and figure must be populated from preserved execution data or explicitly marked as pending. Second, the exact code revision used to generate the numbers must be archived and identified in the paper. Third, every theorem, proposition, and asymptotic statement must either have a proof in the manuscript or cite a source whose assumptions match the statement being made. Fourth, the benchmark must include a post-insertion correctness test that verifies that every inserted key remains discoverable through the advertised lookup interface.

These conditions do not require the empirical results to confirm the original hypothesis. A result showing substantial stash routing, limited load tolerance, or weak latency improvement is scientifically valid if it is measured correctly and interpreted without overstatement. Indeed, a negative or mixed result can clarify the practical boundary of bounded displacement and can motivate a more targeted algorithmic refinement.

XVI. REPRODUCIBILITY STATEMENT

The experimental artifacts for this study consist of the LaTeX manuscript, the bibliography, the C++ benchmark harness, the compiler invocation, and the raw output generated by each experimental run. The supplied benchmark uses fixed seeds for the initial workload and for the hash structures, generates distinct random 64-bit keys, and defines the comparison and sensitivity experiments directly in source code. The code also records insertion-latency percentiles and event counters for the proposed structure.

For the present revision, the benchmark has not been executed as part of the manuscript-cleaning process. Consequently, this paper version should be understood as an algorithmic and methodological revision with unpopulated empirical results rather than as a completed performance report. Once execution is performed, the raw outputs should be retained without manual transcription and the final tables should be generated from those records.

The reproducibility objective is therefore not merely that another researcher can compile the source, but that another researcher can reconstruct the provenance of every numerical conclusion. This includes the exact code, parameter values, seeds, machine environment, compiler version, optimization

flags, and analysis script used to convert raw observations into tables and figures. Such provenance is especially important for tail-latency measurements, for which small changes in operating conditions can alter upper percentiles even when the algorithm is unchanged.

XVII. CLOSING REMARKS

The principal methodological lesson of this work is that bounded work on an algorithmic fast path and bounded work for an entire dynamic operation are not interchangeable claims. The proposed data structure makes a concrete and useful engineering intervention by capping ordinary displacement work and providing an explicit overflow representation. Whether the resulting system delivers the desired trade-off must be established through measurement under clearly stated conditions.

The cleaned manuscript is consequently structured to support that measurement without presupposing its outcome. The theoretical sections establish the properties that follow from the model and source code; the experimental sections define how the remaining quantities should be measured; and the discussion distinguishes asymptotic thresholds from finite implementation behavior. This structure provides a suitable basis for a journal pre-submission in which the central algorithmic idea can be evaluated on its actual evidence rather than on numerical claims that cannot be reproduced.

APPENDIX A C++ BENCHMARK HARNESS

The following listing is included to make the implementation underlying the experimental protocol explicit. The source is a lightly cleaned version of the supplied benchmark harness. The changes are limited to manuscript-integrity corrections: removal of the assertion that all paper numbers are already directly supported by program output, and correction of the proposed lookup aggregation so that stash hits are not silently capped at d logical accesses. The benchmark itself has not been executed as part of this manuscript revision.

```
1 // Benchmark harness for classical 2-way cuckoo hashing,
  // bounded-displacement
2 // d-ary cuckoo hashing with a stash, chained hashing, and
  // linear probing.
3 // Experimental harness used to generate comparative
  // measurements.
4 // Numerical claims in the manuscript must be tied to an
  // actual recorded run.
5 #include <bits/stdc++.h>
6 using namespace std;
7 using u64 = uint64_t;
8 using Clock = chrono::steady_clock;
9
10 static inline u64 splitmix64(u64 x) {
11     x += 0x9E3779B97F4A7C15ULL;
12     x = (x ^ (x >> 30)) * 0xBF58476D1CE4E5B9ULL;
13     x = (x ^ (x >> 27)) * 0x94D049BB133111EBULL;
14     return x ^ (x >> 31);
15 }
16 struct HashFn {
17     u64 seed;
18     HashFn(u64 s=0): seed(s) {}
19     inline u64 operator()(u64 key) const { return
        splitmix64(key ^ seed); }
20 };
21
22 // ----- Linear Probing -----
23 struct LinearProbing {
```

```
    vector<u64> table; vector<char> occ; size_t cap;
    size_t n=0;
    HashFn h;
    LinearProbing(size_t c, u64 seed): table(c,0), occ(c
        ,0), cap(c), h(seed) {}
    // returns probe count; -1 if table full without
    // finding slot (shouldn't happen under cap)
    long insert(u64 key){
        size_t idx = h(key)%cap; long probes=0;
        for(size_t i=0;i<cap;i++){
            probes++;
            size_t p=(idx+i)%cap;
            if(!occ[p]){ occ[p]=1; table[p]=key; n++;
                return probes; }
            if(occ[p] && table[p]==key) return probes; //
            already present
        }
        return -1;
    }
    long lookup(u64 key){
        size_t idx=h(key)%cap; long probes=0;
        for(size_t i=0;i<cap;i++){ probes++; size_t p=(idx
            +i)%cap; if(!occ[p]) return probes; if(table[p]==key)
            return probes; }
        return -1;
    }
};

// ----- Chained Hashing -----
struct ChainedHashing {
    vector<vector<u64>> table; size_t cap; size_t n=0;
    HashFn h;
    ChainedHashing(size_t c, u64 seed): table(c), cap(c),
        h(seed) {}
    long insert(u64 key){
        size_t idx=h(key)%cap; auto &b=table[idx];
        for(auto k:b) if(k==key) return (long)b.size();
        b.push_back(key); n++; return (long)b.size();
    }
    long lookup(u64 key){
        size_t idx=h(key)%cap; auto &b=table[idx]; long
        probes=0;
        for(auto k:b){ probes++; if(k==key) return probes;
        }
        return probes+1;
    }
};

// ----- Classical 2-way cuckoo (Pagh-Rodler),
    MaxLoop = ceil(3 log_{1+eps} r) -----
    struct ClassicalCuckoo {
        vector<u64> T1,T2; vector<char> occ1,occ2; size_t m;
        // each table size m, r=2m
        HashFn h1,h2; size_t n=0; long maxLoop; long
        fullRehashes=0;
        double eps;
        mt19937_64 rng;
        ClassicalCuckoo(size_t m_, double eps_, u64 seed): T1(
            m_,0), T2(m_,0), occ1(m_,0), occ2(m_,0),
            m(m_), h1(seed*2+1), h2(seed*2+2), eps(eps_), rng(
            seed) {
            double r = 2.0*m;
            maxLoop = (long)ceil(3.0*log(r)/log(1.0+eps));
            if(maxLoop<4) maxLoop=4;
        }
        long lookup(u64 key){
            if(occ1[h1(key)%m] && T1[h1(key)%m]==key) return
                1;
            if(occ2[h2(key)%m] && T2[h2(key)%m]==key) return
                2;
            return -1; // not found
        }
        // Returns the number of kicks used (>=0). If the
        // insertion budget is
        // exhausted, a full-table rehash is performed and the
        // configured bound
        // is returned as a conservative logical cost.
        long insert(u64 key){
            if(lookup(key)>0) return 0;
            u64 cur=key; long kicks=0;
            for(long it=0; it<maxLoop; it++){
                size_t p1=h1(cur)%m;
                if(!occ1[p1]){ occ1[p1]=1; T1[p1]=cur; n++;
```

```

87     return kicks; }
88     u64 evicted=T1[p1]; T1[p1]=cur; occ1[p1]=1;
89     cur=evicted; kicks++;
90     size_t p2=h2(cur)%m;
91     if(!occ2[p2]){ occ2[p2]=1; T2[p2]=cur; n++;
92     return kicks; }
93     evicted=T2[p2]; T2[p2]=cur; occ2[p2]=1; cur=
94     evicted; kicks++;
95     }
96     // trigger full rehash: pick new seeds, reinsert
97     everything including cur
98     fullRehashes++;
99     vector<u64> all;
100     for(size_t i=0;i<m;i++){ if(occ1[i]) all.push_back
101     (T1[i]); if(occ2[i]) all.push_back(T2[i]); }
102     all.push_back(cur);
103     fill(occ1.begin(),occ1.end(),0); fill(occ2.begin()
104     ,occ2.end(),0);
105     size_t oldn=n; n=0;
106     h1=HashFn(rng()); h2=HashFn(rng());
107     for(auto k: all){ insert(k); } // recursive; rare
108     return maxLoop; // report as max cost for latency
109     purposes
110 }
111 };
112 // ----- Proposed: d-ary bounded-displacement cuckoo
113 with stash -----
114 struct BoundedCuckoo {
115     int d; size_t m; vector<vector<u64>> T; vector<vector<
116     char>> occ;
117     vector<HashFn> h; size_t n=0;
118     vector<u64> stash; size_t stashCap; vector<char>
119     stashOcc;
120     long maxKicks;
121     long incrementalRehashes=0, stashOverflowEvents=0,
122     stashRoutedCount=0;
123     long peakStashOcc=0; double sumStashOcc=0; long
124     stashSamples=0;
125     mt19937_64 rng;
126
127     BoundedCuckoo(int d_, size_t m_, long maxKicks_,
128     size_t stashCap_, u64 seed)
129     : d(d_), m(m_), T(d_, vector<u64>(m_,0)), occ(d_,
130     vector<char>(m_,0)),
131     stash(stashCap_,0), stashCap(stashCap_),
132     stashOcc(stashCap_,0),
133     maxKicks(maxKicks_), rng(seed) {
134     for(int i=0;i<d;i++) h.push_back(HashFn(seed*10+i
135     +7));
136     }
137     long lookup(u64 key){
138     for(int i=0;i<d;i++){ size_t p=h[i](key)%m; if(occ
139     [i][p] && T[i][p]==key) return i+1; }
140     for(size_t j=0;j<stashCap;j++) if(stashOcc[j] &&
141     stash[j]==key) return d+1;
142     return -1;
143     }
144     long currentStashOcc(){ long c=0; for(size_t j=0;j<
145     stashCap;j++) if(stashOcc[j]) c++; return c; }
146
147     // Returns kicks used, -2 when the final displaced key
148     is routed to the
149     // stash, and -4 only when repeated stash recovery
150     exceeds the retry bound.
151     long insertInner(u64 key, int retryDepth=0){
152     if(lookup(key)>0) return 0;
153     u64 cur=key; long kicks=0;
154     for(long round=0; round<maxKicks; round++){
155     for(int i=0;i<d;i++){
156     size_t p=h[i](cur)%m;
157     if(!occ[i][p]){ occ[i][p]=1; T[i][p]=cur;
158     n++; return kicks; }
159     }
160     int i = (int)(rng()%d);
161     size_t p=h[i](cur)%m;
162     u64 evicted=T[i][p]; T[i][p]=cur; occ[i][p]=1;
163     cur=evicted; kicks++;
164     }
165     // route to stash
166     for(size_t j=0;j<stashCap;j++){
167     if(!stashOcc[j]){ stashOcc[j]=1; stash[j]=cur;
168     n++; stashRoutedCount++;
169     sumStashOcc+=currentStashOcc();
170     stashSamples++;
171     peakStashOcc=max(peakStashOcc, (long)
172     currentStashOcc());
173     return -2; }
174     }
175     // stash overflow -> incremental rehash of stash
176     contents (same h[], fresh eviction randomness)
177     stashOverflowEvents++;
178     if(retryDepth >= 20){ fullRehashesFromStuckStash
179     ++; return -4; } // pathological fallback
180     incrementalRehashStash(retryDepth+1);
181     // The key in 'cur' is the entry that remains
182     unplaced after the
183     // bounded eviction path. Retain that key across
184     stash recovery;
185     // restarting from the original key would silently
186     lose 'cur'.
187     return insertInner(cur, retryDepth+1);
188     }
189     long fullRehashesFromStuckStash=0;
190     // CORRECTNESS NOTE: an earlier version of this
191     routine redrew fresh global
192     // hash functions h[0..d-1] on every stash overflow.
193     That silently broke
194     // lookups for every key already resident in the main
195     tables (their physical
196     // slot was computed under the OLD h[], so probing
197     under the NEW h[] would
198     // almost never find them). Global hash functions must
199     not change once any
200     // key has been placed under them. The corrected
201     design keeps h[] fixed for
202     // the table's lifetime and instead retries the
203     bounded-kick placement of
204     // the stashed key(s) with fresh *randomized eviction
205     choices* only --
206     // still O(1) extra work, and it never touches already
207     -placed keys.
208     void incrementalRehashStash(int retryDepth=0){
209     incrementalRehashes++;
210     vector<u64> S;
211     for(size_t j=0;j<stashCap;j++) if(stashOcc[j]) S.
212     push_back(stash[j]);
213     fill(stashOcc.begin(),stashOcc.end(),0);
214     n -= S.size();
215     for(auto k: S) insertInner(k, retryDepth); // same
216     h[], fresh random eviction path
217     }
218     long insert(u64 key){ return insertInner(key); }
219     };
220
221     template<typename F>
222     void percentileReport(vector<long>& samples, F unitConv,
223     const string& label){
224     sort(samples.begin(), samples.end());
225     size_t N=samples.size();
226     auto pct=[&](double p)->long{ size_t idx=(size_t)(p*(N
227     -1)); return samples[idx]; };
228     cerr << label << " N=" << N
229     << " p50=" << unitConv(pct(0.50))
230     << " p99=" << unitConv(pct(0.99))
231     << " p999=" << unitConv(pct(0.999))
232     << " max=" << unitConv(samples.back()) << "\n";
233     }
234
235     int main(int argc, char** argv){
236     // ----- Experiment configuration
237     -----
238     size_t N = 2000000; // number of distinct
239     keys inserted (real N, reported honestly)
240     size_t lookupOps = 2000000; // subsequent lookups
241     performed
242     double targetLoad = 0.90; // target load factor for
243     capacity sizing
244     u64 seed = 123456789ULL;
245
246     cerr << fixed << setprecision(6);
247     mt19937_64 keyRng(42);
248
249     // Generate N distinct random 64-bit keys (synthetic
250     5-tuple-like flow IDs)
251     vector<u64> keys(N);

```

```

203 { unordered_set<u64> seen; seen.reserve(N*2);
204   for(size_t i=0;i<N;i++){ u64 k; do{ k = keyRng(); }
205   while(!seen.insert(k).second); keys[i]=k; } }
206
207 // ===== Table II: comparative
208 microbenchmark =====
209 cerr << "\n=== TABLE II: comparative microbenchmark, N
210 = " << N << " keys ===\n";
211
212 // --- Linear Probing, cap so that N/cap = 0.70 ---
213 {
214   size_t cap = (size_t)(N/0.70);
215   LinearProbing lp(cap, seed);
216   vector<long> insLat, lookLat;
217   auto t0=Clock::now();
218   for(auto k: keys){ auto s=Clock::now(); lp.insert(
219   k); auto e=Clock::now();
220   insLat.push_back(chrono::duration_cast<chrono
221   ::nanoseconds>(e-s).count()); }
222   auto t1=Clock::now();
223   double totalInsSec = chrono::duration_cast<double>(t1-
224   t0).count();
225   for(size_t i=0;i<lookupOps;i++){ u64 k=keys[i%N];
226   auto s=Clock::now(); lp.lookup(k); auto e=Clock::now
227   ();
228   lookLat.push_back(chrono::duration_cast<chrono
229   ::nanoseconds>(e-s).count()); }
230   double avgLookProbes=0; for(auto k: keys)
231   avgLookProbes+=lp.lookup(k); avgLookProbes/=N;
232   long maxInsProbe=0; { size_t idx=0; LinearProbing
233   lp2(cap,seed); for(auto k:keys){ long p=lp2.insert(k)
234   ; maxInsProbe=max(maxInsProbe,p); } }
235   cerr << "[LinearProbing] load=" << (double)N/cap
236   << " avgLookupProbes=" << avgLookProbes
237   << " maxInsertProbes=" << maxInsProbe << "
238   totalInsertTimeSec=" << totalInsSec << "\n";
239   percentileReport(insLat, [](long x){return x;}, "[
240   LinearProbing] insertLatencyNs");
241 }
242
243 // --- Chained Hashing, cap so load factor ~1.0 (alpha
244 =n/cap) ---
245 {
246   size_t cap = N; // alpha ~ 1.0
247   ChainedHashing ch(cap, seed);
248   vector<long> insLat;
249   for(auto k: keys){ auto s=Clock::now(); ch.insert(
250   k); auto e=Clock::now();
251   insLat.push_back(chrono::duration_cast<chrono
252   ::nanoseconds>(e-s).count()); }
253   double avgLookProbes=0; for(auto k: keys)
254   avgLookProbes+=ch.lookup(k); avgLookProbes/=N;
255   long maxChain=0; for(auto &b: ch.table) maxChain=
256   max(maxChain, (long)b.size());
257   cerr << "[Chained] load=" << (double)N/cap << "
258   avgLookupProbes=" << avgLookProbes
259   << " maxChainLen=(maxInsertProbes)=" <<
260   maxChain << "\n";
261   percentileReport(insLat, [](long x){return x;}, "[
262   Chained] insertLatencyNs");
263 }
264
265 // --- Classical 2-way cuckoo, eps=1 (50% max load),
266 sized so n/(2m) ~ 0.495 ---
267 {
268   size_t m = (size_t)(N/(0.495*2.0)); // r=2m,
269   target load n/r = 0.495
270   ClassicalCuckoo cc(m, 1.0, seed);
271   vector<long> insLat; vector<long> insKicks;
272   for(auto k: keys){ auto s=Clock::now(); long kk=cc
273   .insert(k); auto e=Clock::now();
274   insLat.push_back(chrono::duration_cast<chrono
275   ::nanoseconds>(e-s).count());
276   insKicks.push_back(kk); }
277   double avgLookProbes=0; for(auto k: keys){ long r=
278   cc.lookup(k); avgLookProbes += (r>0?r:2); }
279   avgLookProbes/=N;
280   long maxKicksObs=*max_element(insKicks.begin(),
281   insKicks.end());
282   cerr << "[ClassicalCuckoo d=2] load=" << (double)
283   cc.n/(2*m) << " maxLoopParam=" << cc.maxLoop
284   << " avgLookupProbes=" << avgLookProbes << "
285   maxKicksObserved=" << maxKicksObs
286
287   << " fullRehashes=" << cc.fullRehashes << "\n
288   ";
289   percentileReport(insLat, [](long x){return x;}, "[
290   ClassicalCuckoo] insertLatencyNs");
291 }
292
293 // --- Proposed bounded scheme, d=3, MaxKicks=16,
294 stash=4, target load 0.90 ---
295 {
296   int d=3; long maxKicks=16; size_t stashCap=4;
297   size_t m = (size_t)(N/(targetLoad*d));
298   BoundedCuckoo bc(d, m, maxKicks, stashCap, seed);
299   vector<long> insLat; vector<long> insKicks;
300   for(auto k: keys){ auto s=Clock::now(); long kk=bc
301   .insert(k); auto e=Clock::now();
302   insLat.push_back(chrono::duration_cast<chrono
303   ::nanoseconds>(e-s).count());
304   insKicks.push_back(kk== -2? maxKicks : (kk<0?
305   maxKicks:kk)); }
306   double avgLookProbes=0; for(auto k: keys){ long r=
307   bc.lookup(k); avgLookProbes += (r>0?r:(long)(d+1)); }
308   avgLookProbes/=N;
309   cerr << "[Proposed d=3,MaxKicks=16,s=4] load=" <<
310   (double)bc.n/(d*m)
311   << " avgLookupProbes=" << avgLookProbes << "
312   maxInsertKicks=" << maxKicks
313   << " stashRoutedCount=" << bc.
314   stashRoutedCount
315   << " incrementalRehashes=" << bc.
316   incrementalRehashes
317   << " stashOverflowEvents=" << bc.
318   stashOverflowEvents << "\n";
319   percentileReport(insLat, [](long x){return x;}, "[
320   Proposed] insertLatencyNs");
321 }
322
323 // --- Classical cuckoo at Pagh-Rodler's own
324 recommended operating point (load ~1/3) ---
325 {
326   size_t m = (size_t)(N/(0.33*2.0));
327   ClassicalCuckoo cc(m, 2.03, seed+555); // eps
328   chosen so r=(1+eps)n roughly matches 1/3 load, m
329   sized directly below
330   vector<long> insKicks;
331   for(auto k: keys){ long kk=cc.insert(k); insKicks.
332   push_back(kk); }
333   sort(insKicks.begin(), insKicks.end());
334   size_t Nn=insKicks.size();
335   cerr << "[ClassicalCuckoo typical-load-third] load
336   =" << (double)cc.n/(2*m) << " maxLoopParam=" << cc.
337   maxLoop
338   << " fullRehashes=" << cc.fullRehashes
339   << " kicks_p50=" << insKicks[(size_t)(0.50*(
340   Nn-1))]
341   << " kicks_p99=" << insKicks[(size_t)(0.99*(
342   Nn-1))]
343   << " kicks_p999=" << insKicks[(size_t)
344   (0.999*(Nn-1))]
345   << " kicks_max=" << insKicks.back() << "\n";
346 }
347
348 // --- Classical cuckoo, pushed near its ~50% ceiling
349 to expose the real long-chain tail ---
350 {
351   size_t m = (size_t)(N/(0.49*2.0));
352   ClassicalCuckoo cc(m, 1.0, seed+999);
353   vector<long> insKicks;
354   for(auto k: keys){ long kk=cc.insert(k); insKicks.
355   push_back(kk); }
356   sort(insKicks.begin(), insKicks.end());
357   size_t Nn=insKicks.size();
358   cerr << "[ClassicalCuckoo near-ceiling] load=" <<
359   (double)cc.n/(2*m) << " maxLoopParam=" << cc.maxLoop
360   << " fullRehashes=" << cc.fullRehashes
361   << " kicks_p50=" << insKicks[(size_t)(0.50*(
362   Nn-1))]
363   << " kicks_p99=" << insKicks[(size_t)(0.99*(
364   Nn-1))]
365   << " kicks_p999=" << insKicks[(size_t)
366   (0.999*(Nn-1))]
367   << " kicks_max=" << insKicks.back() << "\n";
368 }

```

```

308 // ===== Table III: sensitivity to
309 MaxKicks (d=3, alpha=0.88, s=4) =====
310 cerr << "\n=== TABLE III: MaxKicks sensitivity, N=" <<
311 N << ", d=3, target alpha=0.88, s=4 ===\n";
312 for(long mk : {4L,8L,12L,16L,24L}){
313     int d=3; size_t stashCap=4; double alpha=0.88;
314     size_t m = (size_t)(N/(alpha*d));
315     BoundedCuckoo bc(d, m, mk, stashCap, seed+mk);
316     long maxKicksObs=0;
317     for(auto k: keys){ long kk=bc.insert(k); if(kk>=0)
318         maxKicksObs=max(maxKicksObs,kk); }
319     double routingProb = (double)bc.stashRoutedCount /
320     N;
321     double avgStashOcc = bc.stashSamples? bc.
322     sumStashOcc/bc.stashSamples : 0.0;
323     cerr << "MaxKicks=" << mk << " routingProb=" <<
324     routingProb
325     << " avgStashOccAtRouting=" << avgStashOcc
326     << " stashOverflowEvents=" << bc.
327     stashOverflowEvents
328     << " maxObservedKicksBeforeStash=" <<
329     maxKicksObs << " load=" << (double)bc.n/(d*m) << "\n"
330     ;
331 }
332
333 cerr << "\nDone.\n";
334 return 0;
335 }
336

```

Listing 3. C++ benchmark harness for the comparative cuckoo-hashing study

- [16] Y. Arbitman, M. Naor, and G. Segev, “De-amortized Cuckoo hashing: Provable worst-case performance and experimental results,” in *Automata, Languages and Programming*. Springer, 2009, pp. 107–118.
- [17] S. Pontarelli, P. Reviriego, and M. Mitzenmacher, “Parallel d-pipeline: A cuckoo hashing implementation for increased throughput,” *IEEE Transactions on Computers*, vol. 65, no. 1, pp. 326–331, 2016.
- [18] —, “Cuckoo cache: A technique to improve flow monitoring throughput,” *IEEE Internet Computing*, vol. 20, no. 4, pp. 40–47, 2016.
- [19] N. Le Scouarnec, “Cuckoo++ hash tables: high-performance hash tables for networking applications,” in *Proceedings of the 2018 Symposium on Architectures for Networking and Communications Systems (ANCS)*. ACM, 2018, pp. 41–54.
- [20] C. Megías, A. Forencich, E. Ros, and J. Díaz, “Minimizing cuckoo hashing insertion time for networking applications in fpgas,” *IEEE Access*, vol. 13, pp. 216 832–216 841, 2025.
- [21] K. Yeo, “Cuckoo hashing in cryptography: Optimal parameters, robustness and applications,” 2023, arXiv:2306.11220.

REFERENCES

- [1] R. Pagh and F. F. Rodler, “Cuckoo hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [3] L. Devroye and P. Morin, “Cuckoo hashing: Further analysis,” *Information Processing Letters*, vol. 86, no. 4, pp. 215–219, 2003.
- [4] M. Drmota and R. Kutzelnigg, “A precise analysis of Cuckoo hashing,” *ACM Transactions on Algorithms*, vol. 8, pp. 11:1–11:36, 2012.
- [5] N. Fountoulakis, K. Panagiotou, and A. Steger, “On the insertion time of Cuckoo hashing,” *SIAM Journal on Computing*, vol. 42, no. 6, pp. 2156–2181, 2013, preliminary results circulated earlier; see arXiv:0912.1775.
- [6] A. Kirsch, M. Mitzenmacher, and U. Wieder, “More robust hashing: Cuckoo hashing with a stash,” *SIAM Journal on Computing*, vol. 39, no. 4, pp. 1543–1561, 2009, preliminary version in Proc. 16th Annual European Symposium on Algorithms (ESA), 2008.
- [7] R. Kutzelnigg, “A further analysis of cuckoo hashing with a stash and random graphs of excess r ,” *Discrete Mathematics and Theoretical Computer Science*, vol. 12, no. 3, pp. 81–102, 2010.
- [8] M. Aumüller, M. Dietzfelbinger, and P. Woelfel, “Explicit and efficient hash families suffice for cuckoo hashing with a stash,” *Algorithmica*, vol. 70, no. 3, pp. 428–456, 2014, arXiv preprint arXiv:1204.4431.
- [9] D. Noble, “Explicit, closed-form, general bounds for cuckoo hashing with a stash,” Cryptology ePrint Archive, Paper 2021/447, 2021, <https://eprint.iacr.org/2021/447>.
- [10] B. Minaud and C. Papamanthou, “Generalized cuckoo hashing with a stash, revisited,” *Information Processing Letters*, vol. 181, p. 106356, 2023.
- [11] N. Fountoulakis and K. Panagiotou, “Sharp load thresholds for cuckoo hashing,” *Random Structures & Algorithms*, vol. 41, no. 3, pp. 306–333, 2012.
- [12] M. Dietzfelbinger, A. Goerdt, M. Mitzenmacher, A. Montanari, R. Pagh, and M. Rink, “Tight thresholds for Cuckoo hashing via XORSAT,” 2009, arXiv:0912.0287.
- [13] T. Bell and A. Frieze, “ $o(1)$ insertion for random walk d -ary Cuckoo hashing up to the load threshold,” in *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2024, pp. 106–119.
- [14] S. Walzer, “Insertion time of random walk Cuckoo hashing below the peeling threshold,” *Embedded Systems and Applications*, 2022, arXiv:2202.05546.
- [15] W. Kuszmaul and M. Mitzenmacher, “Efficient d -ary cuckoo hashing at high load factors by bubbling up,” in *Proceedings of the 2025 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2025, pp. 3931–3952.